

TypeScript - పూర్తి తెలుగు గైడ్ (End-to-End, SDE2 & SSE)

ఈ document చదివిన తర్వాత TypeScript మళ్ళీ జీవితంలో మర్చిపోలేవు. ప్రతి concept కి ఒక real-life analogy, ఎప్పుడు/ఎందుకు వాడాలి, trade-offs, gotchas (సాధారణ తప్పులు), లోపల ఏం జరుగుతుంది (internals – structural typing, type erasure, inference algorithm, variance, compiler phases), మరియు interview దృష్టి – అన్నీ ఉంటాయి.

లక్ష్యం: JavaScript తెలిసిన engineer ని TypeScript లో absolute basics నుండి **type-level programming, declaration files, compiler performance, migration strategy** వరకు తీసుకెళ్లడం – **SDE2 & SSE (Senior Software Engineer) interview level**. "ఒకసారి చదివితే జీవితంలో మర్చిపోకూడదు."

ముఖ్యమైన హెచ్చరిక: TypeScript ని "JavaScript + కొన్ని annotations" అనుకుంటే మీరు దాని 20% మాత్రమే వాడుతున్నారు. నిజమైన శక్తి – **type system ని ఒక ప్రోగ్రామింగ్ భాషలా వాడటం** (conditional types, mapped types, inference). SSE interviews లో ఇక్కడే తేడా కనిపిస్తుంది.

Companion docs: [JavaScript_Telugu.md](#) (భాష పునాది – ఇది తప్పనిసరి), [React_Telugu.md](#) (React + TS), [OOPS_Telugu.md](#) (classes, SOLID, patterns), [SoftwareEngineering_Telugu.md](#).

ఇది **TypeScript 5.x** (5.0 – 5.7) ఆధారంగా రాయబడింది; పాత versions తో తేడాలు ఎక్కడ ముఖ్యమో అక్కడ చెప్పాను.

విషయ సూచిక (Table of Contents)

Part 1 – పునాదులు (Foundations)

- TypeScript అంటే ఏమిటి, ఎందుకు పుట్టింది (superset, type erasure, tsc, benefits vs costs)
- Setup & మొదటి project (tsc, Vite, tsx, watch mode, build pipeline)
- Basic Types (primitives, arrays, tuples, **any** / unknown / never / **void** , literal types)
- Type Inference & Annotations (ఎప్పుడు రాయాలి, contextual typing, widening)
- Union (**|**) & Intersection (**&**) Types
- Type Narrowing & Type Guards (typeof, instanceof, in, discriminated unions, predicates, exhaustiveness)
- Functions (optional/default/rest params, overloads, **this** , void quirk, function types)
- Objects, Interfaces & Index Signatures (readonly, optional, excess property checks)
- type** vs **interface** – పూర్తి పోలిక + declaration merging
- Enums – numeric, string, **const enum** , ఎందుకు నివారించాలి, ప్రత్యామ్నాయాలు
- Classes in TypeScript (modifiers, abstract, implements, parameter properties, **#private**)

Part 2 – Generics & Advanced Types (SSE core)

- Generics Deep (functions, interfaces, classes, constraints, defaults, inference rules)
- keyof , **typeof** , Indexed Access Types

14. Conditional Types & `infer`
15. Mapped Types (+ key remapping, modifiers `+/-readonly` , `+/-?`)
16. Template Literal Types
17. Built-in Utility Types (అన్నీ + వాటిని స్వయంగా implement చేయడం)
18. Type-Level Programming (recursive types, `DeepPartial`, tuple manipulation, arithmetic)
19. Declaration Files (`.d.ts`), Ambient Types, Module Augmentation, `DefinitelyTyped`
20. Modules, `import` type , `isolatedModules` , `verbatimModuleSyntax` , `namespaces`

Part 3 — Type System Semantics (interview differentiators)

21. Structural Typing vs Nominal Typing + Branded/Opaque Types
22. Variance — covariance, contravariance, bivarience, `strictFunctionTypes`
23. Widening, Narrowing, `as const` , `satisfies` operator
24. Assignability Rules & Excess Property Checking (fresh object literals)
25. `any` vs `unknown` vs `never` — top & bottom types deep

Part 4 — TypeScript in Practice

26. TypeScript + React (props, hooks, generic components, events — deep)
27. TypeScript + Node/Express (typed requests, env, errors, DI)
28. Async, Promises & Error Handling Typing (Result type, `catch` (e: `unknown`))
29. Runtime Validation (Zod) — type boundary vs runtime boundary
30. API Types at Scale (OpenAPI codegen, tRPC, GraphQL codegen)
31. Testing with TypeScript (Vitest/Jest, type tests, `expectTypeOf`)

Part 5 — Configuration, Tooling & Performance

32. `tsconfig.json` Deep Dive (ప్రతి ముఖ్య flag, strict family, module resolution)
33. Compiler Internals & Type-Checking Performance (project references, incremental, ఎందుకు నెమ్మది)
34. ESLint + typescript-eslint (ముఖ్య rules, type-aware linting)
35. Common Compiler Errors Decoded (TS2322, TS2345, TS7006, TS2589...)

Part 6 — Architecture, Migration & Interview

36. JavaScript → TypeScript Migration Strategy (incremental, JSDoc, `any` debt)
 37. Design Patterns in TypeScript (state machines, builder, DI, factory, branded types)
 38. Anti-Patterns & Best Practices Cheat-Sheet
 39. Rapid-Fire Interview Q&A (SDE2 + SSE, 80+ questions)
 40. Memory Tips — ఒకసారి చదివి గుర్తుపెట్టుకునే framework
-

Part 1 – పునాదులు (Foundations)

1. TypeScript అంటే ఏమిటి, ఎందుకు పుట్టింది

వివరణ

TypeScript = JavaScript + static types. ఇది Microsoft (Anders Hejlsberg – C# రూపకర్త) 2012 లో విడుదల చేసింది.

మూడు కీలక వాస్తవాలు – ఇవి అర్థమైతే TypeScript మొత్తం అర్థమవుతుంది:

- 1 TypeScript ఒక *superset* of JavaScript.** ప్రతి valid JavaScript file ఒక valid TypeScript file. అంటే `.js` ని `.ts` గా rename చేస్తే (చాలావరకు) పని చేస్తుంది.
- 2 Types compile time లో మాత్రమే ఉంటాయి – runtime లో ఉండవు.** దీన్ని **type erasure** అంటారు. `tsc` types ని పూర్తిగా తీసేసి plain JavaScript ఇస్తుంది.

```
// మనం రాసేది (input.ts)
interface User { id: number; name: string; }
function greet(user: User): string {
  return `Hello ${user.name}`;
}
const admin: User = { id: 1, name: "Surya" };
```

```
// tsc ఇచ్చేది (output.js) – types మాయం!
function greet(user) {
  return `Hello ${user.name}`;
}
const admin = { id: 1, name: "Surya" };
```

- 3 TypeScript ఒక *type checker*, కాదు *runtime validator*.** Server నుండి తప్పు data వస్తే TypeScript ఏమీ చేయలేదు – అది compile time లోనే పని ముగించింది.

```
const res = await fetch("/api/user");
const user: User = await res.json(); // ⚠ ఇది కేవలం "నమ్మకం" – API `{ }` పంపినా TS ఆపదు!
console.log(user.name.toUpperCase()); // ✨ runtime crash
```

Real-life Scenario

TypeScript = building plan approval + site engineer. మీరు ఇల్లు కట్టేముందు plan ని engineer చూసి "ఈ గోడ మీద రెండో అంతస్తు పెట్టలేవు, కూలుతుంది" అని చెప్తాడు (compile-time error). ఇది కట్టడం

మొదలుపెట్టకముందే తప్పులు పట్టుకుంటుంది – కూలిన తర్వాత కాదు (runtime error).

కానీ ఇల్లు కట్టేసాక ఆ **engineer అక్కడ ఉండడు** (type erasure). ఎవరైనా వచ్చి గోడ కొట్టేస్తే (API నుండి తప్పు data) అతను ఆపలేడు. అందుకే **entrance దగ్గర security** (runtime validation – Zod) విడిగా అవసరం.

TypeScript ఎందుకు — నిజమైన లాభాలు

లాభం	వివరణ
Bugs ముందే పట్టుబడతాయి	Typo, <code>undefined</code> access, తప్పు argument — production కి వెళ్ళకముందే
Autocomplete & IntelliSense	Editor కి మీ data shape తెలుస్తుంది → వేగవంతమైన development
Refactoring నిర్భయం	Property rename చేస్తే, వాడిన అన్ని చోట్లా error → ఏదీ మిస్ కాదు
Self-documenting code	<code>function pay(a, b, c)</code> కంటే <code>function pay(amount: Money, from: AccountId, idempotencyKey: string)</code>
Team scaling	కొత్త developer types చూసి contract అర్థం చేసుకోగలడు; comments అబద్ధం చెప్పొచ్చు, types చెప్పలేవు
Better libraries	Library API ని explore చేయడానికి docs కి వెళ్ళక్కర్లేదు

Microsoft/Airbnb studies: TypeScript ~15% production bugs ని ముందే పట్టుకుంటుందని అంచనా.

ఖర్చులు (honest view — interview లో ఇది చెప్తే maturity కనిపిస్తుంది)

- **Build step అవసరం** (JS లో లేదు) — compile time, tooling complexity.
- **Learning curve** — generics, conditional types నేర్చుకోవడానికి సమయం.
- **Type gymnastics ఉచ్చు** — కొందరు అనవసరంగా సంక్లిష్ట types రాసి codebase ని అర్థంకానిదిగా చేస్తారు.
- **False confidence** — `any`, `as` casts, unvalidated API data వల్ల "typed" అనుకుంటూ runtime crash.
- **Third-party types లేకపోవడం** — పాత libraries కి `.d.ts` లేకపోవచ్చు.

TypeScript ఎలా run అవుతుంది

```
.ts file
|
├─ tsc (type check + emit) —————> .js (types తీసేసి)
├─ esbuild/swc (transpile only) —> .js (≠ వేగం, కానీ type check చేయదు!)
└─ tsx / ts-node (dev లో నేరుగా run)
```

⚠ **ముఖ్యమైన విషయం:** Vite, esbuild, swc, Babel — ఇవి **type check చేయవు**, కేవలం types ని strip చేసి transpile చేస్తాయి (వేగం కోసం). అందుకే build pipeline లో **tsc --noEmit** విడిగా run చేయాలి (CI లో తప్పనిసరి). లేకపోతే type errors ఉన్నా build పాస్ అవుతుంది!

TypeScript vs ఇతర ఎంపికలు

	TypeScript	JSDoc + checkJs	Flow	plain JS
Build step	✅ అవసరం	❌ అవసరం లేదు	✅	❌
Ecosystem	అత్యధికం	TS types వాడొచ్చు	దాదాపు చనిపోయింది	—
Syntax	కొత్తది నేర్చుకోవాలి	comments లో types	TS లాంటిది	—
ఎప్పుడు	95% projects ✅	libraries, build-free setups	❌ కొత్తగా వాడొద్దు	prototypes

```
// JSDoc విధానం – build step లేకుండా type checking (Node built-in modules ఇలానే చేస్తాయి)
/**
 * @param {string} name
 * @param {number} [age]
 * @returns {{ id: string, name: string }}
 */
function createUser(name, age) { /* ... */ }
```

TypeScript యొక్క design philosophy (interview లో అడుగుతారు)

TypeScript team యొక్క ప్రకటిత లక్ష్యాలు:

1. **JavaScript తో సంపూర్ణ compatibility** – valid JS ఎప్పుడూ valid TS.
2. **Runtime behaviour మార్చకూడదు** – types కేవలం compile time (అందుకే **enum** ని ఇప్పుడు తప్పుగా భావిస్తారు – అది runtime code emit చేస్తుంది!).
3. **Soundness కంటే productivity** – TypeScript ఉద్దేశపూర్వకంగా "unsound". కొన్ని చోట్ల type-safe కాని కానీ ఆచరణాత్మకమైన నియమాలు (array covariance, **any**, type assertions) అనుమతిస్తుంది.

```
// ఉద్దేశపూర్వక unsoundness – ఇది compile అవుతుంది కానీ runtime లో crash!
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs; // ✅ TS అనుమతిస్తుంది (array covariance)
animals.push(new Cat()); // ✅ compile OK – కానీ dogs లో ఇప్పుడు ఒక Cat ఉంది! ✖
```

Interview answer: "TypeScript deliberately trades soundness for pragmatism. A fully sound type system would reject too much real-world JavaScript. Knowing *where* it's unsound – array covariance, **any**, assertions, method bivarience – is what separates a user from an expert."

Gotchas (సాధారణ తప్పులు)

- "TypeScript runtime లో type check చేస్తుంది" అనుకోవడం – ❌ చేయదు. API data ని ఎప్పుడూ validate చేయాలి.
- **any** వాడి type safety ని చంపడం – **any** ఒక "type checking off" switch.
- **Types ని అలంకరణ అనుకోవడం** – అవి contracts; సరిగ్గా design చేస్తే bugs impossible అవుతాయి.
- **tsc --noEmit ని CI లో run చేయకపోవడం** (Vite/esbuild projects లో) – type errors production కి వెళ్తాయి.

- **strict: false** — TypeScript యొక్క సగం విలువ పోతుంది.

Key Points

- TypeScript = JS **superset** + **compile-time** static types.
- **Type erasure** — runtime లో types ఉండవు (enum, decorators మినహా).
- Bundlers types ని strip చేస్తాయి, **check చేయవు** → `tsc --noEmit` తప్పనిసరి.
- **Runtime validation విడిగా అవసరం** (Zod) — types API ని నమ్మలేవు.
- TypeScript **ఉద్దేశపూర్వకంగా unsound** — productivity కోసం.

Interview దృష్టి

- "TypeScript ఎందుకు?" → early bugs, refactoring safety, DX, self-documentation, team scale — trade-offs కూడా చెప్పాలి.
- "Types runtime లో ఉంటాయా?" → లేదు, type erasure; **enum** / **decorators** / **namespace** మాత్రమే code emit చేస్తాయి.
- "TypeScript sound ఆ?" → కాదు, ఉద్దేశపూర్వకంగా; ఉదాహరణలు చెప్పగలగాలి.

2. Setup & మొదటి Project

వివరణ

```
# 1. Minimal setup
npm init -y
npm i -D typescript @types/node
npx tsc --init # tsconfig.json సృష్టిస్తుంది

# 2. Compile
npx tsc # tsconfig ప్రకారం compile
npx tsc --watch # file మారిన ప్రతిసారి
npx tsc --noEmit # ✅ type check మాత్రమే (CI లో ఇదే)

# 3. Dev లో నేరుగా run (build లేకుండా)
npm i -D tsx
npx tsx src/index.ts
npx tsx watch src/index.ts

# Node 22+ లో native (experimental type stripping)
node --experimental-strip-types src/index.ts
```

మొదటి tsconfig (2026 recommended baseline)

```
{
  "compilerOptions": {
    /* భాష & environment */
    "target": "ES2022", // ఏ JS version కి compile చేయాలి
```

```

"lib": ["ES2022", "DOM", "DOM.Iterable"], // ఏ built-in APIs అందుబాటులో ఉన్నాయి
"module": "ESNext", // ఏ module system emit చేయాలి
"moduleResolution": "bundler", // Vite/webpack కి; Node కి "NodeNext"

/* Strictness – ఇవన్నీ ON ఉండాలి */
"strict": true, // 🔑 అన్ని strict flags
"noUncheckedIndexedAccess": true, // 🔑 arr[i] → T | undefined (strict లో లేదు!)
"noImplicitOverride": true,
"noFallthroughCasesInSwitch": true,
"exactOptionalPropertyTypes": true, // { a?: string } కి undefined explicit గా ఇవ్వలేం

/* Emit */
"outDir": "./dist",
"rootDir": "./src",
"sourceMap": true,
"declaration": true, // .d.ts generate (libraries కి)
"noEmit": true, // bundler emit చేస్తుంటే tsc emit వద్దు

/* Interop */
"esModuleInterop": true,
"forceConsistentCasingInFileNames": true,
"skipLibCheck": true, // node_modules .d.ts check skip (build వేగం)
"resolveJsonModule": true,
"isolatedModules": true, // ప్రతి file ని స్వతంత్రంగా transpile చేయగలగాలి

/* Paths */
"baseUrl": ".",
"paths": { "@/*": ["./src/*"] } // import x from "@/utils/x"
},
"include": ["src/**/*"],
"exclude": ["node_modules", "dist"]
}

```

strict: true ఏం enable చేస్తుంది: noImplicitAny, strictNullChecks, strictFunctionTypes, strictBindCallApply, strictPropertyInitialization, noImplicitThis, useUnknownInCatchVariables, alwaysStrict. (Topic 32 లో ఒక్కొక్కటి deep.)

Real-life Scenario

strict: false = seatbelt లేకుండా కారు నడవడం. కారు నడుస్తుంది, వేగంగానే వెళ్తుంది. కానీ ప్రమాదం జరిగినప్పుడు (undefined access) మీకు ఏ రక్షణా ఉండదు. **కొత్త project లో strict: true తప్పనిసరి –** తర్వాత ఆన్ చేయడం 10 రెట్లు కష్టం.

Build strategies – ఏది ఎప్పుడు

Strategy	Type check	వేగం	ఎప్పుడు
tsc only	✓	నెమ్మది	Node backends, libraries
Vite/esbuild + tsc --noEmit	✓ (ఏడిగా)	⚡ అత్యంత వేగం	Frontend apps ✓
swc/Babel + tsc --noEmit	✓ (ఏడిగా)	⚡	Next.js, big monorepos
tsup / unbuild	✓ (dts)	వేగం	npm packages (dual ESM/CJS + .d.ts)

```
// package.json – ప్రామాణిక scripts
{
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc --noEmit && vite build",    // ⚡ type check ముందు
    "typecheck": "tsc --noEmit",
    "typecheck:watch": "tsc --noEmit --watch",
    "lint": "eslint .",
    "test": "vitest"
  }
}
```

Project structure

```
project/
  src/
    index.ts
    types/          # shared types (global.d.ts, api.ts)
    features/
    lib/
  tsconfig.json      # base config
  tsconfig.build.json # build-specific (tests exclude చేయడం)
  dist/
```

@types packages – ఎలా పని చేస్తాయి

```
npm i lodash          # library (JS)
npm i -D @types/lodash # దాని type definitions (DefinitelyTyped నుండి)
```

- Library లోనే types ఉంటే (`package.json` లో `"types": "./dist/index.d.ts"`) → `@types` అవసరం లేదు (ఉదా. axios, zod).
- లేకపోతే → `@types/<name>` (DefinitelyTyped community project).
- అదీ లేకపోతే → మనమే `.d.ts` రాయాలి (Topic 19).

Gotchas (సాధారణ తప్పులు)

- `strict: false` తో మొదలుపెట్టడం – తర్వాత migrate చేయడం నరకం.

- **tsc --noEmit** ని CI లో మర్చిపోవడం – Vite build type errors ని పట్టించుకోదు.
- **skipLibCheck: false** – node_modules లోని అన్ని **.d.ts** check → build చాలా నెమ్మది (సాధారణంగా **true** ఉండాలి).
- **target: ES5** ని అనవసరంగా వాడటం – output పెద్దది, నెమ్మది; ఆధునిక browsers/Node కి ES2020+ .
- **noUncheckedIndexedAccess off** – **strict** లో ఇది **లేదు**, కానీ ఇది real bugs ఆపుతుంది (Topic 32).
- **paths** ని **tsc** లో మాత్రమే **set చేయడం** – bundler/Node కి కూడా చెప్పాలి (vite-tsconfig-paths , tsconfig-paths).

Key Points

- **tsc --noEmit** = type check; bundlers types ని strip మాత్రమే చేస్తాయి.
- **strict: true + noUncheckedIndexedAccess** = కొత్త projects కి baseline.
- Dev లో tsx ; production build లో bundler + separate typecheck.
- **@types/*** = DefinitelyTyped; ఆధునిక libraries తమ types తామే ఇస్తాయి.

Interview దృష్టి

- "Vite లో type errors ఎందుకు build ని ఆపవు?" → esbuild transpile-only; tsc --noEmit విడిగా.
- "strict flags ఏమిటి?" → 8 flags జాబితా + strictNullChecks ప్రాముఖ్యత.

3. Basic Types

వివరణ

```
// Primitives (lowercase! String/Number/Boolean ✗ – అవి wrapper objects)
let name: string = "Surya";
let age: number = 30; // int/float తేడా లేదు (JS లానే)
let isActive: boolean = true;
let big: bigint = 100n;
let sym: symbol = Symbol("key");
let nothing: null = null;
let notDefined: undefined = undefined;

// Arrays – రెండు రూపాలు (ఒకటే అర్థం)
let nums: number[] = [1, 2, 3];
let strs: Array<string> = ["a", "b"];
let matrix: number[][] = [[1, 2], [3, 4]];
let mixed: (string | number)[] = [1, "a"]; // ⚠️ brackets ముఖ్యం
let arrOfUnion: string | number[] = 5; // ఇది "string లేదా number[]" – వేరే అర్థం!

// Tuple – fixed length + fixed types per position
let point: [number, number] = [10, 20];
let entry: [string, number, boolean?] = ["a", 1]; // optional element
let named: {x: number, y: number} = [1, 2]; // labeled (docs కి)
let rest: [string, ...number[]] = ["a", 1, 2, 3]; // rest element
```

```
// Object
let user: { id: number; name: string; email?: string } = { id: 1, name: "S" };

// Function
let add: (a: number, b: number) => number = (a, b) => a + b;
```

Real-life Scenario

Type = బాక్స్ మీద label. "పప్పు" అని label ఉన్న డబ్బాలో బియ్యం పెట్టలేరు (type error). **Tuple = ప్రత్యేకంగా విభజించిన lunch box** – మొదటి అరలో అన్నం, రెండో అరలో కూర – వరుస, పరిమాణం రెండూ fixed. **Array = ఒకే రకం వస్తువుల సంచి** – ఎన్నయినా పట్టవచ్చు కానీ అన్నీ ఒకే రకం.

any – type checking ని ఆపే switch

```
let x: any = 5;
x = "string";           // ✓
x.foo.bar.baz();        // ✓ compile OK – ✖ runtime crash
x();                    // ✓ compile OK

// any అంటున్నాదిలా వ్యాపిస్తుంది
function process(data: any) { return data.value; } // return type కూడా any
const result = process(x);                        // result: any
result.anything.goes();                          // ✓ TS మౌనం
```

any ఎప్పుడు ఆమోదయోగ్యం: migration లో తాత్కాలికంగా, third-party untyped library, నిజంగా dynamic data (అప్పుడు కూడా unknown మేలు).

unknown – సురక్షితమైన any

```
let value: unknown = fetchData();

value.foo;           // ✖ Error: 'value' is of type 'unknown'
value();             // ✖ Error
const s: string = value; // ✖ Error

// ✓ ముందు narrow చేయాలి
if (typeof value === "string") {
  console.log(value.toUpperCase()); // ✓ ఇక్కడ value: string
}
if (value instanceof Date) { value.getTime(); } // ✓
```

నియమం: any బదులు ఎప్పుడూ unknown వాడండి. unknown = "ఇది ఏమిటో నాకు తెలియదు, కాబట్టి వాడేముందు నిరూపించు." any = "ఇది ఏమైనా కావచ్చు, నన్ను ఇబ్బంది పెట్టకు."

void VS undefined VS never

```
// void – function ఏమీ return చేయదు
function log(msg: string): void { console.log(msg); }

// undefined – నిజంగా undefined value
function find(): string | undefined { return undefined; }

// never – ఈ function ఎప్పుడూ *పూర్తి కాదు* (throw లేదా infinite loop)
function fail(msg: string): never { throw new Error(msg); }
function loop(): never { while (true) {} }

// never – impossible types
type Impossible = string & number; // never
```

void యొక్క ప్రసిద్ధ quirk:

```
type Callback = () => void;
const cb: Callback = () => 42; // ✅ అనుమతిస్తుంది! (return value ignore అవుతుంది)
const r = cb(); // r: void – value వాడలేం

// ఎందుకు: array.forEach(x => arr.push(x)) లాంటివి పని చేయాలని.
// push() number return చేస్తుంది కానీ forEach void అశిస్తుంది.
```

Literal types

```
let x: "hello" = "hello"; // "hello" మాత్రమే!
x = "world"; // ❌ Error

type Direction = "up" | "down" | "left" | "right";
type Status = 200 | 400 | 404 | 500;
type Flag = true;

function move(dir: Direction) {}
move("up"); // ✅
move("north"); // ❌ Error – typo compile time లోనే పట్టుబడింది ⚡
```

ఇదే TypeScript యొక్క గొప్ప ఆచరణాత్మక శక్తి – **string** బదులు literal union వాడితే typos అసాధ్యం.

let vs const inference (widening)

```
let a = "hello"; // type: string (let → widen అవుతుంది, మారొచ్చు కాబట్టి)
const b = "hello"; // type: "hello" (const → literal type, ఎప్పటికీ మారదు)

let n = 5; // number
const m = 5; // 5

const obj = { name: "x" }; // { name: string } ⚠️ properties widen అవుతాయి
const obj2 = { name: "x" } as const; // { readonly name: "x" }
```

Type aliases

```
type ID = string | number;
type Point = { x: number; y: number };
type Handler = (e: Event) => void;
type Dict = Record<string, number>;
type Nullable<T> = T | null;
```

Gotchas (సాధారణ తప్పులు)

- **String** , **Number** , **Boolean** (capital) వాడటం – అవి wrapper object types; ఎప్పుడూ lowercase వాడాలి.
- **any** widespread – type safety పోతుంది; unknown + narrowing వాడాలి.
- **string | number[]** vs **(string | number)[]** గందరగోళం.
- **Tuple** ని **array** అనుకోవడం – tuple కి **push()** చేయొచ్చు (TS ఆపదు!):

```
const t: [number, number] = [1, 2];
t.push(3); // ✅ compile OK 🤖 – tuple కి readonly వాడితే మేలు
const safe: readonly [number, number] = [1, 2];
```

- **noUncheckedIndexedAccess** లేకపోవడం → **arr[10]** కి **T** type వస్తుంది (నిజానికి **undefined** కావచ్చు).
- **object** vs **Object** vs **{}** – **{}** దాదాపు అన్నిటినీ అంగీకరిస్తుంది (null/undefined తప్ప), **object** = non-primitive, **Object** = wrapper. ఏదీ నిజంగా ఉపయోగకరం కాదు; నిర్దిష్ట shape లేదా **Record<string, unknown>** వాడాలి.

Key Points

- Primitives lowercase; **any** = escape hatch, **unknown** = safe alternative.
- **Literal union types** = typo-proof APIs (TS యొక్క అత్యంత ఆచరణాత్మక feature).
- **void** = ignored return; **never** = ఎప్పుడూ పూర్తికాని/impossible.
- **const** → literal type; **let** → widened type.
- **Tuple** = fixed length/positions (కానీ **push** నుండి రక్షణ లేదు → **readonly**).

Interview దృష్టి

- "**any** vs **unknown** ?" → **unknown** వాడేముందు narrowing తప్పనిసరి; **any** అన్ని checks ని ఆపేస్తుంది.
- "**never** ఎప్పుడు వస్తుంది?" → throw/infinite functions, impossible intersections, exhaustiveness checks లో.
- "**void** return type ఉన్న callback value return చేయొచ్చా?" → అవును (ఉద్దేశపూర్వక design), కానీ ఆ value వాడలేం.

4. Type Inference & Annotations

వివరణ

TypeScript యొక్క అతిపెద్ద బలం – మీరు **types** రాయనవసరం లేకపోవడం. Compiler చాలావరకు తనే కనిపెడుతుంది (inference).

```
// ✗ అనవసర annotations – TS ఇవి తనే తెలుసుకుంటుంది
const name: string = "Surya";
const nums: number[] = [1, 2, 3];
const doubled: number[] = nums.map((n: number): number => n * 2);

// ✔ Inference ని నమ్మండి
const name = "Surya"; // "Surya"
const nums = [1, 2, 3]; // number[]
const doubled = nums.map((n) => n * 2); // number[] – n కూడా number గా inferred
```

ఎప్పుడు annotate చేయాలి (బంగారు నియమాలు)

```
// ✔ 1. Function parameters – ఎప్పుడూ (inference ఇక్కడ పని చేయదు)
function greet(name: string, age: number) {}

// ✔ 2. Function return types – public APIs/exported functions కి
export function getUser(id: string): Promise<User> { /* ... */ }
// ఎందుకు: implementation లో పొరపాటున తప్పు type return చేస్తే *ఇక్కడే* error వస్తుంది,
// వాడిన చోట కాదు → error message స్పష్టంగా ఉంటుంది

// ✔ 3. ఖాళీ containers
const items: string[] = []; // లేకపోతే never[]
const map = new Map<string, User>(); // generic argument

// ✔ 4. Variable declaration లో value వెంటనే ఇవ్వకపోతే
let user: User | null = null;

// ✔ 5. Object literal ని ఒక contract కి కట్టుబడేలా చేయాలంటే
const config: AppConfig = { port: 3000 }; // లేకపోతే typo silent గా పోతుంది

// ✗ 6. Local variables కి annotation సాధారణంగా అనవసరం
const total = items.reduce((s, i) => s + i.price, 0); // number – inferred
```

Real-life Scenario

Inference = అనుభవజ్ఞుడైన assistant. మీరు "టీ తీసుకురా" అంటే – ఎంత చక్కెర, ఏ కప్పు అని అడగడు; సందర్భం చూసి తెలుసుకుంటాడు (inference). కానీ **అతిథి కోసం అయితే** (public API) స్పష్టంగా చెప్పాలి – లేకపోతే అతను ఊహించినది మీరు కోరుకున్నది కాకపోవచ్చు.

Contextual typing – TS parameters ని ఎలా ఊహిస్తుంది

```
// TS "వెనుక నుండి ముందుకు" కూడా ఆలోచిస్తుంది
const names = ["a", "b"];
names.forEach((name) => { // name: string – array type నుండి inferred ✓
  console.log(name.toUpperCase());
});

window.addEventListener("click", (e) => {
  e.clientX; // e: MouseEvent – event name నుండి inferred! ✓
});

type Handler = (id: number, name: string) => void;
const h: Handler = (id, name) => {}; // ✓ ఇక్కడ annotations అవసరం లేదు

// ⚠️ కానీ ఇక్కడ కాదు (context లేదు)
const h2 = (id, name) => {}; // ✗ TS7006: Parameter 'id' implicitly has an 'any' type
```

noImplicitAny – ఏం చేస్తుంది

```
// strict: false అయితే – ఇది silent గా పోతుంది (ప్రమాదం!)
function process(data) { // data: any (implicit)
  return data.whatever.deeply.nested; // ✓ compile OK, ✗ runtime
}

// strict: true – TS7006 error → annotate చేయమని బలవంతం
function process(data: UserData) { }
```

Best common type & widening

```
const arr = [1, 2, 3]; // number[]
const mixed = [1, "a"]; // (string | number)[]
const objs = [{ a: 1 }, { b: 2 }]; // ({ a: number; b?: undefined } | { a?: undefined; b: number })[]

// Literal widening
const s = "hello"; // "hello"
let s2 = s; // string ← widened!
const obj = { x: "hello" }; // { x: string } ← property widen అవుతుంది

// as const – widening ఆపడం
const config = { url: "/api", method: "GET" } as const;
// { readonly url: "/api"; readonly method: "GET" }

function req(m: "GET" | "POST") {}
const opts = { method: "GET" };
req(opts.method); // ✗ Error! opts.method: string
const opts2 = { method: "GET" } as const;
req(opts2.method); // ✓
```

Return type inference – ఎప్పుడు explicit ఇవ్వాలి

```
// ❌ Inferred return – bug ఇక్కడ కాదు, వాడిన చోట కనిపిస్తుంది
export function getUser(id: string) {
  if (!id) return null; // return type: User | null (కావాలనుకున్నది User)
  return db.find(id);
}

// ✅ Explicit – తప్పు ఇక్కడే పట్టుబడుతుంది
export function getUser(id: string): User {
  if (!id) return null; // ❌ Error ఇక్కడే! ⚡
  return db.find(id);
}
```

Team నియమం: *exported/public functions* కి return type explicit; *internal helpers* కి inference. ఇది build performance కి కూడా సహాయపడుతుంది (compiler infer చేయాల్సిన పని తగ్గుతుంది).

Gotchas (సాధారణ తప్పులు)

- అన్నిటికీ annotation రాయడం – code noise; inference ని నమ్మాలి.
- `const arr = []` → `never[]` → ఏమీ push చేయలేం. `const arr: string[] = []`.
- Object literal widening వల్ల literal union mismatch → `as const` లేదా explicit type.
- Callback params కి `any` implicit – context లేకపోతే annotate చేయాలి.
- Return type ఇవ్వకపోవడం వల్ల error message తప్పు చోట కనిపించడం.
- `let` తో literal type పొగొట్టుకోవడం – `const` వాడాలి.

Key Points

- Inference ని నమ్మండి; **parameters + public return types + empty containers** కి మాత్రమే annotate.
- **Contextual typing** – callbacks లో TS parameters ని ఊహిస్తుంది.
- `const` → literal, `let` → widened; `as const` widening ని ఆపుతుంది.
- Explicit return types = errors సర్దుకొనే చోట + వేగవంతమైన compilation.

Interview దృష్టి

- "ఎప్పుడు type annotation రాయాలి?" → parameters, public APIs, empty collections, deferred initialization.
- "`as const` ఏం చేస్తుంది?" → literal types + deep readonly; widening ఆపుతుంది.

5. Union & Intersection Types

వివరణ

```
// Union (|) - "ఇది లేదా అది" (OR)
type ID = string | number;
type Result = Success | Error;
type Padding = number | string | undefined;

// Intersection (&) - "ఇది మరియు అది" (AND)
type Person = { name: string };
type Employee = { empId: string };
type Staff = Person & Employee;           // { name: string; empId: string } - రెండూ ఉండాలి
```

Real-life Scenario

Union = "టీ లేదా కాఫీ" – రెండింటిలో ఏదో ఒకటి వస్తుంది. కాబట్టి మీరు దేనికైనా సిద్ధంగా ఉండాలి – "టీ అయితే ఇలా, కాఫీ అయితే అలా" (narrowing). రెండింటిలో **ఉమ్మడిగా ఉన్నది** (వేడిగా ఉండటం) మాత్రమే నిర్ధారించుకోకుండా వాడొచ్చు.

Intersection = "టీ + బిస్కెట్ combo" – రెండూ కలిపి వస్తాయి. రెండింటి లక్షణాలూ ఉంటాయి.

Union – వాడేముందు narrow చేయాలి

```
function printId(id: string | number) {
  id.toUpperCase();           // ❌ Error - number కి toUpperCase లేదు

  // ✅ Union లో అన్ని members కి ఉమ్మడిగా ఉన్నవి మాత్రమే నేరుగా వాడొచ్చు
  console.log(id.toString()); // ✅ రెండింటికీ ఉంది

  if (typeof id === "string") {
    console.log(id.toUpperCase()); // ✅ narrowed to string
  } else {
    console.log(id.toFixed(2));    // ✅ narrowed to number
  }
}
```

Discriminated (tagged) union – TypeScript యొక్క అత్యంత శక్తివంతమైన pattern

```
type LoadingState =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: User[] }
  | { status: "error"; error: Error; retryable: boolean };
//   ↑ "status" = discriminant (common literal property)

function render(state: LoadingState) {
  switch (state.status) {
    case "idle":    return "Ready";
  }
}
```

```

    case "loading": return "Loading...";
    case "success": return state.data.length; // ✅ data ఇక్కడ మాత్రమే ఉంది
    case "error": return state.error.message; // ✅ error ఇక్కడ మాత్రమే
  }
}

```

// ❌ ఇలా చేస్తే impossible states సాధ్యం
 type Bad = { status: string; data?: User[]; error?: Error };
 // status: "success" కానీ data undefined – ఇది సాధ్యం! bug wait చేస్తోంది

ఇదే "make illegal states unrepresentable" సూత్రం – TypeScript లో అత్యంత విలువైన నైపుణ్యం. Interview లో దీన్ని ప్రదర్శిస్తే బలమైన signal.

Intersection – ఎప్పుడు ఉపయోగం

```

// 1. Props composition (React లో సాధారణం)
type ButtonProps = BaseProps & { variant: "primary" | "danger" } & AriaProps;

// 2. Mixins
type Timestamped = { createdAt: Date; updatedAt: Date };
type SoftDeletable = { deletedAt: Date | null };
type Entity = { id: string };
type User = Entity & Timestamped & SoftDeletable & { email: string };

// 3. Function overload-ish
type Callable = { (x: number): string } & { description: string };

```

Intersection యొక్క ఆశ్చర్యకరమైన ప్రవర్తనలు

```

// (a) Conflicting primitive properties → never
type A = { x: string };
type B = { x: number };
type C = A & B; // { x: never } – ఏ value ఇవ్వలేం!
const c: C = { x: "?" }; // ❌ Error

// (b) Primitives intersection → never
type Impossible = string & number; // never

// (c) Union of objects vs intersection – గందరగోళం
type U = { a: string } | { b: number }; // "a ఉంది లేదా b ఉంది"
type I = { a: string } & { b: number }; // "రెండూ ఉన్నాయి"
// ⚠️ objects లో | = fewer guarantees, & = more properties (సెట్ల intuition కి తలకిందులు!)

```

గుర్తుంచుకోండి: Types = values యొక్క సమూహాలు (sets). $A \mid B$ = రెండు సెట్ల **union** (ఎక్కువ values, తక్కువ properties హామీ). $A \& B$ = **intersection** (తక్కువ values, ఎక్కువ properties). Objects లో ఇది తలకిందులుగా కనిపిస్తుంది కానీ నిజానికి సరైనదే.

Union distribution over utility types

```
type Shape = Circle | Square;
type PartialShape = Partial<Shape>; // Partial<Circle> | Partial<Square> (distributed)

// Union members ని extract చేయడం
type SuccessState = Extract<LoadingState, { status: "success" }>;
type NonError = Exclude<LoadingState, { status: "error" }>;
```

Gotchas (సాధారణ తప్పులు)

- **Union ని narrow చేయకుండా వాడటం** – ఉమ్మడి members మాత్రమే అందుబాటులో.
- **Discriminant లేని union** → narrowing కష్టం, `in` operator hacks అవసరం.
- **Optional properties తో "union" simulate చేయడం** ({ `data?`, `error?` }) – impossible states అనుమతిస్తుంది.
- **Intersection తో conflicting types** → `silent never` → గందరగోళ error messages.
- **| ని "and" గా చదవడం** – `"a" | "b"` అంటే `"a లేదా b"`, రెండూ కాదు.
- **Big unions తో compile నెమ్మది** – 1000+ member unions type checking ని పేల్చేస్తాయి.

Key Points

- Union = OR (narrow చేయాలి); Intersection = AND (properties కలుస్తాయి).
- **Discriminated unions** = impossible states ని అసాధ్యం చేసే ప్రధాన సాధనం.
- Types = sets; `|` = set union, `&` = set intersection.
- Conflicting intersections → `never` .

Interview దృష్టి

- *"Discriminated union అంటే?"* → common literal tag + narrowing + exhaustive switch – ఉదాహరణతో చెప్పాలి.
- *"{a: string} & {a: number} ఏమవుతుంది?"* → { `a: never` } .
- *"State ని ఎలా model చేస్తావు?"* → discriminated union, boolean flags కాదు.

6. Type Narrowing & Type Guards

వివరణ

Narrowing = TypeScript ఒక variable యొక్క type ని కొన్ని code paths లో మరింత నిర్దిష్టంగా చేసే ప్రక్రియ. దీనికి compiler వాడేదాన్ని **control flow analysis** అంటారు.

1. typeof guard

```
function pad(value: string | number, padding: string | number) {
  if (typeof value === "number") {
    value.toFixed(2); // number
  } else {
    value.trim(); // string
  }
}
// ⚠️ typeof null === "object" (JavaScript యొక్క ప్రసిద్ధ bug - TS దీన్ని తెలుసు)
function f(x: string[] | null) {
  if (typeof x === "object") { /* x: string[] | null - ఇంకా null కావచ్చు! */ }
  if (x !== null) { x.length; } // ✅ సరైనది
}
```

2. Truthiness narrowing

```
function print(s: string | null | undefined) {
  if (s) { s.toUpperCase(); } // ✅ null, undefined తొలగించబడ్డాయి
  // ⚠️ కానీ "" (empty string) కూడా falsy → అది కూడా filter అవుతుంది!
}
// ✅ నిర్దిష్టంగా ఉండటం మేలు
if (s !== null) { } // null మరియు undefined రెండూ (loose != ఉద్దేశపూర్వకం)
if (s !== undefined) { }
```

3. Equality narrowing

```
function compare(a: string | number, b: string | boolean) {
  if (a === b) {
    a.toUpperCase(); // ✅ రెండూ string అయితేనే equal కాగలవు!
    b.toUpperCase(); // ✅ TS దీన్ని deduce చేసింది
  }
}
```

4. in operator

```
type Fish = { swim: () => void };
type Bird = { fly: () => void };

function move(animal: Fish | Bird) {
  if ("swim" in animal) { animal.swim(); } // Fish
  else { animal.fly(); } // Bird
}
```

5. instanceof

```
function logValue(x: Date | string) {
  if (x instanceof Date) { x.toISOString(); }
  else { x.toUpperCase(); }
```

```

}

// Error handling లో అత్యంత ఉపయోగం
try { /* ... */ }
catch (e) { // e: unknown (useUnknownInCatchVariables)
  if (e instanceof ApiError) { console.log(e.status); }
  else if (e instanceof Error) { console.log(e.message); }
  else { console.log("Unknown error", e); }
}

```

6. Discriminated union narrowing (అత్యుత్తమం)

```

type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "square"; side: number }
  | { kind: "rect"; width: number; height: number };

function area(s: Shape): number {
  switch (s.kind) {
    case "circle": return Math.PI * s.radius ** 2;
    case "square": return s.side ** 2;
    case "rect":   return s.width * s.height;
  }
}

```

7. Custom type guards (type predicates) — x is T

```

// Return type `arg is Type` – ఇదే type guard ని ప్రత్యేకం చేసింది
function isString(value: unknown): value is string {
  return typeof value === "string";
}

function isUser(obj: unknown): obj is User {
  return (
    typeof obj === "object" && obj !== null &&
    "id" in obj && typeof (obj as User).id === "string" &&
    "email" in obj && typeof (obj as User).email === "string"
  );
}

const data: unknown = await res.json();
if (isUser(data)) {
  console.log(data.email); // ✅ data: User
}

// Array filtering లో అత్యంత ఉపయోగం
const values: (string | null)[] = ["a", null, "b"];
const clean = values.filter((v): v is string => v !== null); // string[] ✅
// ❌ v => v !== null మాత్రమే రూప్యం → (string | null)[] (TS 5.5 కి ముందు)
// ✅ TS 5.5+ లో inferred type predicates – ఇది automatic గా పని చేస్తుంది!

```

⚠️ **Type predicates unsafe కావచ్చు** – compiler మీరు చెప్పింది నమ్ముతుంది. `return true` అని రాసినా అంగీకరిస్తుంది. అందుకే **Zod** వంటి validation libraries మేలు (Topic 29).

8. Assertion functions — `asserts x is T`

```
function assertIsString(value: unknown): asserts value is string {
  if (typeof value !== "string") throw new TypeError("Expected string");
}

function assertDefined<T>(value: T): asserts value is NonNullable<T> {
  if (value == null) throw new Error("Value must be defined");
}

const x: unknown = getData();
assertIsString(x);
x.toUpperCase();           // ✅ ఈ line నుండి x: string

// ⚠️ assertion functions కి explicit type annotation తప్పనిసరి
const assertFn = (v: unknown): asserts v is string => {}; // ❌ Error
// ✅ function declaration లేదా explicit typed const వాడాలి
```

9. Exhaustiveness checking (`never`) — production-critical

```
function area(shape: Shape): number {
  switch (shape.kind) {
    case "circle": return Math.PI * shape.radius ** 2;
    case "square": return shape.side ** 2;
    default: {
      const _exhaustive: never = shape;           // 🔑
      throw new Error(`Unhandled shape: ${JSON.stringify(shape)}`);
    }
  }
}

// ఇప్పుడు Shape కి కొత్త member ("rect") add చేస్తే –
// `shape` type "rect" అవుతుంది, `never` కి assign కాదు → **compile error** ⚡
// అంటే కొత్త case handle చేయడం మర్చిపోవడం అసాధ్యం!
```

Narrowing ఎప్పుడు "పోతుంది" (సాధారణ ఉచ్చు)

```
function process(user: User | null) {
  if (!user) return;
  user.name;           // ✅ narrowed

  const cb = () => { user.name; }; // ⚠️ closure లో TS narrowing ని కోల్పోవచ్చు
                                // (let variable అయితే – reassign కావచ్చు కాబట్టి)
}

// Object property narrowing – mutation తర్వాత reset
if (obj.data) {
  doSomething();       // ఈ call లోపల obj.data ని null చేయొచ్చు
}
```

```

    obj.data.value; // ⚠️ TS ఇంకా narrowed అనుకుంటుంది (unsound!)
  }
  // ✅ local variable లోకి తీసుకోవడం మేలు
  const data = obj.data;
  if (data) { data.value; } // ✅ సురక్షితం

```

Gotchas (సాధారణ తప్పులు)

- `typeof null === "object"` ఉచ్చు.
- Truthiness narrowing తో `0 / ""` పోగొట్టుకోవడం — `!== undefined` వాడాలి.
- Type predicate లో తప్పు logic — compiler నమ్ముతుంది, runtime crash.
- `as` casting ని narrowing అనుకోవడం — `as` = "నన్ను నమ్ము", ఏ check లేదు.
- Exhaustiveness check లేకపోవడం → కొత్త union member silent గా unhandled.
- `let` + closure లో narrowing పోవడం — `const` వాడాలి.
- Optional chaining తర్వాత narrowing — `a?.b` narrowing ఇవ్వదు, `if (a?.b)` ఇస్తుంది.

Key Points

- Narrowing = control flow analysis; guards: `typeof`, `in`, `instanceof`, equality, truthiness, discriminants.
- Type predicate (`x is T`) = custom guard (కానీ unsafe కావచ్చు).
- `asserts x is T` = throw చేసే guard.
- `never` exhaustiveness check = కొత్త union members ని మర్చిపోవడం అసాధ్యం చేస్తుంది.
- Narrowing local `const` లో స్థిరంగా ఉంటుంది; properties/closures లో పోవచ్చు.

Interview దృష్టి

- "Type guard ఎలా రాస్తావు?" → `value is T` predicate + ఉదాహరణ + దాని unsafety.
- "Exhaustiveness ఎలా enforce చేస్తావు?" → `never` assignment in default case.
- "unknown ని ఎలా వాడతావు?" → catch blocks, API responses + narrowing/validation.

7. Functions in TypeScript

వివరణ

```

// Function declaration
function add(a: number, b: number): number { return a + b; }

// Function expression / arrow
const mul = (a: number, b: number): number => a * b;

// Function type
type BinaryOp = (a: number, b: number) => number;
const div: BinaryOp = (a, b) => a / b; // params inferred ✅

```

```
// Optional, default, rest
function greet(name: string, greeting = "Hello", title?: string, ...tags: string[]) {}
//           ↑ default (type inferred) ↑ optional ↑ rest → string[]
// ⚠ optional params ఎప్పుడూ required params తర్వాత మాత్రమే

// Object destructuring params
function create({ name, age = 18 }: { name: string; age?: number }) {}
```

Real-life Scenario

Function signature = restaurant menu లో వివరణ. "Chicken Biryani – 2 pieces, serves 1" – మీరు order చేసేముందే ఏం వస్తుందో తెలుసు (parameter/return types). **Overloads = ఒకే వంటకం వేర్వేరు sizes లో** – half plate అడిగితే ఒక rate, full plate అడిగితే ఇంకో rate; కానీ **వంటగదిలో ఒకటే recipe** (ఒకే implementation).

Function Overloads

```
// Overload signatures (బయటికి కనిపించేవి)
function parse(input: string): object;
function parse(input: string, reviver: Function): object;
function parse(input: Buffer): object;
// Implementation signature (బయటికి కనిపించదు – union ని handle చేయాలి)
function parse(input: string | Buffer, reviver?: Function): object {
    const text = typeof input === "string" ? input : input.toString("utf8");
    return JSON.parse(text, reviver as any);
}

parse("{}"); // ✔
parse(Buffer.from("{}")); // ✔
parse(123); // ✖ Error
```

వాస్తవిక ఉదాహరణ – return type input మీద ఆధారపడటం:

```
function getElement(selector: string): HTMLElement | null;
function getElement(selector: string, required: true): HTMLElement;
function getElement(selector: string, required?: boolean): HTMLElement | null {
    const el = document.querySelector<HTMLElement>(selector);
    if (required && !el) throw new Error(`Element not found: ${selector}`);
    return el;
}

const a = getElement(".btn"); // HTMLElement | null
const b = getElement(".btn", true); // HTMLElement ✔ null check అవసరం లేదు
```

⚠ **Overloads కంటే union/generics మేలు** చాలా సందర్భాల్లో. Overloads maintenance కష్టం, implementation signature type-safe కాదు.

```
// ✅ తరచుగా ఇది మెరుగు - conditional type తో
function getEl<T extends boolean = false>(
  sel: string, required?: T
): T extends true ? HTMLElement : HTMLElement | null { /* ... */ }
```

this parameter

```
// TS లో `this` ని మొదటి "fake" parameter గా declare చేయొచ్చు (runtime లో ఉండదు)
interface Button { label: string; onClick(this: Button, e: Event): void; }

function handler(this: HTMLElement, e: Event) {
  this.classList.add("clicked"); // ✅ this typed
}
element.addEventListener("click", handler); // ✅

// strictBindCallApply - bind/call/apply కూడా type-checked
function fn(a: number, b: string) {}
fn.call(null, 1, "x"); // ✅
fn.call(null, "x", 1); // ❌ Error (strictBindCallApply తో)
```

Callback types & void quirk

```
// ⚠️ void return type ఉన్న callback - ఏదైనా return చేయొచ్చు (ignore అవుతుంది)
type Cb = () => void;
const c: Cb = () => 42; // ✅ అనుమతి
const r = c(); // r: void

// ఇది ఎందుకు design చేశారు - ఇలాంటివి పని చేయాలని:
const arr: number[] = [];
[1, 2, 3].forEach((x) => arr.push(x)); // push() number return చేస్తుంది, forEach void అిస్తుంది

// ⚠️ ప్రమాదం: async callback లో
setTimeout(async () => { await save(); }, 100); // Promise return - errors swallow అవుతాయి!
```

Higher-order functions & generics

```
// Generic function types
function map<T, U>(arr: T[], fn: (item: T, index: number) => U): U[] {
  return arr.map(fn);
}

// Currying
const curry = <A, B, C>(fn: (a: A, b: B) => C) => (a: A) => (b: B): C => fn(a, b);

// Function composition (typed)
function pipe<A, B, C>(f: (a: A) => B, g: (b: B) => C): (a: A) => C {
  return (a) => g(f(a));
}

// Debounce - typed wrapper (parameters preserve చేయడం)
function debounce<T extends (...args: any[]) => void>(fn: T, ms: number) {
```

```
let timer: ReturnType<typeof setTimeout>; // ✓ Node/browser రెండింటిలోనూ పని చేస్తుంది
return (...args: Parameters<T>) => {
  clearTimeout(timer);
  timer = setTimeout(() => fn(...args), ms);
};
}
const search = debounce((q: string, page: number) => {}, 300);
search("abc", 1); // ✓ typed
search(1); // ✗ Error
```

Optional vs undefined in parameters

```
function f1(a?: string) {} // f1() ✓ f1(undefined) ✓
function f2(a: string | undefined) {} // f2() ✗ f2(undefined) ✓ – argument తప్పనిసరి

// exactOptionalPropertyTypes: true అయితే objects లో కూడా ఈ తేడా వర్తిస్తుంది
type T1 = { a?: string }; // a లేకపోవచ్చు
type T2 = { a: string | undefined }; // a తప్పనిసరిగా ఉండాలి (value undefined కావచ్చు)
```

Gotchas (సాధారణ తప్పులు)

- **Optional param** ని **required** కి ముందు పెట్టడం → syntax error.
- **Overload implementation signature** ని **callable** అనుకోవడం – అది బయటికి కనిపించదు.
- **Overload order** తప్పు – TS మొదటి match అయ్యే overload ని ఎంచుకుంటుంది; నిర్దిష్టమైనవి ముందు రాయాలి.
- **Function type** వాడటం – **Function** = ఏ function అయినా (any లాంటిది). (...args: any[]) => void లేదా నిర్దిష్ట signature వాడాలి.
- **async callback** ని **void** callback స్థానంలో ఇవ్వడం → unhandled rejections.
- **this** ని **arrow functions** లో annotate చేయడం – arrow కి own this లేదు.
- **NodeJS.Timeout** vs **number** – **ReturnType<typeof setTimeout>** వాడాలి.

Key Points

- Params ఎప్పుడూ annotate; return types public APIs కి.
- **Overloads** = input-dependent signatures; కానీ generics/conditional types తరచుగా మేలు.
- **this** parameter = compile-time only.
- **void** callback ఏదైనా return చేయొచ్చు (ఉద్దేశపూర్వక design).
- **Parameters<T>** , **ReturnType<T>** తో wrapper functions ని type-safe గా రాబోయిచ్చు.

Interview దృష్టి

- "Overloads ఎప్పుడు వాడతావు?" → argument shapes ని బట్టి return type మారినప్పుడు; alternatives (union, generic, conditional) కూడా చెప్పాలి.
- "Function type ఎందుకు చెడ్డది?" → any-like, arguments/return unchecked.

8. Objects, Interfaces & Index Signatures

వివరణ

```
interface User {
  readonly id: string;           // మార్పులేం (compile-time మాత్రమే!)
  name: string;
  email?: string;               // optional
  readonly tags: readonly string[]; // readonly array
  greet(): void;                // method
  onUpdate: (u: User) => void;   // property with function type
}

// Index signature – dynamic keys
interface StringMap { [key: string]: string; }
interface Config {
  name: string;                 // ✅ index signature type కి assignable ఉండాలి
  [key: string]: string;
}
interface Bad {
  name: string;
  age: number;                  // ❌ Error: 'number' is not assignable to 'string'
  [key: string]: string;
}
```

Real-life Scenario

Interface = ఉద్యోగ వివరణ (job description). "ఈ పాత్రలో ఉన్న వ్యక్తికి name ఉండాలి, email ఉండొచ్చు, greet() చేయగలగాలి." ఎవరు ఆ పనిచేస్తున్నారనేది ముఖ్యం కాదు – ఆ లక్షణాలు ఉంటే చాలు (structural typing). Employee, Contractor, Robot – ఎవరైనా సరిపోతారు, వాళ్ళు "implements" అని ప్రకటించనవసరం లేదు.

readonly – compile-time మాత్రమే

```
interface Point { readonly x: number; readonly y: number; }
const p: Point = { x: 1, y: 2 };
p.x = 5; // ❌ compile error
(p as any).x = 5; // ✅ runtime లో ఏమీ ఆపదు!

// Deep readonly కాదు – ఒక level మాత్రమే
interface Config { readonly db: { host: string } }
const c: Config = { db: { host: "a" } };
c.db.host = "b"; // ✅ అనుమతిస్తుంది! (db reference readonly, దాని లోపలిది కాదు)

// నిజమైన immutability కావాలంటే – Object.freeze (runtime) లేదా DeepReadonly type
```

Excess property checking (object literal freshness)

```
interface Options { width?: number; height?: number }

const o1: Options = { width: 10, wwidth: 20 }; // ❌ Error: 'wwidth' does not exist
//                                     ↑ typo పట్టుబడింది ⚡

// ⚠️ కానీ variable ద్వారా ఇన్ఫర్ చేక జరగదు!
const raw = { width: 10, wwidth: 20 };
const o2: Options = raw; // ✅ No error! (freshness పోయింది)

// ఎందుకు: TS structural typing – raw కి Options కి కావలసినవన్నీ ఉన్నాయి.
// Excess property check అనేది typos పట్టుకోవడానికి ఉన్న *ప్రత్యేక* నియమం,
// అది "fresh" object literals కి మాత్రమే వర్తిస్తుంది.
```

Index signatures – నియమాలు & ప్రత్యామ్నాయాలు

```
// (a) Index signature
type Scores = { [studentId: string]: number };
const s: Scores = { alice: 90 };
s.bob; // number – ⚠️ నిజానికి undefined!
// ✅ noUncheckedIndexedAccess: true → number | undefined

// (b) Record utility (ఎక్కువ చదవగలిగేది)
type Scores2 = Record<string, number>;
type Roles = Record<"admin" | "user", Permission[]>; // finite keys

// (c) Map – నిజమైన dynamic keys కి మేలు
const cache = new Map<string, User>(); // any key type, size, iteration order
```

ఎప్పుడు ఏది:

	Index signature / Record	Map
Key types	string / number / symbol	ఏదైనా (objects కూడా)
JSON serialization	✅	❌ (manual)
Iteration order	insertion (string keys) కానీ numeric keys ముందు!	insertion guaranteed
Size	Object.keys().length	.size ✅
Prototype pollution	⚠️ ప్రమాదం (__proto__)	✅ సురక్షితం

Object type modifiers combos

```
type Base = { a: string; b?: number; readonly c: boolean };

Partial<Base> // అన్నీ optional
Required<Base> // అన్నీ mandatory
Readonly<Base> // అన్నీ readonly
```

```
Pick<Base, "a" | "b">    // కొన్ని మాత్రమే
Omit<Base, "c">         // కొన్ని తీసేసి
```

Interface extends & implements

```
interface Animal { name: string }
interface Dog extends Animal { breed: string }
interface Pet extends Animal, Timestamped { owner: string } // multiple extends ✓

class Labrador implements Dog {
  name = "Bruno";
  breed = "Labrador";
  // ⚠ implements కేవలం *check* చేస్తుంది - inherit చేయదు
}
```

Gotchas (సాధారణ తప్పులు)

- **readonly** runtime immutability అనుకోవడం – కాదు; compile-time మాత్రమే, shallow.
- **Excess property check** variable ద్వారా **bypass** అవ్వడం – typos దాగిపోతాయి.
- **Index signature** తో **noUncheckedIndexedAccess** లేకపోవడం → **undefined** ప్రమాదం.
- **Index signature** లో **specific properties conflict** → compile error.
- **{}** ని "empty object" అనుకోవడం – **{}** దాదాపు అన్నిటినీ అంగీకరిస్తుంది (null/undefined తప్ప). ఖాళీ object కి **Record<string, never>**.
- **Optional property** **undefined** value తో **గందరగోళం** – **exactOptionalPropertyTypes** దీన్ని స్పష్టం చేస్తుంది.
- **Method shorthand vs property function** – method shorthand parameters **bivariant** (unsafe); property form **contravariant** (safe with **strictFunctionTypes**) – Topic 22.

Key Points

- **readonly** = compile-time, shallow.
- **Excess property checking** = fresh object literals కి మాత్రమే (typo safety net).
- **Index signature** vs **Record** vs **Map** – trade-offs తెలియాలి.
- **interface extends** multiple; **implements** = check only.
- **{}** / **object** / **Object** – ఏవీ నిజమైన "object" ని సూచించవు.

Interview దృష్టి

- "Excess property check ఎందుకు variable లో పని చేయదు?" → freshness rule + structural typing.
- "Record vs Map?" → key types, serialization, size, prototype safety.

9. type vs interface

వివరణ — పూర్తి పోలిక

	interface	type
Objects/classes	✓	✓
Primitives (<code>type ID = string</code>)	✗	✓
Unions	✗	✓
Tuples	✗	✓
Mapped types	✗	✓
Conditional types	✗	✓
Template literal types	✗	✓
Extends	<code>extends</code> ✓	<code>&</code> (intersection) ✓
Declaration merging	✓	✗
Class <code>implements</code>	✓	✓ (object type అయితే)
Error messages	తరచుగా స్పష్టం (పేరు నిలుస్తుంది)	పెద్ద types లో expand అవుతాయి
Performance (పెద్ద codebases)	కొంచెం మెరుగు (caching)	complex types నెమ్మది

```
// రెండూ ఒకటే పని
interface User { id: string; name: string }
type User2 = { id: string; name: string };

// type మాత్రమే చేయగలిగేవి
type ID = string | number; // union
type Point = [number, number]; // tuple
type Keys = keyof User; // keyof
type Partial2<T> = { [K in keyof T]?: T[K] }; // mapped
type IsString<T> = T extends string ? true : false; // conditional
type Route = `/${string}`; // template literal
type Fn = () => void; // (interface కూడా చేయగలదు కానీ awkward)
```

Declaration merging — interface యొక్క ప్రత్యేక శక్తి

```
interface Window { myApp: AppConfig } // global Window ని extend చేయడం
interface Window { analytics: Analytics } // మళ్ళీ - merge అవుతుంది ✓
// ఫలితం: Window కి రెండూ ఉన్నాయి

type A = { x: number };
type A = { y: number }; // ✗ Error: Duplicate identifier 'A'
```

ఇది ఎక్కడ కీలకం — module augmentation (Topic 19):

```
// Express request కి custom property జోడించడం
declare global {
  namespace Express {
    interface Request { user?: AuthUser }
  }
}
// ఇప్పుడు req.user అన్ని చోట్లా typed ✔
```

Real-life Scenario

interface = ఒక ఇంటి పేరు (family name). కొత్త సభ్యులు కుటుంబంలో చేరొచ్చు (declaration merging) — పేరు అదే ఉంటుంది. **type** = ఒక ఫోటో — తీసేశాక మార్చలేరు; కానీ ఏదైనా capture చేయగలదు (union, tuple, computed).

ఆచరణాత్మక నియమం (team convention)

- ✔ Objects/classes కి **public** API shapes → **interface** (extendable, merging, better errors)
- ✔ Unions, tuples, primitives, utility/mapped/conditional types → **type**
- ✔ ఒక నియమం **team** అంతా పాటించాలి — **mixing** గందరగోళం

TypeScript team స్వయంగా internal codebase లో ఎక్కువగా **interface** వాడుతుంది objects కి. React types (@types/react) కూడా **interface** (extendable కావాలని).

extends vs intersection — సూక్ష్మ తేడా

```
// interface extends — conflicting members ఉంటే error ✔ (ముందే పట్టుబడుతుంది)
interface A { x: string }
interface B extends A { x: number } // ✗ Error: Interface 'B' incorrectly extends 'A'

// type intersection — silent never
type C = { x: string } & { x: number }; // { x: never } — error లేదు, కానీ వాడలేం
const c: C = { x: ??? }; // ఏమీ ఇవ్వలేం
```

Gotchas (సాధారణ తప్పులు)

- "interface వాడకూడదు" అనే గుడ్డి నియమం — రెండూ చెల్లుబాటు; సందర్భం ముఖ్యం.
- Library public types కి **type** వాడటం — consumers extend/augment చేయలేరు.
- Intersection తో conflicts silent never — **interface extends** ముందే error ఇస్తుంది.
- Union కి interface వాడటానికి ప్రయత్నించడం — సాధ్యం కాదు.
- అనవసరంగా declaration merging — accidental global pollution.

Key Points

- **type** = ఎక్కువ శక్తివంతం (unions, tuples, computed types).
- **interface** = **declaration merging** + extendable public APIs + మంచి errors.
- నియమం: objects/public API → **interface** ; మిగతావన్నీ → **type** .
- **interface extends** conflicts ని ముందే పట్టుకుంటుంది; & silent never ఇస్తుంది.

Interview దృష్టి

- "type vs interface?" → capability table + declaration merging + మీ team convention + ఎందుకు.
- "Library types కి ఏది?" → interface (augmentation, extendability).

10. Enums — మరియు వాటిని ఎందుకు నివారించాలి

వివరణ

```
// Numeric enum (default)
enum Direction { Up, Down, Left, Right } // 0, 1, 2, 3
enum Status { Active = 1, Inactive = 2 }

// String enum (మేల్కొనబడి)
enum Role { Admin = "ADMIN", User = "USER", Guest = "GUEST" }

const r: Role = Role.Admin;
console.log(r); // "ADMIN"
```

Enum runtime లో ఏమవుతుంది (type erasure కి మినహాయింపు!)

```
enum Direction { Up, Down }
```

```
// Compiled output – enum నిజమైన JavaScript object ని emit చేస్తుంది!
var Direction;
(function (Direction) {
    Direction[Direction["Up"] = 0] = "Up"; // reverse mapping!
    Direction[Direction["Down"] = 1] = "Down";
})(Direction || (Direction = {}));
// Direction = { 0: "Up", 1: "Down", Up: 0, Down: 1 }
```

ఇదే enum యొక్క మొదటి సమస్య – TypeScript యొక్క "types runtime ని ప్రభావితం చేయవు" సూత్రాన్ని enum ఉల్లంఘిస్తుంది. అందుకే **Node.js native type stripping** (`--experimental-strip-types`) enum ని support చేయదు; `isolatedModules` తో `const enum` పని చేయదు.

Real-life Scenario

Numeric enum = గదులకి పేర్లు లేకుండా సంబంధ మార్తమే. "Room 2 కి వెళ్ళు" – సంబరింగ్ మారితే (మధ్యలో కొత్త గది కట్టితే) అందరూ తప్పు గదికి వెళ్తారు. **String enum** = గదులకి పేర్లు – "Kitchen" ఎప్పుడూ Kitchen. **Union of literals** = గోడ మీద direct గా పేర్లు రాయడం – extra structure అవసరం లేదు, అదే ప్రయోజనం.

Numeric enums యొక్క ప్రమాదాలు

```
enum Status { Active, Inactive }

const s: Status = 5; // ✓ TS 5.0 కి ముందు అనుమతించేది! (ఇప్పుడు error)
console.log(Status[0]); // "Active" – reverse mapping (bundle size పెరుగుతుంది)

// DB/API లో numbers store అయితే – order మారితే data corrupt
enum Priority { Low, Medium, High } // 0, 1, 2
// తర్వాత ఎవరో: enum Priority { None, Low, Medium, High } // అన్నీ shift! ✨
```

✓ ప్రత్యామ్నాయం 1 – Union of string literals (ఉత్తమం)

```
type Role = "ADMIN" | "USER" | "GUEST";

function check(role: Role) {}
check("ADMIN"); // ✓
check("admin"); // ✗ typo compile time లో

// ✓ లాభాలు: zero runtime code, autocomplete, JSON-friendly, narrowing పని చేస్తుంది
```

✓ ప్రత్యామ్నాయం 2 – as const object (values + type రెండూ కావాలంటే)

```
const Role = {
  Admin: "ADMIN",
  User: "USER",
  Guest: "GUEST",
} as const;

type Role = (typeof Role)[keyof typeof Role]; // "ADMIN" | "USER" | "GUEST"

// వాడకో – enum లాగే
const r: Role = Role.Admin;
Object.values(Role); // ✓ iterate చేయొచ్చు
Object.keys(Role);
// runtime లో ఇది సాధారణ object – reverse mapping లేదు, tree-shakeable
```

ఈ pattern ని **enum** కి పూర్తి ప్రత్యామ్నాయంగా వాడొచ్చు:

```
const HttpStatus = { OK: 200, NotFound: 404, ServerError: 500 } as const;
type HttpStatus = (typeof HttpStatus)[keyof typeof HttpStatus]; // 200 | 404 | 500
```

```
const Colors = ["red", "green", "blue"] as const;
type Color = (typeof Colors)[number]; // "red" | "green" | "blue"
```

const enum — inline అవుతుంది కానీ సమస్యత్వం

```
const enum Dir { Up, Down }
const x = Dir.Up; // compile → const x = 0; (enum object emit కాదు ✓)

// ✗ కానీ: isolatedModules తో పని చేయదు, declaration files లో ప్రమాదకరం,
// Babel/esbuild/swc సరిగా handle చేయవు, cross-module inlining bugs
// → 2026 లో వాడొద్దు
```

Enum ఎప్పుడు ఆమోదయోగ్యం

- **Legacy codebase** లో ఇప్పటికే విస్తృతంగా ఉంటే (consistency కోసం).
- **NestJS/TypeORM** వంటి frameworks కొన్ని చోట్ల expect చేస్తే.
- Reverse mapping నిజంగా అవసరమైతే (అరుదు).

TypeScript 5.0 enums ని మెరుగుపరిచింది (అన్ని enums ఇప్పుడు union enums; arbitrary number assign చేయలేం). కానీ **runtime emission** సమస్య అలాగే ఉంది.

Gotchas (సాధారణ తప్పులు)

- **Numeric enum values** ని DB/API లో **persist చేయడం** — order మారితే data corrupt.
- **const enum** ని **library public API** లో — consumers build tools తో break అవుతుంది.
- Enum ని **Object.values()** తో **iterate చేయడం (numeric)** → reverse mappings కూడా వస్తాయి:

```
enum E { A, B }
Object.values(E); // ["A", "B", 0, 1] 🤯
Object.values(E).filter(v => typeof v === "number"); // workaround
```

- Enum members ని type గా వాడటం **మర్చిపోవడం** — **Role.Admin** ఒక value మరియు type రెండూ.
- **isolatedModules / verbatimModuleSyntax** తో **enum re-export** సమస్యలు.

Key Points

- Enum = **runtime code emit చేసే ఏకైక type construct** (type erasure మినహాయింపు).
- **Numeric enums ప్రమాదకరం** (implicit values, reverse mapping, order fragility).
- ✓ **ప్రత్యామ్నాయాలు**: string literal union (సాధారణం), **as const** object (values కావాలంటే).
- **const enum** = వాడొద్దు (tooling incompatibility).

Interview దృష్టి

- "Enum వాడతావా?" → సాధారణంగా కాదు; union/ **as const** ఎందుకు మేలో (runtime cost, tree-shaking, JSON, tooling) చెప్పగలగడం seniority signal.

- "Enum runtime లో ఏమవుతుంది?" → IIFE + object with forward/reverse mappings.

11. Classes in TypeScript

వివరణ

```
class Account {
  // Access modifiers (compile-time మాత్రమే!)
  public readonly id: string;
  private balance: number;           // TS-only private
  protected owner: string;           // subclasses కి అందుబాటులో
  #pin: string;                       // ✅ JS native private (runtime enforced)
  static bankCode = "SBIN";           // class-level
  static readonly MAX = 1_000_000;

  constructor(id: string, balance: number, owner: string, pin: string) {
    this.id = id;
    this.balance = balance;
    this.owner = owner;
    this.#pin = pin;
  }

  // Getter/setter
  get formattedBalance(): string { return `₹${this.balance.toFixed(2)}`; }
  set deposit(amount: number) {
    if (amount <= 0) throw new Error("Invalid amount");
    this.balance += amount;
  }

  // Method
  withdraw(amount: number): void {
    if (amount > this.balance) throw new Error("Insufficient funds");
    this.balance -= amount;
  }

  static create(owner: string): Account { return new Account(uuid(), 0, owner, "0000"); }
}
```

Parameter properties — boilerplate తగ్గించే shorthand

```
// ❌ Verbose
class User {
  private name: string;
  public age: number;
  constructor(name: string, age: number) { this.name = name; this.age = age; }
}

// ✅ Parameter properties – declare + assign ఒకేసారి
class User {
  constructor(private name: string, public age: number, readonly id: string) {}
  // TS automatic గా this.name = name; ... చేస్తుంది
}
```

```
}  
// ⚠️ ఇది TS-only syntax – Node native type stripping లో పని చేయదు
```

private vs #private (అత్యంత ముఖ్యమైన తేడా)

```
class A {  
  private secret = "ts-private";  
  #real = "js-private";  
}  
  
const a = new A();  
a.secret; // ❌ compile error  
(a as any).secret; // ✅ "ts-private" – runtime లో access అవుతుంది! 🤖  
console.log(JSON.stringify(a)); // {"secret":"ts-private"} – leak!  
  
a.#real; // ❌ SyntaxError (runtime లో కూడా enforced ✅)  
JSON.stringify(a); // #real కనిపించదు ✅
```

	private (TS)	#field (JS)
Enforcement	compile-time	runtime ✅
JSON.stringify లో కనిపిస్తుందా	✅ (leak)	❌
Subclass same name	❌ conflict	✅ స్వతంత్రం
Performance	native	కొంచెం overhead (WeakMap-ish)
ఎప్పుడు	legacy/simple	నిజమైన encapsulation కావాలంటే ✅

Abstract classes

```
abstract class Shape {  
  constructor(protected name: string) {}  
  abstract area(): number; // subclass తప్పనిసరిగా implement చేయాలి  
  abstract perimeter(): number;  
  
  describe(): string { // concrete method – inherit అవుతుంది  
    return `${this.name}: area=${this.area()}`;  
  }  
}  
  
class Circle extends Shape {  
  constructor(private radius: number) { super("Circle"); }  
  area(): number { return Math.PI * this.radius ** 2; }  
  perimeter(): number { return 2 * Math.PI * this.radius; }  
}  
  
new Shape("x"); // ❌ Cannot create an instance of an abstract class
```

implements VS extends

```
interface Serializable { serialize(): string }
interface Comparable<T> { compareTo(other: T): number }

// implements – contract check మాత్రమే (code inherit కాదు); multiple ✔️
class Product implements Serializable, Comparable<Product> {
  constructor(public name: string, public price: number) {}
  serialize() { return JSON.stringify(this); }
  compareTo(other: Product) { return this.price - other.price; }
}

// extends – actual inheritance; ఒక్కటే ✔️
class DiscountedProduct extends Product {
  constructor(name: string, price: number, private discount: number) {
    super(name, price); // 🔑 తప్పనిసరి, this వాడేముందు
  }
  override serialize() { // noImplicitOverride: true → override keyword
    తప్పనిసరి
    return JSON.stringify({ ...this, final: this.price * (1 - this.discount) });
  }
}
```

⚠️ **implements** types ని infer చేయదు!

```
class P implements Serializable {
  serialize() { return 42; } // ❌ Error – కానీ parameters implicit any అవుతాయి
}
```

అంటే **implements** కేవలం check; parameter types మీరే రాయాలి.

strictPropertyInitialization

```
class User {
  name: string; // ❌ Error: Property 'name' has no initializer
  email!: string; // ✔️ definite assignment assertion ("నేను చూసుకుంటాను")
  age: number = 0; // ✔️ initializer
  id?: string; // ✔️ optional
  constructor(public role: string) {} // ✔️ parameter property
}
```

Generic classes

```
class Repository<T extends { id: string }> {
  private items = new Map<string, T>();

  add(item: T): void { this.items.set(item.id, item); }
  get(id: string): T | undefined { return this.items.get(id); }
  getAll(): T[] { return [...this.items.values()]; }
  findBy<K extends keyof T>(key: K, value: T[K]): T[] {
```

```

    return this.getAll().filter((i) => i[key] === value);
  }
}

const userRepo = new Repository<User>();
userRepo.findBy("email", "a@b.com"); // ✅ key & value type-checked
userRepo.findBy("email", 42);        // ❌ Error

```

Static side & typeof Class

```

class Animal { static create(): Animal { return new Animal(); } }

// Class యొక్క "static side" ని type చేయడం
type AnimalConstructor = typeof Animal;           // static members + constructor
type AnimalInstance = InstanceType<typeof Animal>; // Animal

function factory<T>(Ctor: new (...args: any[]) => T): T { // constructor type
  return new Ctor();
}

```

Decorators (TS 5.0+ = ECMAScript standard decorators)

```

// Stage 3 standard decorators (TS 5.0+) – legacy experimentalDecorators కి వేరు
function logged<This, Args extends any[], Return>(
  target: (this: This, ...args: Args) => Return,
  context: ClassMethodDecoratorContext
) {
  return function (this: This, ...args: Args): Return {
    console.log(`Calling ${String(context.name)}`);
    return target.call(this, ...args);
  };
}

class Service {
  @logged
  fetchData(id: string) { return db.get(id); }
}

```

⚠️ **NestJS/TypeORM/Angular** ఇంకా **legacy decorators** (`experimentalDecorators: true` + `emitDecoratorMetadata`) వాడతాయి – అవి standard వాటికి incompatible. Config జాగ్రత్తగా చూడాలి.

Gotchas (సాధారణ తప్పులు)

- **private** ని security అనుకోవడం – compile-time మాత్రమే; # వాడాలి.
- **super()** కి ముందు **this** వాడటం → error.
- **implements** inherit చేస్తుందని అనుకోవడం – కాదు, check మాత్రమే.
- **Arrow method vs prototype method:**

```
class A {
  handleA = () => {}    // instance property - ప్రతి instance కి కొత్త function (memory), కానీ
  this bound ✓
  handleB() {}          // prototype - memory efficient, కానీ this unbound (React handlers
లో సమస్య)
}
```

- **strictPropertyInitialization** ని ! తో గుడ్డిగా **bypass చేయడం** – నిజమైన bugs దాగుతాయి.
- **Class** ని **type** గా వాడినప్పుడు **structural typing surprise** – **private** members ఉంటే **nominal-ish** ప్రవర్తన:

```
class A { private x = 1 }
class B { private x = 1 }
const a: A = new B();    // ✗ Error - private members వేర్వేరు declarations
```

- **Decorators config mismatch** (legacy vs standard).

Key Points

- Access modifiers compile-time; **#private** = నిజమైన runtime privacy.
- **Parameter properties** = boilerplate తగ్గింపు (TS-only syntax).
- **abstract** = incomplete base; **implements** = contract check (multiple), **extends** = inheritance (single).
- **strictPropertyInitialization** + **override** keyword = correctness.
- Private members ఉంటే classes **nominal-ish** గా ప్రవర్తిస్తాయి.

Interview దృష్టి

- "**private** vs # ?" → compile vs runtime, JSON leak, subclass behaviour.
- "**Classes structural typing** కి మినహాయింపా?" → private/protected members ఉంటే అవును.
- "**Abstract class vs interface**?" → implementation + state vs pure contract; single vs multiple; runtime existence.

Part 2 — Generics & Advanced Types

12. Generics Deep

వివరణ

Generic = types కి parameters. Function కి values ఇచ్చినట్లే, type కి types ఇవ్వడం.

```
// ❌ Generic లేకుండా – type safety పోతుంది
function firstAny(arr: any[]): any { return arr[0]; }
const x = firstAny([1, 2, 3]); // any – తర్వాత ఏ safety లేదు

// ❌ Overloads – scale కాదు
function firstNum(arr: number[]): number;
function firstStr(arr: string[]): string;

// ✅ Generic – ఒకే definition, అన్ని types
function first<T>(arr: T[]): T | undefined { return arr[0]; }
const a = first([1, 2, 3]); // number | undefined ✅
const b = first(["x", "y"]); // string | undefined ✅
```

Real-life Scenario

Generic = "ఏదైనా పట్టి" డబ్బా, కానీ ఒక్క రకమే. మీరు డబ్బాలో బియ్యం పెడితే – తీసినప్పుడు **బియ్యమే** వస్తుంది (T in, T out). డబ్బా ఏ రకం ధాన్యానికైనా పని చేస్తుంది (reusable), కానీ ఒక డబ్బాలో బియ్యం + పప్పు కలవవు (type safety). **any = అడుగు లేని సంచి** – ఏదైనా వేయొచ్చు, ఏం తీస్తారో తెలియదు.

Generic constraints — extends

```
// ❌ T గురించి ఏమీ తెలియదు
function logLength<T>(item: T) { console.log(item.length); } // ❌ Error

// ✅ Constraint – "T కి తప్పనిసరిగా length ఉండాలి"
function logLength<T extends { length: number }>(item: T): T {
  console.log(item.length);
  return item; // ✅ input type preserve అవుతుంది
}
logLength("hello"); // ✅ string
logLength([1, 2]); // ✅ number[]
logLength(42); // ❌ Error

// keyof constraint – property access కి type-safe
function getProp<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
const user = { name: "S", age: 30 };
```

```
getProp(user, "name"); // string ✔️
getProp(user, "email"); // ❌ Error – typo compile time లో ⚡
```

Generic defaults

```
interface ApiResponse<TData = unknown, TError = Error> {
  data: TData | null;
  error: TError | null;
  status: number;
}
type R1 = ApiResponse; // ApiResponse<unknown, Error>
type R2 = ApiResponse<User[]>; // data: User[] | null
type R3 = ApiResponse<User, ValidationError>;
```

Generic inference — ఎలా పని చేస్తుంది

```
// TS arguments నుండి T ని infer చేస్తుంది
function identity<T>(x: T): T { return x; }
identity(5); // T = 5 (literal! const context)... నిజానికి T = number (widened)
identity<string>("a"); // explicit

// Multiple inference sites
function merge<T, U>(a: T, b: U): T & U { return { ...a, ...b }; }
const m = merge({ x: 1 }, { y: "a" }); // { x: number } & { y: string }

// Inference నుండి literal types పొందడం
function pick<const T extends readonly string[]>(keys: T): T { return keys; }
// ↑ TS 5.0 const type parameter
const k = pick(["a", "b"]); // readonly ["a", "b"] ✔️ (string[] కాదు)
```

Generic classes & interfaces

```
interface Repository<T, ID = string> {
  findById(id: ID): Promise<T | null>;
  findAll(filter?: Partial<T>): Promise<T[]>;
  create(data: Omit<T, "id" | "createdAt">): Promise<T>;
  update(id: ID, data: Partial<T>): Promise<T>;
  delete(id: ID): Promise<void>;
}

class InMemoryRepo<T extends { id: string }> implements Repository<T> {
  private store = new Map<string, T>();
  async findById(id: string) { return this.store.get(id) ?? null; }
  async findAll(filter?: Partial<T>) {
    const all = [...this.store.values()];
    if (!filter) return all;
    return all.filter((item) =>
      (Object.entries(filter) as [keyof T, T[keyof T]][]).every(([k, v]) => item[k] === v)
    );
  }
  async create(data: Omit<T, "id" | "createdAt">) {
```

```

    const item = { ...data, id: crypto.randomUUID(), createdAt: new Date() } as unknown as T;
    this.store.set(item.id, item);
    return item;
  }
  async update(id: string, data: Partial<T>) {
    const existing = await this.findById(id);
    if (!existing) throw new Error("Not found");
    const updated = { ...existing, ...data };
    this.store.set(id, updated);
    return updated;
  }
  async delete(id: string) { this.store.delete(id); }
}

```

వాస్తవిక generic patterns

```

// 1. Type-safe event emitter
type EventMap = {
  login: { userId: string };
  logout: void;
  error: { code: number; message: string };
};

class TypedEmitter<T extends Record<string, any>> {
  private listeners: { [K in keyof T]?: Array<(payload: T[K]) => void> } = {};

  on<K extends keyof T>(event: K, cb: (payload: T[K]) => void): () => void {
    (this.listeners[event] ??= []).push(cb);
    return () => this.off(event, cb);
  }
  off<K extends keyof T>(event: K, cb: (payload: T[K]) => void) {
    this.listeners[event] = this.listeners[event]?.filter((l) => l !== cb);
  }
  emit<K extends keyof T>(event: K, payload: T[K]) {
    this.listeners[event]?.forEach((cb) => cb(payload));
  }
}

const emitter = new TypedEmitter<EventMap>();
emitter.on("login", (p) => console.log(p.userId)); // ✓ p typed
emitter.emit("login", { userId: "1" });           // ✓
emitter.emit("login", { userid: "1" });           // ✗ typo caught
emitter.emit("unknown", {});                      // ✗ Error

// 2. Type-safe API client
async function apiGet<T>(url: string): Promise<T> {
  const res = await fetch(url);
  if (!res.ok) throw new ApiError(res.status);
  return res.json() as Promise<T>; // ⚠ ఇది "నమ్మకం" - Zod తో validate చేయడం మేలు
}

const users = await apiGet<User[]>("/api/users");

// 3. Deep partial (recursive generic)
type DeepPartial<T> = T extends object ? { [K in keyof T]?: DeepPartial<T[K]> } : T;

```

Generics ఎప్పుడు వాడకూడదు

```
// ❌ ఒకేసారి వాడే generic parameter → అర్థరహితం
function log<T>(x: T): void { console.log(x); } // T ఎందుకు? `unknown` సరిపోతుంది

// ❌ Return type లో మాత్రమే ఉన్న generic → ఇది `as` casting లాంటిదే (unsafe)
function parse<T>(s: string): T { return JSON.parse(s); }
const u = parse<User>("{}"); // u: User – కానీ నిజానికి {} !

// ✅ మేలు
function parse(s: string): unknown { return JSON.parse(s); }
const u = UserSchema.parse(parse(str)); // validated ✅
```

నియమం (Golden Rule of Generics): "ఒక type parameter కనీసం **రెండు చోట్ల** కనిపించాలి" (ఉదా. parameter లో + return లో). ఒకే చోట ఉంటే అది generic కాదు – అది disguised **any**.

Gotchas (సాధారణ తప్పులు)

- **Return-only generics** → unsafe assertion.
- **అతిగా generic చేయడం** – చదవడం కష్టం, benefit లేదు.
- **Constraint మర్చిపోవడం** → T మీద ఏ operation చేయలేం.
- **T inference literal కాకపోవడం** → **as const** లేదా **const T** (TS 5.0).
- **Generic + any కలపడం** – safety పోతుంది.
- **extends {} constraint** – దాదాపు అర్థరహితం (null/undefined మాత్రమే ఆపుతుంది).
- **అర్థంకాని single-letter names** – T, K, V OK; complex cases లో TData, TError వంటివి మేలు.

Key Points

- Generics = type-level parameters; reuse + safety.
- **extends** = constraint; **= X** = default; **keyof** తో property access safety.
- **Golden rule:** type parameter కనీసం 2 చోట్ల ఉండాలి.
- Return-only generic = **as** cast (unsafe).
- Real patterns: repositories, event emitters, API clients, utility types.

Interview దృష్టి

- "Generic ఎందుకు, **any** ఎందుకు కాదు?" → relationship preserve (in-type ↔ out-type), autocomplete, refactor safety.
- "**getProp** రాయి" → **<T, K extends keyof T>(o: T, k: K): T[K]** – classic question.
- "Generic constraint ఎప్పుడు?" → T మీద operations చేయాలంటే.

13. keyof , typeof , Indexed Access Types

వివరణ — type-level లో "reading"

ఈ మూడూ కలిసి **types నుండి types ని derive చేసే** ప్రాథమిక సాధనాలు. ఇవి అర్థమైతే advanced TypeScript సగం అర్థమైనట్టే.

```
interface User { id: string; name: string; age: number; address: { city: string } }

// 1️⃣ keyof – object type యొక్క keys ని union గా
type UserKeys = keyof User; // "id" | "name" | "age" | "address"

// 2️⃣ typeof – *value* నుండి *type* ని పొందడం (type-level operator)
const config = { host: "localhost", port: 3000, debug: true };
type Config = typeof config; // { host: string; port: number; debug: boolean }

// 3️⃣ Indexed access – type లోని ఒక property యొక్క type
type Name = User["name"]; // string
type IdOrAge = User["id" | "age"]; // string | number
type AllValues = User[keyof User]; // string | number | { city: string }
type City = User["address"]["city"]; // string (nested)
```

Real-life Scenario

typeof = ఒక వస్తువుని చూసి దాని blueprint తయారు చేయడం. మే దగ్గర ఇప్పటికే ఒక కుర్చీ (value) ఉంది – దాని design (type) విడిగా రాయనవసరం లేదు, కుర్చీని కొలిచి blueprint తీయొచ్చు. **keyof** = ఆ blueprint లోని భాగాల జాబితా. Indexed access = "కాలు" భాగం యొక్క వివరణ చూడటం.

typeof — value world నుండి type world కి వంతెన

```
// TypeScript లో రెండు "worlds" ఉన్నాయి:
const x = 5; // value world
type X = number; // type world
// `typeof` వాటి మధ్య వంతెన (JS యొక్క runtime typeof కి పూర్తిగా వేరు!)

// వాడకాలు
const DEFAULT_CONFIG = { retries: 3, timeout: 5000, baseUrl: "/api" };
type Config = typeof DEFAULT_CONFIG;
function init(cfg: Config) {} // ✅ duplicate చేయనవసరం లేదు

// Function types
function createUser(name: string, age: number) { return { id: "1", name, age }; }
type CreateUser = typeof createUser; // (name: string, age: number) => {...}
type User = ReturnType<typeof createUser>; // { id: string; name: string; age: number }

// Class static side
class Service {}
type ServiceCtor = typeof Service; // constructor + statics
type ServiceInstance = InstanceType<typeof Service>; // Service
```

```
// Array/const objects
const ROLES = ["admin", "user", "guest"] as const;
type Role = (typeof ROLES)[number]; // "admin" | "user" | "guest" ★
```

(typeof X)[number] – array/tuple నుండి element type ని పొందే అత్యంత ఉపయోగకరమైన pattern. గుర్తుపెట్టుకోండి.

keyof యొక్క సూక్ష్మతలు

```
type A = keyof { a: string; b: number }; // "a" | "b"
type B = keyof any; // string | number | symbol
type C = keyof string[]; // number | "length" | "push" | "pop" | ...
type D = keyof { [k: string]: number }; // string | number ⚠ (numeric keys కూడా!)
type E = keyof {}; // never

// Numeric keys – JS లో obj[1] === obj["1"] కాబట్టి
type F = keyof { 1: string; 2: number }; // 1 | 2
```

ముఖ్యమైన combos (ఇవి production లో పదే పదే వస్తాయి)

```
// 1. Type-safe property getter
function get<T, K extends keyof T>(obj: T, key: K): T[K] { return obj[key]; }

// 2. Type-safe property setter
function set<T, K extends keyof T>(obj: T, key: K, value: T[K]): void { obj[key] = value; }

// 3. Pick keys by value type – "ఏ properties string లు?"
type StringKeys<T> = { [K in keyof T]: T[K] extends string ? K : never }[keyof T];
type UserStringKeys = StringKeys<User>; // "id" | "name"

// 4. Function keys మాత్రమే
type FunctionKeys<T> = { [K in keyof T]: T[K] extends Function ? K : never }[keyof T];

// 5. Object entries typed
function entries<T extends object>(obj: T): [keyof T, T[keyof T]][] {
  return Object.entries(obj) as [keyof T, T[keyof T]][];
}
// ⚠ Object.keys() ఎప్పుడూ string[] return చేస్తుంది (structural typing వల్ల – extra keys ఉండొచ్చు)

// 6. Nested path type (advanced – form libraries ఇలానే చేస్తాయి)
type Paths<T> = T extends object
  ? { [K in keyof T]: K extends string
    ? T[K] extends object ? K | `${K}.${Paths<T[K]>}` : K
    : never }[keyof T]
  : never;
type UserPaths = Paths<User>; // "id" | "name" | "age" | "address" | "address.city"
```

Gotchas (సాధారణ తప్పులు)

- **typeof** (type-level) vs **typeof** (runtime) గందరగోళం – ఒకే keyword, రెండు వేర్వేరు worlds.
- **Object.keys()** **keyof T** ఇవ్వదు – **string**[] ఇస్తుంది (structural typing: object లో extra properties ఉండిచ్చు కాబట్టి ఇది సరైనదే).
- **keyof** on index signature → **string** | **number** (కేవలం **string** కాదు).
- **typeof** ని type మీద వాడటం – **typeof** SomeType ❌; **typeof** values మీద మాత్రమే.
- **as const** మర్చిపోవడం → (**typeof** arr)[**number**] కి **string** వస్తుంది, literals రావు.

Key Points

- **typeof** = value → type (compile-time operator).
- **keyof** = object type → keys union.
- **T[K]** = indexed access; **T[keyof T]** = అన్ని values union.
- (**typeof** ARR)[**number**] = array → element union (const arrays కి).
- **<T, K extends keyof T>(o: T, k: K): T[K]** = type-safe accessor pattern.

Interview దృష్టి

- "keyof **typeof** obj అంటే?" → value నుండి type → దాని keys union (enum ప్రత్యామ్నాయాల్లో కీలకం).
- "**Object.keys** ఎందుకు **keyof T** ఇవ్వదు?" → structural typing/excess properties – soundness కారణం.

14. Conditional Types & infer

వివరణ

Conditional type = type-level **if/else** .

```
type IsString<T> = T extends string ? true : false;
type A = IsString<"hello">;      // true
type B = IsString<42>;          // false

// Syntax: T extends U ? X : Y
//          ("T అనేది U కి assignable అ?" – subtype check, equality కాదు)
```

Real-life Scenario

Conditional type = మధ్యాహ్న భోజన నియమం. "వ్యక్తి vegetarian అయితే → పప్పు, కాకపోతే → చికెన్." ఒకే rule, వ్యక్తిని బట్టి వేరే ఫలితం. **infer** = "వాళ్ళు తెచ్చిన బాక్స్ లో ఏముందో చూసి, దాని పేరు X అని పెట్టు, తర్వాత X ని వాడు."

వాస్తవిక ఉదాహరణలు

```
// 1. Nullable ని తీసేయడం
type NonNull<T> = T extends null | undefined ? never : T;
type A = NonNull<string | null>; // string

// 2. Function అయితే దాని return type
type Unwrap<T> = T extends (...args: any[]) => infer R ? R : T;
type B = Unwrap<() => string>; // string
type C = Unwrap<number>; // number

// 3. Promise ని unwrap
type Awaited2<T> = T extends Promise<infer U> ? Awaited2<U> : T; // recursive
type D = Awaited2<Promise<Promise<string>>>>; // string

// 4. Array element type
type ElementOf<T> = T extends (infer E)[] ? E : never;
type E = ElementOf<string[]>; // string
```

infer — deep dive

infer = "ఇక్కడ ఉన్న type ని ఒక variable లో పట్టుకో."

```
// Parameters
type Params<T> = T extends (...args: infer P) => any ? P : never;
type P1 = Params<(a: string, b: number) => void>; // [a: string, b: number]

// First parameter
type FirstParam<T> = T extends (first: infer F, ...rest: any[]) => any ? F : never;

// Constructor parameters
type CtorParams<T> = T extends new (...args: infer P) => any ? P : never;

// Tuple head/tail (type-level list processing)
type Head<T extends any[]> = T extends [infer H, ...any[]] ? H : never;
type Tail<T extends any[]> = T extends [any, ...infer R] ? R : never;
type Last<T extends any[]> = T extends [...any[], infer L] ? L : never;
type H = Head<[1, 2, 3]>; // 1
type T2 = Tail<[1, 2, 3]>; // [2, 3]
type L = Last<[1, 2, 3]>; // 3

// Multiple infer positions
type SwapPair<T> = T extends [infer A, infer B] ? [B, A] : never;
type S = SwapPair<[string, number]>; // [number, string]

// infer with constraint (TS 4.7+)
type FirstString<T> = T extends [infer F extends string, ...any[]] ? F : never;
```

Distributive conditional types (★ అత్యంత ముఖ్యం, తరచుగా అడుగుతారు)

```
type ToArray<T> = T extends any ? T[] : never;
type A = ToArray<string | number>;
```

```
// అంచనా: (string | number)[]
// నిజం: string[] | number[] ⚠️ - union ప్రతి member కి *విడిగా* apply అవుతుంది!
```

నియమం: conditional type యొక్క checked type ఒక **naked type parameter** (T) అయితే, unions మీద **distribute** అవుతుంది.

```
// Distribution అపడం - square brackets తో "wrap" చేయడం
type ToArrayNonDist<T> = [T] extends [any] ? T[] : never;
type B = ToArrayNonDist<string | number>; // (string | number)[] ✔️

// ఇదే Exclude/Extract ఎలా పని చేస్తాయో వివరిస్తుంది
type Exclude2<T, U> = T extends U ? never : T;
type C = Exclude2<"a" | "b" | "c", "a">;
// distribute: ("a" extends "a" ? never : "a") | ("b" extends "a" ? never : "b") | (...)
// = never | "b" | "c" = "b" | "c" ✔️

// ⚠️ ఉచ్చు - never కూడా distribute అవుతుంది (ఖాళీ union కాబట్టి ఫలితం never)
type D = ToArray<never>; // never (not never[])
```

వాస్తవిక advanced ఉదాహరణలు

```
// 1. Overloaded return type based on options
type QueryResult<T, Opts> = Opts extends { single: true } ? T | null : T[];

function query<T, O extends { single?: boolean }>(
  sql: string, opts?: O
): QueryResult<T, O> { /* ... */ }

// 2. Deep readonly
type DeepReadonly<T> = T extends (infer R)[]
  ? ReadonlyArray<DeepReadonly<R>>
  : T extends Function
  ? T
  : T extends object
  ? { readonly [K in keyof T]: DeepReadonly<T[K]> }
  : T;

// 3. Function overload resolution
type UnionToIntersection<U> =
  (U extends any ? (x: U) => void : never) extends (x: infer I) => void ? I : never;
type I = UnionToIntersection<{ a: string } | { b: number }>; // { a: string } & { b: number }

// ⚠️ ఇది contravariance ని exploit చేస్తుంది (Topic 22) - advanced interview question

// 4. Type-safe route params
type ExtractParams<T extends string> =
  T extends `${string}:${infer P}/${infer Rest}`
  ? P | ExtractParams<`/${Rest}`>
  : T extends `${string}:${infer P}`
  ? P
  : never;
type Params2 = ExtractParams<"/users/:userId/posts/:postId">; // "userId" | "postId"
```

Gotchas (సాధారణ తప్పులు)

- **Distribution ని మర్చిపోవడం** → అనుకోని results; [T] extends [U] తో ఆపాలి.
- **extends = "equals" అనుకోవడం** – అది **assignability (subtype)** check.

```
type X = "a" extends string ? true : false;           // true (subtype)
// Exact equality కి: type Equals<A, B> = (<T>() => T extends A ? 1 : 2) extends
//                                     (<T>() => T extends B ? 1 : 2) ? true : false;
```

- **any conditional లో** → రెండు branches యొక్క union ఇస్తుంది!

```
type Y = IsString<any>;           // boolean (true | false) 🤖
```

- **Recursion depth limit** → TS2589 "Type instantiation is excessively deep" (సుమారు 50 levels / 1000 instantiations).
- **అతి-సంక్లిష్ట conditional types** → compile నెమ్మది, error messages అర్థంకావు, team maintain చేయలేరు.

Key Points

- T extends U ? X : Y = type-level ternary (assignability check).
- **infer** = pattern matching – type లోని భాగాన్ని పట్టుకోవడం.
- **Naked T unions మీద distribute అవుతుంది**; [T] extends [U] ఆపుతుంది.
- **any conditional** → both branches union; **never** → never.
- Recursion limits ఉన్నాయి (TS2589).

Interview దృష్టి

- "Return Type <T> ని స్వయంగా రాయి" → T extends (...a: any) => infer R ? R : never – classic.
- "Distributive conditional types అంటే?" → union distribution + [T] trick – SSE-level question.
- "Exclude ఎలా పని చేస్తుంది?" → distribution + never filtering.

15. Mapped Types

వివరణ

Mapped type = ఒక type యొక్క ప్రతి property మీద "loop" చేసి కొత్త type తయారు చేయడం.

```
type Mapped<T> = { [K in keyof T]: T[K] };           // identity (కాపీ)

// Syntax: { [K in Keys]: ValueType }
```

Real-life Scenario

Mapped type = ఒక జాబితాలోని ప్రతి item కి ఒక మార్పు చేయడం. విద్యార్థుల జాబితా ఉంది – "ప్రతి ఒక్కరికీ 5 marks కలుపు" ($T[K] \rightarrow$ కొత్త type), లేదా "ప్రతి ఒక్కరినీ optional గా mark చేయి" (?). List structure అదే, ప్రతి entry transform అవుతుంది.

Built-in utility types — మనమే రాయడం

```
type MyPartial<T> = { [K in keyof T]?: T[K] };
type MyRequired<T> = { [K in keyof T]-?: T[K] }; // -? = optional తీసేయడం
type MyReadOnly<T> = { readonly [K in keyof T]: T[K] };
type MyMutable<T> = { -readonly [K in keyof T]: T[K] }; // -readonly = readonly తీసేయడం
type MyPick<T, K extends keyof T> = { [P in K]: T[P] };
type MyRecord<K extends keyof any, V> = { [P in K]: V };
type MyOmit<T, K extends keyof any> = MyPick<T, Exclude<keyof T, K>>;
```

Modifiers: + / - తో ? మరియు readonly ని add/remove చేయొచ్చు.

```
type Nullable<T> = { [K in keyof T]: T[K] | null };
type Stringify<T> = { [K in keyof T]: string };
type Getters<T> = { [K in keyof T]: () => T[K] };
type Promisify<T> = { [K in keyof T]: Promise<T[K]> };
```

Key remapping — as clause (TS 4.1+)

```
// Key names ని మార్చడం
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K]
};

interface Person { name: string; age: number }
type PersonGetters = Getters<Person>;
// { getName: () => string; getAge: () => number } ✓

// Keys ని filter చేయడం (never → key తొలగింపు)
type RemoveKind<T> = { [K in keyof T as Exclude<K, "kind">]: T[K] };

// Value type ఆధారంగా filter
type PickByType<T, V> = {
  [K in keyof T as T[K] extends V ? K : never]: T[K]
};

interface Mixed { a: string; b: number; c: string; d: () => void }
type OnlyStrings = PickByType<Mixed, string>; // { a: string; c: string }
type OnlyFunctions = PickByType<Mixed, Function>; // { d: () => void }
```

వాస్తవిక ఉపయోగాలు

```
// 1. API response → form state
type FormState<T> = {
  [K in keyof T]: { value: T[K]; error: string | null; touched: boolean }
}
```

```

};
type UserForm = FormState<{ name: string; age: number }>;
// { name: { value: string; error: string|null; touched: boolean }, age: {...} }

// 2. Event handlers auto-generate
type Events = { click: MouseEvent; keydown: KeyboardEvent };
type Handlers = { [K in keyof Events as `on${Capitalize<K>}`]?: (e: Events[K]) => void };
// { onClick?: (e: MouseEvent) => void; onKeyDown?: (e: KeyboardEvent) => void }

// 3. Deep partial (recursive mapped)
type DeepPartial<T> = T extends object
  ? { [K in keyof T]?: DeepPartial<T[K]> }
  : T;

// 4. Required తప్పని keys
type PartialBy<T, K extends keyof T> = Omit<T, K> & Partial<Pick<T, K>>;
type RequiredBy<T, K extends keyof T> = Omit<T, K> & Required<Pick<T, K>>;
type NewUser = PartialBy<User, "id" | "createdAt">; // create payload ✔️

// 5. Optional keys ని కనిపెట్టడం
type OptionalKeys<T> = { [K in keyof T]-?: {} extends Pick<T, K> ? K : never }[keyof T];
type RequiredKeys<T> = Exclude<keyof T, OptionalKeys<T>>;

// 6. Snake_case → camelCase (API adapters కి)
type CamelCase<S extends string> =
  S extends `${infer P1}_${infer P2}${infer P3}`
    ? `${Lowercase<P1>}${Uppercase<P2>}${CamelCase<P3>}`
    : Lowercase<S>;
type CamelKeys<T> = { [K in keyof T as CamelCase<string> & K]: T[K] };
type Api = { user_name: string; created_at: string };
type App = CamelKeys<Api>; // { userName: string; createdAt: string } ★

```

Homomorphic mapped types (సూక్ష్మమైన కానీ ముఖ్యమైన concept)

```

// `[K in keyof T]` రూపంలో ఉంటే అది "homomorphic" -
// అది modifiers (readonly, ?) మరియు array/tuple structure ని *కాపాడుతుంది*
type Partial2<T> = { [K in keyof T]?: T[K] };
type A = Partial2<string>[]; // (string | undefined)[] ✔️ array అలాగే ఉంది

// `[K in SomeOtherUnion]` అయితే homomorphic కాదు - structure పోతుంది
type B<T> = { [K in keyof T as K]: T[K] }; // as clause → homomorphic కాదు

```

Gotchas (సాధారణ తప్పులు)

- `keyof T` బదులు వేరే union వాడితే modifiers/array structure పోతుంది.
- **Key remapping** లో `never` → ఆ key తొలగించబడుతుంది (ఇది feature, bug కాదు).
- **Capitalize<K>** కి `string & K` అవసరం — `K` కి `string | number | symbol` ఉండొచ్చు.
- **Recursive mapped types + పెద్ద objects** → compile నెమ్మది / TS2589.
- **Mapped type లో methods పోగొట్టుకోవడం** — `T[K] extends Function` handle చేయాలి (DeepReadonly లో చూశాం).
- **Omit type-safe కాదు** — `Omit<User, "typo">` error ఇవ్వదు! (Key constraint `keyof any`):

```
type StrictOmit<T, K extends keyof T> = Omit<T, K>; // ✅ typo పట్టుబడుతుంది
```

Key Points

- { [K in keyof T]: ... } = type-level map/loop.
- Modifiers: ? , **readonly** , **-?** , -readonly .
- **Key remapping (as)** = rename/filter keys; never = key తొలగింపు.
- **Homomorphic** mapped types modifiers & array structure ని కాపాడతాయి.
- Real uses: form state, handlers, camelCase adapters, PartialBy/RequiredBy.

Interview దృష్టి

- "Partial<T> రాయ" → { [K in keyof T]?: T[K] } .
- "Value type ఆధారంగా keys filter చేయ" → **as** T[K] **extends** V ? K : **never** .
- "snake_case → camelCase type" → template literal + recursive conditional (SSE-level).

16. Template Literal Types

వివరణ

TypeScript 4.1 లో వచ్చిన feature — **string literal types** ని **JavaScript template literals** లా కలపడం.

```
type Greeting = `Hello ${string}`;  
const g1: Greeting = "Hello world"; // ✅  
const g2: Greeting = "Hi world"; // ❌ Error  
  
type Lang = "en" | "te";  
type Region = "IN" | "US";  
type Locale = `${Lang}-${Region}`; // "en-IN" | "en-US" | "te-IN" | "te-US"  
// ↑ cross product automatically! ⭐
```

Real-life Scenario

Template literal type = form లో "సమూహం" (pattern). బ్యాంకు form లో "IFSC code ఇలా ఉండాలి: 4 అక్షరాలు + 0 + 6 అంకెలు" — pattern సరిపోకపోతే form తిరస్కరించబడుతుంది. Template literal types compile time లోనే string shape ని enforce చేస్తాయి.

Built-in string manipulation types

```
type A = Uppercase<"hello">; // "HELLO"  
type B = Lowercase<"HELLO">; // "hello"
```

```

type C = Capitalize<"hello">; // "Hello"
type D = Uncapitalize<"Hello">; // "hello"

```

వాస్తవిక ఉపయోగాలు

```

// 1. CSS/design tokens
type Size = "sm" | "md" | "lg";
type Color = "red" | "blue";
type ClassName = `bg-${Color}-${100 | 500 | 900}` | `text-${Size}`;
const c: ClassName = "bg-red-500"; // ✓
const d: ClassName = "bg-red-300"; // ✗

// 2. Event names
type DomEvent = "click" | "focus" | "blur";
type HandlerName = `on${Capitalize<DomEvent>}`; // "onClick" | "onFocus" | "onBlur"

// 3. API routes - path params extraction
type Route = "/users/:id" | "/posts/:postId/comments/:commentId";

type ParamsOf<T extends string> =
  T extends `${infer _}:${infer P}/${infer Rest}`
    ? { [K in P | keyof ParamsOf<Rest>]: string }
    : T extends `${infer _}:${infer P}`
      ? { [K in P]: string }
      : {};

type P1 = ParamsOf<"/users/:id">; // { id: string }
type P2 = ParamsOf<"/posts/:postId/comments/:commentId">; // { postId: string; commentId: string }

function navigate<T extends Route>(route: T, params: ParamsOf<T>) {}
navigate("/users/:id", { id: "1" }); // ✓
navigate("/users/:id", { userId: "1" }); // ✗ Error ⚡

// 4. Nested object paths (from libraries, i18n, lodash get)
type Path<T, Prefix extends string = ""> = {
  [K in keyof T & string]: T[K] extends object
    ? `${Prefix}${K}` | Path<T[K], `${Prefix}${K}.`>
    : `${Prefix}${K}`
}[keyof T & string];

type Config = { db: { host: string; port: number }; app: { name: string } };
type ConfigPath = Path<Config>;
// "db" | "db.host" | "db.port" | "app" | "app.name" ★

function getConfig<P extends ConfigPath>(path: P): unknown {}
getConfig("db.host"); // ✓
getConfig("db.hosts"); // ✗ typo caught ⚡

// 5. SQL-ish / query builders
type Table = "users" | "posts";
type Query = `SELECT * FROM ${Table}` | `SELECT * FROM ${Table} WHERE ${string}`;

```

```
// 6. Env variable validation
type EnvKey = `VITE_${Uppercase<string>}`;
```

Advanced — string parsing at type level

```
// Split a string type
type Split<S extends string, D extends string> =
  S extends `${infer Head}${D}${infer Tail}` ? [Head, ...Split<Tail, D>] : [S];
type S = Split<"a.b.c", ".">; // ["a", "b", "c"]

// Join
type Join<T extends string[], D extends string> =
  T extends [infer F extends string, ...infer R extends string[]]
    ? R["length"] extends 0 ? F : `${F}${D}${Join<R, D>}`
    : "";
type J = Join<["a", "b", "c"], "-">; // "a-b-c"

// Trim
type Trim<S extends string> =
  S extends `${infer R}` ? Trim<R> : S extends `${infer R} ` ? Trim<R> : S;
```

Gotchas (సాధారణ తప్పులు)

- **Union combinatorial explosion** — `${A}-${B}-${C}` లో ప్రతి union పెద్దదైతే types వేలల్లో పెరిగి compiler ఆగిపోతుంది (TS లో union limit ~100,000).

```
type Big = `${string}-${string}`; // ✅ (string wildcard OK)
type Explode = `${Letters}${Letters}${Letters}`; // 26³ = 17,576 — జాగ్రత్త!
```

- `${number}` / `${string}` **wildcards** — ఇవి infinite patterns; useful కానీ narrowing పరిమితం.
- **Recursive parsing depth limits** → TS2589.
- **Over-engineering** — route typing బాగుంది కానీ 200-line type puzzle codebase ని బాధిస్తుంది; సాధారణ union తరచుగా సరిపోతుంది.

Key Points

- Template literal types = compile-time string patterns + cross products.
- Uppercase/Lowercase/Capitalize/Uncapitalize built-in.
- infer తో string parsing (routes, paths, split/join).
- **Combinatorial explosion** ప్రమాదం — పరిమాణం గమనించాలి.

Interview దృష్టి

- "Route params ని type-safe గా ఎలా చేస్తావు?" → template literal + infer — impressive demo.
- "Path<T> type ఎలా?" → recursive mapped + template literal.

17. Built-in Utility Types

వివరణ — పూర్తి జాబితా + implementation

TypeScript తో వచ్చే ready-made type transformers. వీటిని స్వయంగా implement చేయగలగడం interview లో తరచుగా అడుగుతారు.

Object utilities

```
interface User { id: string; name: string; email?: string; readonly createdAt: Date }

Partial<User>           // అన్నీ optional
Required<User>         // అన్నీ mandatory (optional తీసేసి)
Readonly<User>         // అన్నీ readonly
Pick<User, "id" | "name"> // { id: string; name: string }
Omit<User, "createdAt">  // createdAt తప్ప అన్నీ
Record<"a" | "b", number> // { a: number; b: number }

// Implementations
type MyPartial<T> = { [K in keyof T]?: T[K] };
type MyRequired<T> = { [K in keyof T]-?: T[K] };
type MyReadonly<T> = { readonly [K in keyof T]: T[K] };
type MyPick<T, K extends keyof T> = { [P in K]: T[P] };
type MyRecord<K extends keyof any, T> = { [P in K]: T };
type MyOmit<T, K extends keyof any> = Pick<T, Exclude<keyof T, K>>;
```

Union utilities

```
Exclude<"a" | "b" | "c", "a">           // "b" | "c"
Extract<"a" | "b", "a" | "z">           // "a"
Nullable<string | null | undefined>     // string

// Implementations (distributive conditional types)
type MyExclude<T, U> = T extends U ? never : T;
type MyExtract<T, U> = T extends U ? T : never;
type MyNullable<T> = T & {};
```

Function utilities

```
type Fn = (name: string, age: number) => Promise<User>;

Parameters<Fn>           // [name: string, age: number]
ReturnType<Fn>            // Promise<User>
Awaited<ReturnType<Fn>>  // User ★ (nested promises కూడా unwrap)
ThisParameterType<Fn>    // unknown
OmitThisParameter<Fn>    // Fn without this

class Service { constructor(a: string, b: number) {} }
ConstructorParameters<typeof Service> // [a: string, b: number]
InstanceType<typeof Service>           // Service
```

```
// Implementations
type MyParameters<T extends (...a: any) => any> = T extends (...a: infer P) => any ? P : never;
type MyReturnType<T extends (...a: any) => any> = T extends (...a: any) => infer R ? R : any;
type MyAwaited<T> = T extends Promise<infer U> ? MyAwaited<U> : T;
type MyInstanceType<T extends new (...a: any) => any> = T extends new (...a: any) => infer R ? R : never;
```

String utilities

```
Uppercase<"abc"> // "ABC"
Lowercase<"ABC"> // "abc"
Capitalize<"abc"> // "Abc"
Uncapitalize<"Abc"> // "abc"
// ⚠️ ఇవి compiler-intrinsic - TypeScript లో implement చేయలేం
```

ఇతరాలు

```
NoInfer<T> // TS 5.4 - ఈ position నుండి inference ఆపడం
type Fn2 = <T>(items: T[], defaultValue: NoInfer<T>) => T;
// createStreetLight(["red","green"], "blue") → error ✅ (defaultValue నుండి T infer కాదు)
```

తరచుగా కావలసిన custom utilities (మీ codebase లో ఉండాల్సినవి)

```
// 1. Strict Omit (typo safety)
type StrictOmit<T, K extends keyof T> = Omit<T, K>;

// 2. Partial by keys
type PartialBy<T, K extends keyof T> = Omit<T, K> & Partial<Pick<T, K>>;
type RequiredBy<T, K extends keyof T> = Omit<T, K> & Required<Pick<T, K>>;

// 3. Deep variants
type DeepPartial<T> = T extends object ? { [K in keyof T]?: DeepPartial<T[K]> } : T;
type DeepReadonly<T> = T extends (infer R)[] ? ReadonlyArray<DeepReadonly<R>> : T extends Function ? T : T extends object ? { readonly [K in keyof T]: DeepReadonly<T[K]> } : T;
type DeepRequired<T> = T extends object ? { [K in keyof T]-?: DeepRequired<T[K]> } : T;

// 4. Nullable helpers
type Nullable<T> = T | null;
type Maybe<T> = T | null | undefined;

// 5. Value union
type ValueOf<T> = T[keyof T];
type ArrayElement<T> = T extends readonly (infer E)[] ? E : never;

// 6. Exactly one of (mutually exclusive props)
type XOR<T, U> =
  | (T & { [K in Exclude<keyof U, keyof T>]?: never })
```

```

    | (U & { [K in Exclude<keyof T, keyof U>]?: never });
type Auth = XOR<{ token: string }, { apiKey: string }>;
const a1: Auth = { token: "x" }; // ✓
const a2: Auth = { token: "x", apiKey: "y" }; // ✗ Error ✓

// 7. Prettify – IntelliSense లో types ని విస్తరించి చూపడం (debugging కి బంపారం)
type Prettify<T> = { [K in keyof T]: T[K] } & {};
type Ugly = Omit<User, "id"> & { role: string };
type Nice = Prettify<Ugly>; // hover చేస్తే పూర్తి object shape కనిపిస్తుంది ★

// 8. Mutable (readonly తీసేయడం)
type Mutable<T> = { -readonly [K in keyof T]: T[K] };

// 9. Non-empty array
type NonEmptyArray<T> = [T, ...T[]];
function first<T>(arr: NonEmptyArray<T>): T { return arr[0]; } // undefined check అవసరం లేదు ✓

// 10. Type-safe Object.keys
function objectKeys<T extends object>(obj: T): (keyof T)[] {
    return Object.keys(obj) as (keyof T)[];
}

```

type-fest library

Production లో ఈ utilities స్వయంగా రాయకుండా **type-fest** వాడొచ్చు – **Simplify**, **SetRequired**, **Merge**, **CamelCasedProperties**, **JsonValue**, **Opaque**, **LiteralUnion**, **PackageJson** వంటి 200+ tested utilities.

```

import type { Simplify, SetOptional, Merge, LiteralUnion } from "type-fest";

// LiteralUnion – autocomplete + arbitrary strings రెండూ
type Color = LiteralUnion<"red" | "green", string>;
// "red"/"green" autocomplete వస్తుంది కానీ ఏ string అయినా అనుమతిస్తుంది

```

Gotchas

- **Omit key typos ని పట్టుకోడు** (K extends keyof any) → StrictOmit వాడాలి.
- **Omit unions మీద distribute కాదు:**

```

type U = { a: 1; x: string } | { a: 2; y: number };
type O = Omit<U, "a">; // { x: string } – ⚠ union collapse! (y పోయింది)
// ✓ distributive version
type DistOmit<T, K extends keyof any> = T extends any ? Omit<T, K> : never;

```

- **Partial deep కాదు** – ఒక level మాత్రమే.
- **Record<string, T> తో noUncheckedIndexedAccess** – access కి | undefined వస్తుంది (సరైనదే).
- **ReturnType overloaded functions మీద** – చివరి overload మాత్రమే తీసుకుంటుంది.

Key Points

- Object: **Partial**/Required/**Readonly**/Pick/Omit/**Record** .
- Union: **Exclude**/**Extract**/NonNullable (distributive conditionals).
- Function: **Parameters**/ReturnType/Awaited/InstanceType/ConstructorParameters .
- String: Uppercase/**Lowercase**/Capitalize/Uncapitalize (intrinsic).
- ము codebase కి: Prettify , PartialBy , DeepPartial , **XOR** , NonEmptyArray .
- Omit — typo-unsafe + union-collapsing (జాగ్రత్త).

Interview దృష్టి

- "Pick / Omit / Exclude implement చెయ్యి" → mapped + conditional types.
- "Omit లో ఏ లోపం ఉంది?" → key constraint + union distribution — SSE-level answer.

18. Type-Level Programming

వివరణ

TypeScript యొక్క type system **Turing-complete** — అంటే types తోనే "programs" రామొచ్చు (loops = recursion, conditionals = conditional types, variables = type parameters). ఇది SSE interviews లో "wow" factor, కానీ ఆచరణలో పరిమితంగా వాడాలి.

Recursion — type-level loops

```
// Length of tuple
type Length<T extends readonly any[]> = T["length"];
type L = Length<[1, 2, 3]>; // 3

// Reverse a tuple
type Reverse<T extends any[]> = T extends [infer H, ...infer R] ? [...Reverse<R>, H] : [];
type R = Reverse<[1, 2, 3]>; // [3, 2, 1]

// Repeat/build tuple of length N (arithmetic కి అధారం)
type BuildTuple<N extends number, R extends any[] = []> =
  R["length"] extends N ? R : BuildTuple<N, [...R, any]>;
type T5 = BuildTuple<5>; // [any, any, any, any, any]

// Type-level addition!
type Add<A extends number, B extends number> =
  [...BuildTuple<A>, ...BuildTuple<B>]["length"];
type Sum = Add<3, 4>; // 7 ★

// Subtraction
type Subtract<A extends number, B extends number> =
  BuildTuple<A> extends [...BuildTuple<B>, ...infer Rest] ? Rest["length"] : never;
type Diff = Subtract<10, 3>; // 7
```

Practical type-level utilities

```
// 1. Exact equality check (TS లో built-in లేదు)
type Equals<A, B> =
  (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true : false;
type E1 = Equals<string, string>; // true
type E2 = Equals<any, string>; // false ✅ (extends తో ఇది కుదరదు)

// 2. Union → Tuple (advanced, ordering guaranteed కాదు)
type UnionToIntersection<U> =
  (U extends any ? (x: U) => void : never) extends (x: infer I) => void ? I : never;
type LastOf<U> =
  UnionToIntersection<U extends any ? () => U : never> extends () => infer R ? R : never;
type UnionToTuple<U, R extends any[] = []> =
  [U] extends [never] ? R : UnionToTuple<Exclude<U, LastOf<U>>, [LastOf<U>, ...R]>;
type Tup = UnionToTuple<"a" | "b" | "c">; // ["a", "b", "c"]

// 3. Object flatten (dot paths → flat type)
type Flatten<T, Prefix extends string = ""> = {
  [K in keyof T & string as T[K] extends object
    ? never : `${Prefix}${K}`]: T[K]
} & UnionToIntersection<{
  [K in keyof T & string]: T[K] extends object ? Flatten<T[K], `${Prefix}${K}.`> : never
}[keyof T & string]>;

// 4. Type-safe deep get
type DeepGet<T, P extends string> =
  P extends `${infer K}.${infer Rest}`
    ? K extends keyof T ? DeepGet<T[K], Rest> : never
    : P extends keyof T ? T[P] : never;

type Cfg = { db: { host: string; pool: { max: number } } };
type Host = DeepGet<Cfg, "db.host">; // string
type Max = DeepGet<Cfg, "db.pool.max">; // number ✨

function get<T, P extends Paths<T>>(obj: T, path: P): DeepGet<T, P> { /* ... */ }
```

Compiler limits (తెలియకపోతే production లో దెబ్బ)

పరిమితి	విలువ (సుమారు)	error
Recursion depth	~50 (tail-recursion elimination తో 1000)	TS2589
Union members	~100,000	"Expression produces a union type that is too complex"
Instantiation count	~5,000,000	TS2589
Type instantiation depth	100	TS2589

```
// ⚠️ TS 4.5+ tail-recursion elimination – conditional type చివర్లో recursion ఉంటే
// depth limit 50 → 1000 కి పెరుగుతుంది
type Loop<N extends number, R extends any[] = []> =
  R["length"] extends N ? R : Loop<N, [...R, any]>; // ✅ tail-recursive
```

ఇది ఎప్పుడు వాడాలి / వాడకూడదు

✅ చెల్లుబాటు	❌ నివారించాలి
Library public APIs (routing, ORM, forms)	Application business logic
Route/path type safety	Team కి అర్థంకాని puzzles
API schema → types	Compile time 3× చేసే types
ORM query builders (Prisma, Drizzle, Kysely)	"నేను చేయగలను కాబట్టి"

అచరణాత్మక సూత్రం: "ఒక type ని అర్థం చేసుకోవడానికి 5 నిమిషాలు పడితే, అది తప్పు abstraction." Library authors కి ఇది సరైనది; application code లో సరళత ముఖ్యం.

Debugging type-level code

```
// 1. Prettify – expanded shape చూడటానికి
type Prettify<T> = { [K in keyof T]: T[K] } & {};

// 2. Type assertions in tests (compile-time tests)
type Expect<T extends true> = T;
type _test1 = Expect<Equals<Add<2, 3>, 5>>; // ✅ compile అయితే pass
type _test2 = Expect<Equals<Add<2, 3>, 6>>; // ❌ compile error = test fail

// 3. @ts-expect-error – తప్పు జరగాలని ఆశించడం
// @ts-expect-error
const bad: number = "string"; // ✅ error వచ్చింది కాబట్టి pass

// 4. Debug hover – intermediate types ని alias చేయడం
type Debug<T> = T;
type Step1 = Debug<SomeComplexType>; // hover చేసి చూడొచ్చు

// 5. vitest / expect-type
import { expectTypeOf } from "vitest";
expectTypeOf<Add<2, 3>>().toEqualTypeOf<5>();
```

Gotchas

- TS2589 "Type instantiation is excessively deep"** – recursion ని tail-recursive చేయాలి లేదా depth తగ్గించాలి.
- Compile time పీలడం** – ఒక్క complex type మొత్తం build ని నెమ్మది చేయొచ్చు (`--diagnostics` , `--generateTrace` తో కొలవాలి).

- **Editor freeze** — IDE ప్రతి keystroke కి type check చేస్తుంది.
- **Error messages అర్థంకావు** — 50-line type errors.
- **Equals<A, B> internal compiler behaviour మీద ఆధారపడుతుంది** — official కాదు.

Key Points

- Type system **Turing-complete** — recursion + conditionals + tuples తో arithmetic కూడా.
- **Limits ఉన్నాయి** (depth, union size, instantiations) → TS2589.
- Tail-recursive conditional types depth limit నీ 1000 కి పెంచుతాయి.
- Expect / **Equals** / expectTypeOf = type-level tests.
- **Library code కి శక్తివంతం; application code కి సరళత ముఖ్యం.**

Interview దృష్టి

- "Type-level programming ఎప్పుడు వాడతావు?" → library APIs; trade-offs (compile time, readability) చెప్పడం ముఖ్యం.
- "TS2589 ఎందుకు వస్తుంది, ఎలా fix?" → recursion depth; tail-recursion, memoization, simplification.

19. Declaration Files, Ambient Types & Module Augmentation

వివరణ

.d.ts file = types మాత్రమే, implementation లేదు. ఇవి JavaScript code కి types జోడించడానికి.

```
// math.d.ts — JS library కి types చెప్పడం
declare function add(a: number, b: number): number;
declare const VERSION: string;
declare class Calculator { add(n: number): this }
export { add, VERSION, Calculator };
```

Real-life Scenario

.d.ts = ఒక పరికరం యొక్క user manual. పరికరం (JS library) ఎలా పని చేస్తుందో లోపల ఏముందో మీకు తెలియదు, కానీ manual చెప్తుంది — "ఈ button 3 సెకన్లు నొక్కితే ఇలా జరుగుతుంది." Manual ని బట్టి మీరు సరిగ్గా వాడతారు (type checking), కానీ **manual తప్పుగా ఉంటే** (types actual JS తో match కాకపోతే) — మీరు తప్పుగా వాడతారు, పరికరం పాడవుతుంది (runtime error).

Types ఎక్కడ నుండి వస్తాయి — 3 మార్గాలు

1. **Library** లోనే (package.json → "types": "./dist/index.d.ts") ← ఆధునికం ✓
2. @types/<package> (**DefinitelyTyped**) ← community
3. మనమే రాయడం (src/types/*.d.ts) ← చివరి ఎంపిక

Module declaration — types లేని library

```
// src/types/untyped-lib.d.ts
declare module "untyped-lib" {
  export interface Options { debug?: boolean; retries?: number }
  export function init(options?: Options): void;
  export default class Client {
    constructor(url: string);
    request<T>(path: string): Promise<T>;
  }
}

// ⚡ త్వరిత (unsafe) పరిష్కారం - migration లో తాత్కాలికంగా
declare module "some-legacy-lib"; // implicitly any
```

Non-code imports (assets)

```
// src/types/assets.d.ts - Vite/webpack లో అవసరం
declare module "*.svg" {
  const content: React.FC<React.SVGProps<SVGSVGElement>>;
  export default content;
}
declare module "*.css" { const classes: Record<string, string>; export default classes; }
declare module "*.png" { const src: string; export default src; }
declare module "*.json" { const value: unknown; export default value; }
// (Vite లో `vite/client` types ఇవి ఇస్తాయి - `"types": ["vite/client"]`)
```

Global augmentation

```
// src/types/global.d.ts
declare global {
  // Window కి custom properties
  interface Window {
    dataLayer: any[];
    __APP_VERSION__: string;
  }

  // Environment variables - typed process.env ⭐
  namespace NodeJS {
    interface ProcessEnv {
      NODE_ENV: "development" | "production" | "test";
      DATABASE_URL: string;
      PORT?: string;
    }
  }

  // Global types
  type Nullable<T> = T | null;
}
export {}; // 🔑 ఇది తప్పనిసరి - లేకపోతే file "script" గా పరిగణించబడుతుంది, module కాదు
```

export {} ఎందుకు: ఒక file లో top-level **import** / **export** ఉంటే TS దాన్ని **module** గా చూస్తుంది. లేకపోతే అది **global script** – అప్పుడు **declare global** అనవసరం (మరియు error).

Module augmentation – ఉన్న library types ని extend చేయడం

```
// Express Request కి user property జోడించడం
import "express";
declare module "express-serve-static-core" {
  interface Request {
    user?: { id: string; role: "admin" | "user" };
  }
}
// ఇప్పుడు: app.get("/", (req) => req.user?.id) ✅ typed

// Vite env variables
interface ImportMetaEnv {
  readonly VITE_API_URL: string;
  readonly VITE_SENTRY_DSN?: string;
}
interface ImportMeta { readonly env: ImportMetaEnv }

// styled-components theme
import "styled-components";
declare module "styled-components" {
  export interface DefaultTheme {
    colors: { primary: string; danger: string };
    spacing: (n: number) => string;
  }
}

// React Query meta typing
declare module "@tanstack/react-query" {
  interface Register { defaultError: ApiError }
}
```

⚠ **Augmentation నియమాలు:** (1) **interface** లనే augment చేయగలం (declaration merging), **type** కాదు. (2) File ఒక module అయి ఉండాలి (import/export ఉండాలి). (3) కొత్త top-level declarations add చేయలేం, ఉన్నవాటిని extend మాత్రమే.

Library రాస్తున్నప్పుడు **.d.ts** generate చేయడం

```
// tsconfig.json
{
  "compilerOptions": {
    "declaration": true,           // .d.ts generate
    "declarationMap": true,       // .d.ts.map – go-to-definition source కి తీసుకెళ్ళండి ✅
    "emitDeclarationOnly": false
  }
}
```

```
// package.json – ఆధునిక dual-format package
{
  "type": "module",
  "types": "./dist/index.d.ts",
  "exports": {
    ".": {
      "types": "./dist/index.d.ts",    // 🔑 ఎప్పుడూ మొదట
      "import": "./dist/index.mjs",
      "require": "./dist/index.cjs"
    },
    "./package.json": "./package.json"
  },
  "files": ["dist"]
}
```

isolatedDeclarations (TS 5.5) – explicit types తప్పనిసరి చేసి **.d.ts** generation ని అత్యంత వేగవంతం చేస్తుంది (large monorepos కి).

Triple-slash directives (legacy కానీ ఇంకా కనిపిస్తాయి)

```
/// <reference types="node" />
/// <reference path="./other.d.ts" />
/// <reference lib="dom" />
```

ఆధునిక code లో `tsconfig` `types` / `lib` options వాడాలి; triple-slash ని **.d.ts** files లో మాత్రమే.

Gotchas (సాధారణ తప్పులు)

- export {} మర్చిపోవడం** `declare global` లో → error/ignored.
- .d.ts లో implementation రాయడం** – అక్కడ కేవలం declarations.
- declare module "x";** (body లేకుండా) → మొత్తం library **any** (silent type loss).
- Types actual runtime కి match కాకపోవడం** – **.d.ts** అబద్ధం చెప్పొచ్చు; runtime crash.
- @types version mismatch** – `@types/react 18` తో `react 19` → గందరగోళ errors.
- typeRoots / types config తప్పు** → global types కనిపించవు.
- Augmentation కి import లేకపోవడం** → module augmentation కాకుండా **module replacement** అవుతుంది (మొత్తం library types నాశనం!).

Key Points

- .d.ts** = types only; JS కి contracts.
- Types మూలాలు: `bundled` → `@types` → స్వయంగా.
- declare global + export {}** = global augmentation.
- declare module "lib" { interface X {} }** = **module augmentation** (interfaces మాత్రమే).
- Libraries: `declaration` + `declarationMap` + `exports.types` first.

Interview దృష్టి

- "Types లేని library ని ఎలా వాడతావు?" → @types వెతకడం → module declaration రాయడం → చివరి ఎంపిక any + TODO.
- "Express req.user ఎలా type చేస్తావు?" → module augmentation ఉదాహరణ.

20. Modules, import type , isolatedModules

వివరణ

```
// Named / default / namespace imports
import { User, getUser } from "./user";
import DefaultThing from "./thing";
import * as utils from "./utils";

// ✅ Type-only imports (TS 3.8+)
import type { User } from "./types";           // types మాత్రమే – runtime లో ఈ import మాయం
import { type User, getUser } from "./user";    // inline type modifier (TS 4.5+)
export type { User };
export { type Config, loadConfig };
```

import type ఎందుకు ముఖ్యం

```
// ❌ సాధారణ import – bundler దీన్ని runtime import అనుకోవచ్చు
import { User } from "./models";           // User కేవలం interface అయినా, import statement
మిగిలిపోవచ్చు
// → circular dependency, side-effect execution, పెద్ద bundle

// ✅ import type – compile అయ్యాక పూర్తిగా తొలగించబడుతుంది
import type { User } from "./models";
```

మూడు లాభాలు: (1) circular imports నివారణ, (2) accidental side effects లేవు, (3) transpiler (esbuild/swc) కి స్పష్టత – అవి ఒక file ని మాత్రమే చూస్తాయి కాబట్టి "ఇది type ఆ value ఆ" అని తెలియదు.

isolatedModules & verbatimModuleSyntax

```
{ "compilerOptions": { "isolatedModules": true, "verbatimModuleSyntax": true } }
```

isolatedModules: true – "ప్రతి file ని ఇతర files ని చూడకుండా transpile చేయగలగాలి" అని enforce చేస్తుంది (Babel/esbuild/swc ఇలానే పని చేస్తాయి). ఇది కొన్ని patterns ని నిషేధిస్తుంది:

```
// ❌ isolatedModules తో error – ఇది type ఆ value ఆ అని transpiler కి తెలియదు
export { SomeType } from "./types";

// ✅
export type { SomeType } from "./types";
```

```
// ✗ const enum cross-module
// ✗ ambient const enum
```

verbatimModuleSyntax: true (TS 5.0) – import/export statements ని అలాగే emit చేస్తుంది (type-only వాటిని తప్ప). ఇది importsNotUsedAsValues + preserveValueImports ని replace చేసింది. ESM/CJS interop స్పష్టత కోసం ఆధునిక projects లో recommended.

Module resolution — moduleResolution options

Value	ఎప్పుడు
"bundler" (TS 5.0+)	Vite, webpack, esbuild ✔️ (extensions అవసరం లేదు, exports గౌరవిస్తుంది)
"NodeNext" / "Node16"	Node.js native ESM/CJS ✔️ (.js extension తప్పనిసరి imports లో!)
"Node10" (పాత "node")	legacy CJS

```
// moduleResolution: "NodeNext" + ESM - ⚠️ .js extension రాయాలి (.ts కాదు!)
import { helper } from "./utils.js"; // ✔️ (file నిజానికి utils.ts)
import { helper } from "./utils"; // ✗ Error

// TS 5.7+ - "rewriteRelativeImportExtensions" తో .ts రాసి .js కి emit చేయొచ్చు
```

Barrel files — ఉపయోగం & ప్రమాదాలు

```
// features/user/index.ts (barrel)
export * from "./UserCard";
export * from "./useUser";
export type * from "./types";

// ✔️ లాభం: శుభ్రమైన imports - import { UserCard, useUser } from "@/features/user"
// ✗ ప్రమాదాలు:
// - Circular dependencies (barrel → file → barrel)
// - Tree-shaking విఫలం (side effects ఉంటే మొత్తం barrel load)
// - Build/IDE నెమ్మది (ఒక import మొత్తం feature ని load చేస్తుంది)
// - Vite dev లో గమనించదగిన నెమ్మది (వందల modules)
```

సిఫార్సు: public API boundaries లో (feature root) barrel OK; లోపల deep barrels నివారించండి. export * బదులు explicit re-exports మేలు.

Namespaces (legacy — వాడొద్దు)

```
// ✗ పాత విధానం (ES modules కి ముందు)
namespace MyApp {
  export namespace Utils { export function helper() {} }
}
MyApp.Utils.helper();
```

```
// ✔ అధునికం - ES modules
export function helper() {}
```

Namespaces ఇంకా **.d.ts** files లో (global types organize చేయడానికి, ఉదా. **NodeJS.ProcessEnv**) మాత్రమే సముచితం.

Circular dependencies

```
// a.ts
import { b } from "./b";
export const a = () => b();

// b.ts
import { a } from "./a"; // ⚠ circular!
export const b = () => a();

// Types మాత్రమే అయితే - import type తో సురక్షితం ✔
import type { AType } from "./a";
```

పరిష్కారాలు: types ని వేరే file లోకి (types.ts), dependency inversion, **import type** , madge/dpdm తో detect చేయడం.

Gotchas (సాధారణ తప్పులు)

- **import type వాడకపోవడం** → circular deps, bundle bloat, transpiler errors.
- **isolatedModules లేకుండా bundler వాడటం** → runtime errors (re-exported types).
- **NodeNext లో .js extension మర్చిపోవడం** → module not found.
- **export default overuse** – refactor/rename tooling బలహీనం, auto-import inconsistent. **Named exports మేలు.**
- **allowSyntheticDefaultImports / esModuleInterop గందరగోళం** – CJS libraries ని ESM లో import చేసేటప్పుడు.
- **Deep barrel chains** → build/IDE నెమ్మది.

Key Points

- **import type** = runtime నుండి పూర్తిగా తొలగింపు (circular deps + bundle కి కీలకం).
- **isolatedModules + verbatimModuleSyntax** = అధునిక bundler-friendly setup.
- **moduleResolution: "bundler"** (frontend) vs **"NodeNext"** (Node, **.js** extensions).
- Barrels ని పరిమితంగా; namespaces legacy.
- Named exports > default exports.

Interview దృష్టి

- **"import type ఎందుకు?"** → erasure guarantee, circular deps, transpiler isolation.

- "*isolatedModules* ఏం *enforce* చేస్తుంది?" → per-file transpilability; re-exported types explicit ఉండాలి.
-

Part 3 — Type System Semantics

21. Structural Typing vs Nominal Typing

వివరణ

TypeScript structural typing వాడుతుంది ("duck typing"): "అది బాతులా నడిస్తే, బాతులా అరిస్తే — అది బాతే." పేరు కాదు, shape ముఖ్యం.

```
interface Point { x: number; y: number }

class Vector { constructor(public x: number, public y: number) {} }

const p: Point = new Vector(1, 2);           // ✅ Vector "implements Point" అని ఎక్కడా చెప్పలేదు!
const q: Point = { x: 1, y: 2, z: 3 };       // ⚠️ variable ద్వారా అయితే OK (excess property check bypass)

// Java/C# (nominal) లో: class Vector implements Point అని *explicit* గా చెప్పాలి
```

Real-life Scenario

Structural = ఉద్యోగ interview లో skills చూడటం. "నాకు React వచ్చు, TypeScript వచ్చు" అని నిరూపిస్తే చాలు — ఏ college నుండి వచ్చారనేది ముఖ్యం కాదు.

Nominal = "మా college వాళ్ళకి మాత్రమే ఉద్యోగం." Skills ఉన్నా certificate పేరు match కాకపోతే కుదరదు.

Structural flexible, కానీ కొన్నిసార్లు ప్రమాదకరం — UserId మరియు ProductId రెండూ **string** అయితే, TypeScript వాటిని కలిపేస్తుంది!

Structural typing యొక్క ప్రమాదం

```
type UserId = string;
type ProductId = string;

function getUser(id: UserId) {}
const productId: ProductId = "prod_123";
getUser(productId);           // ✅ TS అనుమతిస్తుంది! — కానీ ఇది bug ❗

// అదే number లతో — మరింత ప్రమాదకరం
function transfer(amountInRupees: number, accountId: number) {}
transfer(accountId, amount);  // ✅ compile OK — arguments తారుమారు! ❗❗
```

✅ పరిష్కారం — Branded (Nominal/Opaque) Types

```
// Brand = type కి ఒక "అదృశ్య tag" జోడించడం (runtime లో ఉండదు)
declare const brand: unique symbol;
```

```

type Brand<T, B> = T & { readonly [brand]: B };

type UserId = Brand<string, "UserId">;
type ProductId = Brand<string, "ProductId">;
type Email = Brand<string, "Email">;
type PositiveInt = Brand<number, "PositiveInt">;

// Constructor functions (validation తో – ఇదే అసలు విలువ)
function toUserId(id: string): UserId {
    if (!id.startsWith("usr_")) throw new Error("Invalid user id");
    return id as UserId;
}
function toEmail(s: string): Email {
    if (!/^[^@\\s]+@[^@\\s]+\\. [^@\\s]+$/ .test(s)) throw new Error("Invalid email");
    return s as Email;
}

function getUser(id: UserId) {}
getUser("usr_1"); // ✗ Error – plain string అనుమతించబడదు ✓
getUser(toUserId("usr_1")); // ✓
getUser(productId); // ✗ Error ✓ – bug compile time లోనే పట్టుబడింది ⚡

// Branded types normal ops కి పని చేస్తాయి (T & {...} కాబట్టి)
const id = toUserId("usr_1");
id.toUpperCase(); // ✓ ఇంకా string

```

వాస్తవిక ఉపయోగాలు: IDs (UserId/OrderId), validated inputs (Email, URL, NonEmptyString), units (Meters vs Feet, Rupees vs Dollars), states (SanitizedHtml vs RawHtml – XSS నివారణ!), tokens (AccessToken vs RefreshToken).

```

// Security ఉదాహరణ – sanitization ని type system తో enforce చేయడం
type RawHtml = Brand<string, "RawHtml">;
type SafeHtml = Brand<string, "SafeHtml">;

function sanitize(html: RawHtml): SafeHtml { return DOMPurify.sanitize(html) as SafeHtml; }
function render(html: SafeHtml) { el.innerHTML = html; }

render(userInput as RawHtml); // ✗ Error – sanitize చేయకుండా render చేయలేం ⚡
render(sanitize(userInput as RawHtml)); // ✓

```

Classes – structural typing కి పాక్షిక మినహాయింపు

```

class A { private x = 1 }
class B { private x = 1 }
const a: A = new B(); // ✗ Error! – private members "nominal-ish" ప్రవర్తన కలిగిస్తాయి

class C { public x = 1 }
class D { public x = 1 }
const c: C = new D(); // ✓ structural

```

Empty interface / {} ఉచ్చు

```
interface Empty {}  
const e: Empty = "hello"; // ✓ !? - string కి Empty కి కావలసినవన్నీ (ఏమీ లేవు) ఉన్నాయి  
const e2: Empty = 42; // ✓  
const e3: Empty = null; // ✗ (null/undefined మాత్రమే కాదు)
```

Gotchas (సాధారణ తప్పులు)

- String/number aliases ని type safety అనుకోవడం – `type UserId = string` ఏ రక్షణా ఇవ్వదు.
- Excess property check మీద ఆధారపడటం – variable ద్వారా bypass అవుతుంది.
- Brand types ని runtime validation లేకుండా **as** తో create చేయడం – brand యొక్క సగం విలువ పోతుంది.
- {} ని "empty object" అనుకోవడం.
- Branded types ని అతిగా వాడటం – ప్రతి string కి brand అనవసరం; IDs, validated values, units కి మాత్రమే.

Key Points

- TypeScript = **structural** (shape matters, name doesn't).
- Aliases (`type UserId = string`) ఏ రక్షణా ఇవ్వవు.
- **Branded types** = nominal typing simulation (zero runtime cost).
- Brand + validation constructor = "parse, don't validate" pattern.
- **private** members classes ని nominal-ish చేస్తాయి.

Interview దృష్టి

- "Structural vs nominal typing?" → ఉదాహరణ + TS ఏది + ప్రమాదం.
- "రెండు string IDs కలవకుండా ఎలా ఆపుతావు?" → branded types – SSE-level answer.

22. Variance — Covariance, Contravariance, Bivariance

వివరణ

Variance = "A అనేది B కి subtype అయితే, $F<A>$ అనేది F కి subtype అ?" అనే ప్రశ్నకు సమాధానం.

Dog **extends** Animal అనుకుందాం.

Covariant	: <code>Dog[] < Animal[]</code>	(అదే దిశ) – arrays, return types
Contravariant	: <code>(Animal) => void < (Dog) => void</code>	(వ్యతిరేక దిశ) – parameters
Bivariant	: రెండు దిశలూ (unsafe) – method parameters	
Invariant	: ఏ దిశా కాదు – mutable generics (ideal గా)	

Real-life Scenario

Covariance (return types) = "నేను జంతువు ఇస్తాను" అని promise చేసి కుక్కను ఇవ్వడం. ✅ సమస్య లేదు – కుక్క ఒక జంతువే, ఎదుటివారు సంతృప్తి చెందుతారు.

Contravariance (parameters) = "నేను కుక్కల్ని చూసుకుంటాను" అన్న చోట "నేను ఏ జంతువునైనా చూసుకుంటాను" అన్న వ్యక్తిని పెట్టడం. ✅ సురక్షితం – అతను ఎక్కువ చేయగలడు.

కానీ వ్యతిరేకం ప్రమాదకరం – "ఏ జంతువునైనా చూసుకుంటాను" అన్న చోట "కుక్కల్ని మాత్రమే చూసుకుంటాను" అన్నవాడిని పెడితే – పిల్లి వచ్చినప్పుడు విఫలం! ❌

Covariance — return types & arrays

```
class Animal { name = "" }
class Dog extends Animal { bark() {} }

// Return type covariance ✅ సురక్షితం
type AnimalFactory = () => Animal;
const dogFactory: AnimalFactory = () => new Dog(); // ✅ Dog ఒక Animal

// Array covariance ⚠️ UNSOUND (TypeScript ఉద్దేశపూర్వకంగా అనుమతిస్తుంది)
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs; // ✅ compile OK
animals.push(new Animal()); // ✅ compile OK
dogs[1].bark(); // ❌ runtime crash – Animal కి bark లేదు!
```

ఇది TypeScript యొక్క ప్రసిద్ధ ఉద్దేశపూర్వక unsoundness. Java లో కూడా ఇదే (ArrayStoreException). Sound చేస్తే చాలా valid JS code reject అవుతుంది.

Contravariance — function parameters (strictFunctionTypes)

```
type DogHandler = (d: Dog) => void;
type AnimalHandler = (a: Animal) => void;

const handleAnimal: AnimalHandler = (a) => console.log(a.name);
const handleDog: DogHandler = (d) => d.bark();

let dh: DogHandler = handleAnimal; // ✅ సురక్షితం (contravariance)
let ah: AnimalHandler = handleDog; // ❌ Error with strictFunctionTypes ✅
// ఎందుకు error: ah(new Cat()) call చేస్తే → handleDog(cat) → cat.bark() ❌
```

Bivariance — methods యొక్క ఉచ్చు (🌟 interview favourite)

```
interface A {
  handle(x: Dog): void; // method shorthand → BIVARIANT (unsafe!)
}
interface B {
  handle: (x: Dog) => void; // property syntax → CONTRAVARIANT (safe ✅)
}
```

```
const a: A = { handle: (x: Animal) => {} }; // ✅ (bivariance అనుమతిస్తుంది)
const a2: A = { handle: (x: Bulldog) => {} }; // ✅ ⚠️ unsafe! Bulldog కాని Dog వస్తే crash
const b: B = { handle: (x: Bulldog) => {} }; // ❌ Error ✅ సరైనది
```

నియమం: strictFunctionTypes **method shorthand** కి వర్తించదు (backward compatibility కోసం – Array.prototype వంటివి పని చేయాలని). **Callbacks** కి **property syntax** వాడండి – అది type-safe.

strictFunctionTypes – ఏం చేస్తుంది

```
{ "strict": true } // → strictFunctionTypes: true
```

Function **type positions** లో parameters ని contravariantly check చేస్తుంది. **Method declarations** ని **bivariant** గానే వదిలేస్తుంది.

Variance annotations (TS 4.7+) – in / out

```
// Compiler కి variance ని explicit గా చెప్పడం (performance + correctness)
interface Producer<out T> { get(): T } // covariant (T output లో మాత్రమే)
interface Consumer<in T> { set(value: T): void } // contravariant (T input లో మాత్రమే)
interface Box<in out T> { get(): T; set(v: T): void } // invariant (రెండూ)

// లాభం: పెద్ద codebases లో variance inference ఖరీదైనది – annotations దాన్ని వేగవంతం చేస్తాయి
```

ఆచరణలో ఎక్కడ ముఖ్యం

```
// 1. Event handler types
type Handler<E> = (event: E) => void;
const onAny: Handler<Event> = (e) => {};
const onClick: Handler<MouseEvent> = onAny; // ✅ contravariance సురక్షితం

// 2. Readonly arrays – covariance ని సురక్షితం చేస్తాయి ⭐
function process(animals: readonly Animal[]) {
  // animals.push(...) ❌ – mutation సాధ్యం కాదు → covariance సురక్షితం
}
const dogs: Dog[] = [];
process(dogs); // ✅ ఇప్పుడు నిజంగా సురక్షితం

// 3. Promise covariance
const p: Promise<Animal> = Promise.resolve(new Dog()); // ✅

// 4. React props – component types
type Comp<P> = (props: P) => JSX.Element; // P input → contravariant
```

Gotchas (సాధారణ తప్పులు)

- Array covariance ని సురక్షితం అనుకోవడం → runtime crashes; `readonly T[]` వాడాలి.

- **Method shorthand తో callbacks define చేయడం** → bivariance hole.
- **strictFunctionTypes off** → అన్ని function params bivariant (unsafe).
- **Generic mutable containers ని covariant అనుకోవడం** — `Box<Dog> ≠ Box<Animal>` (ఉండకూడదు).
- **Variance error messages అర్థంకావు** — *"Types of parameters 'x' and 'y' are incompatible"* — ఇది సాధారణంగా contravariance violation.

Key Points

- **Return types covariant, parameters contravariant** (సురక్షితం).
- **Arrays covariant = ఉద్దేశపూర్వక unsoundness; readonly** వాడితే సురక్షితం.
- **Method shorthand bivariant** (unsafe) vs **property syntax contravariant** (safe).
- `strictFunctionTypes` methods కి వర్తించదు.
- `in / out` annotations = explicit variance (TS 4.7+).

Interview దృష్టి

- *"Covariance vs contravariance?"* → analogy + code + TS ఎక్కడ unsound.
- *"Dog[] ని Animal[] కి assign చేయొచ్చా, సురక్షితమా?"* → చేయొచ్చు, సురక్షితం కాదు, `readonly` పరిష్కారం — బలమైన answer.
- *"Method vs property function types తేడా?"* → bivariance hole.

23. Widening, Narrowing, `as const`, `satisfies`

వివరణ — literal widening

```
const a = "hello";           // "hello" (literal type)
let b = "hello";             // string (widened – reassign కావచ్చు కాబట్టి)
let c = a;                   // string (widening at assignment)

const obj = { name: "x" };   // { name: string } – properties mutable కాబట్టి widen
```

`as const` — widening ని ఆపడం

```
const config = {
  api: "https://api.com",
  timeout: 5000,
  methods: ["GET", "POST"],
} as const;
// {
//   readonly api: "https://api.com";
//   readonly timeout: 5000;
//   readonly methods: readonly ["GET", "POST"];
// }
```

```
// అన్ని levels లో deep readonly + literal ✔️
config.timeout = 10; // ❌ Error
config.methods.push("PUT"); // ❌ Error

// ముఖ్య ఉపయోగాలు
const ROUTES = ["/home", "/about"] as const;
type Route = (typeof ROUTES)[number]; // "/home" | "/about" ★

function req(method: "GET" | "POST") {}
const opts = { method: "GET" } as const;
req(opts.method); // ✔️ (as const లేకపోతే string → error)

// Tuple inference
const pair = [1, "a"]; // (string | number)[]
const tuple = [1, "a"] as const; // readonly [1, "a"] ✔️
```

satisfies operator (TS 4.9) – ★ ఆధునిక TypeScript యొక్క అత్యుత్తమ feature

సమస్య: type annotation ఇస్తే validation వస్తుంది కానీ specific inference పోతుంది.

```
type Config = Record<string, string | number[]>;

// ❌ Annotation – validation ✔️ కానీ inference పోయింది
const c1: Config = { host: "localhost", ports: [80, 443] };
c1.host.toUpperCase(); // ❌ Error – c1.host: string | number[]
c1.ports.map(x => x); // ❌ Error

// ❌ Annotation లేదు – inference ✔️ కానీ validation లేదు
const c2 = { host: "localhost", ports: [80, 443] };
c2.host.toUpperCase(); // ✔️
// కానీ typo/invalid shape పట్టుబడదు

// ✔️ satisfies – రెండూ! validation + precise inference ★
const c3 = { host: "localhost", ports: [80, 443] } satisfies Config;
c3.host.toUpperCase(); // ✔️ host: string
c3.ports.map(p => p * 2); // ✔️ ports: number[]
const bad = { host: true } satisfies Config; // ❌ Error ✔️ validation పని చేస్తోంది
```

satisfies వాస్తవిక ఉపయోగాలు

```
// 1. Theme/design tokens
const theme = {
  colors: { primary: "#0070f3", danger: "#ef4444" },
  spacing: { sm: 4, md: 8, lg: 16 },
} satisfies Record<string, Record<string, string | number>>;

theme.colors.primary.toUpperCase(); // ✔️ string అని తెలుసు
type Spacing = keyof typeof theme.spacing; // "sm" | "md" | "lg" ✔️

// 2. Route definitions
const routes = {
  home: { path: "/", auth: false },
  admin: { path: "/admin", auth: true },
```

```

} satisfies Record<string, { path: string; auth: boolean }>;
type RouteName = keyof typeof routes; // "home" | "admin" ✅

// 3. Exhaustive key checking
type Status = "idle" | "loading" | "error";
const labels = {
  idle: "Ready",
  loading: "Loading...",
  error: "Failed",
} satisfies Record<Status, string>;
// కొత్త Status add చేస్తే → ఇక్కడ compile error ⚡ (మర్చిపోవడం అసాధ్యం)

// 4. as const + satisfies కలిపి (ultimate combo)
const config = {
  env: "production",
  retries: 3,
} as const satisfies { env: "development" | "production"; retries: number };
// literal types ✅ + validation ✅ + readonly ✅

```

as (type assertion) – ఎప్పుడు, ఎందుకు ప్రమాదకరం

```

const el = document.getElementById("x") as HTMLInputElement; // "నన్ను నమ్ము"
el.value; // ✅ compile – కానీ ఆ element నిజంగా input కాకపోతే ❗

// `as` = type checking ని bypass చేయడం. TS ఏ verification చేయదు.
const n = "hello" as unknown as number; // double assertion – ఏదైనా సాధ్యం 🤖

// ✅ మేల్కొన ప్రత్యామ్నాయాలు
const el2 = document.getElementById("x");
if (el2 instanceof HTMLInputElement) { el2.value; } // ✅ runtime check

// ✅ satisfies (validation ఇస్తుంది)
const conf = {...} satisfies Config; // `as Config` కంటే ఎప్పుడూ మేలు

```

	as	satisfies	annotation (: T)
Type check జరుగుతుందా	❌	✅	✅
Inference నిలుస్తుందా	(forces type)	✅	❌ (widens to T)
ఎప్పుడు	చివరి ఎంపిక	✅ objects/configs	function params, explicit contracts

Non-null assertion !

```

const el = document.getElementById("x")!; // "ఇది null కాదని నాకు తెలుసు"
el.click(); // null అయితే ❗

// ✅ మేల్కొనవి
const el2 = document.getElementById("x");
if (!el2) throw new Error("Element not found"); // explicit + మంచి error message
el2.click();

// లేదా assertion function

```

```
function assertExists<T>(v: T, msg?: string): asserts v is NonNullable<T> {
  if (v == null) throw new Error(msg ?? "Value is null/undefined");
}
```

Gotchas

- **as** ని type conversion అనుకోవడం – అది కేవలం compiler ని ఒప్పించడం; runtime లో ఏమీ మారదు.
- **as const** ని function arguments కి మర్చిపోవడం → literal types పోతాయి.
- **as const** deep readonly – తర్వాత mutate చేయాల్సి వస్తే `Mutable<T>` అవసరం.
- **satisfies** ని annotation స్థానంలో అన్నిచోట్లా – function parameters కి annotation కావాలి.
- **!** overuse – `strictNullChecks` యొక్క విలువ పోతుంది; lint rule తో నిషేధించొచ్చు.

Key Points

- **const** → literal; **let** → widened; objects properties widen అవుతాయి.
- **as const** = deep readonly + literal types (tuple inference కూడా).
- **satisfies** = validation + inference రెండూ ★ (configs/themes/routes కి default ఎంపిక).
- **as** = unchecked assertion (చివరి ఎంపిక); **!** = non-null assertion (ప్రమాదకరం).

Interview దృష్టి

- "**satisfies** ఎందుకు వచ్చింది?" → annotation validation ఇస్తుంది కానీ inference చంపుతుంది; **satisfies** రెండూ ఇస్తుంది – TS 4.9+ తెలిసినట్టు చూపిస్తుంది.
- "**as** vs **satisfies** ?" → unchecked vs checked.

24. Assignability & Excess Property Checking

వివరణ – assignability నియమాలు

"S ని T కి assign చేయొచ్చా?" – TypeScript ప్రతి చోటా ఇదే ప్రశ్న అడుగుతుంది.

```
// Object: S కి T లోని *అన్ని* required members ఉండాలి (compatible types తో)
type T = { a: string; b?: number };
const s1 = { a: "x" }; // ✓
const s2 = { a: "x", b: 1, c: true }; // ✓ (variable ద్వారా – extra properties OK)
const s3 = { b: 1 }; // ✗ a లేదు
```

Assignability ని sets గా ఆలోచించడం (mental model)

Type = ఆ type కి చెందిన values యొక్క set

"S assignable to T" $\Leftrightarrow S \subseteq T$ (S యొక్క ప్రతి value T లో ఉంది)

never = ∅ (ఖాళీ set) → అన్నిటికీ assignable (bottom type)

`unknown` = అన్ని `values` → ఏదీ దీనికి assignable కానిది లేదు (top `type`)
`any` = నియమాలకు వెలుపల → రెండు దిశలా assignable (`escape hatch`)

`"hello" ⊆ string ⊆ unknown`

`{a,b} ⊆ {a}` ← ఎక్కువ `properties` = *చిన్న* `set`!

గుర్తుంచుకోండి: objects లో ఎక్కువ `properties` = ఇరుకైన (narrower) `type`. `{a: string, b: number}` అనేది `{a: string}` కి **subtype**. ఇది intuition కి తలకిందులుగా అనిపిస్తుంది కానీ సరైనదే — ఎక్కువ `constraints` = తక్కువ satisfying values.

Excess Property Checking (freshness)

```
interface Options { width?: number; height?: number }

// ✅ Fresh object literal → excess property check జరుగుతుంది
const a: Options = { width: 10, colour: "red" }; // ❌ Error: 'colour' does not exist
// ↑ typo పట్టుబడింది ⚡

// ❌ Variable ద్వారా → freshness పోయింది → check లేదు
const raw = { width: 10, colour: "red" };
const b: Options = raw; // ✅ No error

// Function arguments కి కూడా వర్తిస్తుంది
function setup(o: Options) {}
setup({ width: 1, colour: "red" }); // ❌ Error (fresh)
setup(raw); // ✅ No error

// Bypass మార్గాలు (కావాలనుకుంటే)
const c: Options = { width: 1, colour: "red" } as Options; // ❌ ఇప్పటికీ error
వస్తుంది కొన్ని cases లో
const d: Options & { colour: string } = { width: 1, colour: "red" }; // ✅ explicit
interface Options2 extends Options { [key: string]: unknown } // ✅ index signature
```

ఎందుకు ఈ ప్రత్యేక నియమం: structural typing ప్రకారం extra properties సమస్య కాదు. కానీ **object literal నేరుగా రాసినప్పుడు types చాలా సాధారణం** — అందుకే TypeScript ఒక ప్రత్యేక "freshness" heuristic జోడించింది.

Union assignability — ముఖ్యమైన సూక్ష్మత

```
type A = { kind: "a"; x: number } | { kind: "b"; y: string };

// Fresh literal → ఏదో ఒక member కి *ఖచ్చితంగా* match కావాలి
const a1: A = { kind: "a", x: 1 }; // ✅
const a2: A = { kind: "a", x: 1, y: "z" }; // ❌ excess property
const a3: A = { kind: "a", x: 1, extra: true }; // ❌
```

Function assignability

```
// తక్కువ parameters ఉన్న function ఎక్కువ parameters ఉన్న చోట పని చేస్తుంది ✅
type Cb = (a: string, b: number) => void;
```

```
const f1: Cb = () => {}; // ✓ (extra args ignore చేయొచ్చు - JS లానే)
const f2: Cb = (a) => {}; // ✓
const f3: Cb = (a, b, c) => {}; // ✗ ఎక్కువ params అడగలేం

// ఇదే `arr.map(x => x)` పని చేయడానికి కారణం (map 3 args ఇస్తుంది, మనం 1 తీసుకుంటాం)
```

Optional vs undefined (exactOptionalPropertyTypes)

```
// Default (flag off)
type T1 = { a?: string };
const x: T1 = { a: undefined }; // ✓ అనుమతిస్తుంది

// exactOptionalPropertyTypes: true
const y: T1 = { a: undefined }; // ✗ Error - "a లేకపోవడం" ≠ "a = undefined"
const z: T1 = {}; // ✓

// ఎందుకు ఉపయోగం: `"a" in obj` checks, JSON serialization, DB updates లో తేడా ముఖ్యం
```

Gotchas

- **Excess property check ని type safety అనుకోవడం** – అది కేవలం fresh literals కి heuristic.
- **Variable extract చేసి check ని bypass చేయడం** (అనుకోకుండా) – typos దాగుతాయి.
- **Return type లో excess properties** – check జరగదు:

```
function f(): Options { return { width: 1, colour: "x" }; } // ✗ Error (fresh) ✓
const o = { width: 1, colour: "x" };
function g(): Options { return o; } // ✓ no error
```

- **any assignability** – రెండు దిశలా; type system లో "రంధ్రం".
- **Optional property + undefined గందరగోళం** – exactOptionalPropertyTypes స్పష్టం చేస్తుంది.

Key Points

- Assignability = **subset relation** (types = value sets).
- Objects: **ఎక్కువ properties = narrower type**.
- **Excess property checking** = fresh object literals కి మాత్రమే (typo safety net).
- Functions: తక్కువ params ✓, ఎక్కువ params ✗.
- exactOptionalPropertyTypes = missing vs undefined తేడా.

Interview దృష్టి

- "ఈ code లో error ఎందుకు రాలేదు?" (variable ద్వారా extra props) → freshness explanation.
- "Types ని sets గా వివరించు" → never = ∅, unknown = universe, subtyping = subset.

25. any VS unknown VS never — Top & Bottom Types

వివరణ

```
unknown ← TOP type (అన్ని values ఇందులో ఉన్నాయి)
/   |   \
string number object ...
\   |   /
never ← BOTTOM type (ఏ value లేదు, ∅)
```

any = ఈ hierarchy కి వెలుపల – అన్ని నియమాలను bypass చేస్తుంది

	any	unknown	never
దీనికి ఏం assign చేయొచ్చు	అన్నీ	అన్నీ	ఏదీ కాదు
దీన్ని దీనికి assign చేయొచ్చు	అన్నిటికీ	unknown / any కి మాత్రమే	అన్నిటికీ
Property access	✅ (unchecked)	❌ (narrow చేయాలి)	✅ (theoretically)
ఎప్పుడు	migration, escape	unknown inputs ✅	impossible, exhaustiveness

Real-life Scenario

any = "ఏమైనా సరే, నన్ను ఆపకు" అనే VIP pass. Security check లేకుండా ఎక్కడికైనా వెళ్ళొచ్చు – కానీ తప్పు గదిలోకి వెళ్ళి ప్రమాదం తెచ్చుకుంటారు.

unknown = "ID చూపించే వరకు లోపలికి రానివ్వను." ఎవరైనా రావొచ్చు కానీ నిరూపించాలి (narrowing).

never = "ఈ గది ఉనికిలోనే లేదు." ఎవరూ అక్కడ ఉండలేరు.

never ఎక్కడ కనిపిస్తుంది

```
// 1. Function ఎప్పుడూ return చేయదు
function fail(msg: string): never { throw new Error(msg); }
function forever(): never { while (true) {} }

// 2. Impossible types
type A = string & number; // never
type B = Exclude<"a", "a">; // never

// 3. Exhaustiveness checking ★
function handle(s: Shape) {
  switch (s.kind) {
    case "circle": return 1;
    default: const _: never = s; throw new Error("unreachable");
  }
}

// 4. Narrowing పూర్తిగా అయిపోయినప్పుడు
function f(x: string) {
```

```

    if (typeof x === "string") {}
    else { x; }          // x: never
  }

// 5. Empty arrays
const arr = [];          // never[] (noImplicitAny తో)

// 6. Mapped types లో keys filter చేయడం
type OnlyStrings<T> = { [K in keyof T as T[K] extends string ? K : never]: T[K] };

// 7. Conditional types లో "no match"
type ElementOf<T> = T extends (infer E)[] ? E : never;

```

unknown — సరైన వాడకం

```

// 1. API responses
async function fetchData(url: string): Promise<unknown> {
  return (await fetch(url)).json();
}
const data = await fetchData("/api");
// data.name;          // ❌ narrow చేయాలి
const user = UserSchema.parse(data);          // ✅ Zod తో validate

// 2. catch blocks (useUnknownInCatchVariables – strict లో default)
try { risky(); }
catch (e) {          // e: unknown
  if (e instanceof ZodError) handleValidation(e);
  else if (e instanceof ApiError) handleApi(e);
  else if (e instanceof Error) log(e.message);
  else log("Unknown error:", String(e));
}

// 3. Generic constraints కి కంటే మేలు
function log(value: unknown) { console.log(JSON.stringify(value)); } // any అవసరం లేదు

// 4. Type guards తో
function isRecord(v: unknown): v is Record<string, unknown> {
  return typeof v === "object" && v !== null && !Array.isArray(v);
}

```

any — ఎప్పుడు ఆమోదయోగ్యం (మరియు ఎలా నియంత్రించాలి)

```

// ✅ ఆమోదయోగ్యం:
// 1. JS → TS migration (తాత్కాలికం, TODO comment తో)
// 2. అత్యంత dynamic code (deep clone, generic serializers) – అప్పుడు కూడా unknown ప్రయత్నించాలి
// 3. Third-party library types తప్పగా ఉంటే (workaround + issue file చేయడం)

// ✅ నియంత్రణ – ESLint తో
// "@typescript-eslint/no-explicit-any": "error"
// "@typescript-eslint/no-unsafe-assignment": "error"
// "@typescript-eslint/no-unsafe-member-access": "error"
// "@typescript-eslint/no-unsafe-call": "error"
// "@typescript-eslint/no-unsafe-return": "error"

```

```
// ✅ Escape hatch ని కనిపించేలా చేయడం
type TODO = any; // grep చేయొచ్చు, tech debt track చేయొచ్చు
const data: TODO = legacyApi();
```

any యొక్క దాగిన ప్రమాదం – infection

```
const config: any = loadConfig();
const timeout = config.timeout; // any
const ms = timeout * 1000; // any
setTimeout(fn, ms); // ✅ compile – కానీ timeout "5s" అయితే NaN ✖

// ⚠️ any ఒక్కసారి ప్రవేశిస్తే మొత్తం call chain లో type safety పోతుంది
```

void vs never vs undefined

```
function a(): void {} // return చేస్తుంది కానీ value లేదు
function b(): undefined { return undefined; } // explicitly undefined return
function c(): never { throw new Error(); } // ఎప్పుడూ return కాదు

// Assignability
const v: void = undefined; // ✅
const n: never = undefined; // ❌ (never కి ఏమీ assign కాదు)
```

Gotchas

- **any** ని unknown బదులు వాడటం – safety పోతుంది.
- unknown narrow చేయకుండా **as** తో cast చేయడం – **any** కంటే మెరుగేమీ కాదు.
- **never[]** accidental – `const arr = []` (annotate చేయాలి).
- **any** conditional types లో both-branch union ఇవ్వడం:

```
type IsString<T> = T extends string ? "yes" : "no";
type X = IsString<any>; // "yes" | "no" 🤔
```

- **never** ని return type గా పొరపాటున పొందడం – function లో అన్ని paths throw చేస్తే.
- unknown ని `JSON.stringify` లో – పని చేస్తుంది కానీ output shape తెలియదు.

Key Points

- unknown = safe **any** – narrowing తప్పనిసరి; API/catch కి default.
- never = bottom type – impossible; exhaustiveness checking కి కీలకం.
- **any** = type system bypass – infection risk; ESLint తో నియంత్రించాలి.
- unknown top type, never bottom type, **any** hierarchy కి వెలుపల.

Interview దృష్టి

- "*any* vs *unknown* vs *never* ?" → assignability table + వాడకాలు (అత్యంత సాధారణ TS question).
 - "*never* ఎప్పుడు *useful*?" → exhaustiveness, impossible states, filtering in mapped types.
-

Part 4 — TypeScript in Practice

26. TypeScript + React

వివరణ

React_Telugu.md Topic 51 లో basics ఉన్నాయి; ఇక్కడ **deep + advanced patterns**.

Props typing — అన్ని విధానాలు

```
// 1. Basic
type ButtonProps = {
  label: string;
  variant?: "primary" | "danger";
  onClick: (e: React.MouseEvent<HTMLButtonElement>) => void;
  children?: React.ReactNode;
};

// 2. HTML attributes extend చేయడం ✨ (wrapper components కి ఉత్తమం)
type Props = React.ComponentProps<"button"> & { variant: "primary" | "danger" };
// ComponentPropsWithoutRef<"button"> – ref లేకుండా
// ComponentPropsWithRef<"button"> – ref తో

function Button({ variant, className, ...rest }: Props) {
  return <button className={` ${variant} ${className} ?? ""`} {...rest} />;
}

// 3. ఇంకో component నుండి props తీసుకోవడం
type IconProps = React.ComponentProps<typeof Icon>;
type IconName = React.ComponentProps<typeof Icon>["name"];

// 4. Discriminated union props ✨ (impossible combinations ని అపడం)
type AlertProps =
  | { variant: "error"; error: Error; onRetry: () => void }
  | { variant: "info"; message: string }
  | { variant: "success"; message: string; autoHide?: number };

function Alert(props: AlertProps) {
  switch (props.variant) {
    case "error": return <div>{props.error.message}<button onClick=
{props.onRetry}>Retry</button></div>;
    case "info":
    case "success": return <div>{props.message}</div>;
  }
}
<Alert variant="info" error={e} /> // ✖ Error ✔ – impossible combo అగింది

// 5. Children variants
children: React.ReactNode // ✔ default (అన్నీ)
children: React.ReactElement // ✖ element
```

```

children: (data: T) => React.ReactNode // render prop
children: React.ReactElement<TabProps>[] // నిర్దిష్ట children (fragile)

```

Hooks typing – సూక్ష్మతలు

```

// useState
const [user, setUser] = useState<User | null>(null); // ✅ explicit అవసరం
const [items, setItems] = useState<Item[]>([]); // ✅ లేకపోతే never[]
const [count, setCount] = useState(0); // ✅ inference సరిపోతుంది

// Lazy init typed
const [state] = useState<Config>(() => JSON.parse(localStorage.getItem("c") ?? "{}"));

// useRef – మూడు రూపాలు (తేడా ముఖ్యం!)
const inputRef = useRef<HTMLInputElement>(null); // RefObject – current readonly
const timerRef = useRef<number | null>(null); // MutableRefObject
const countRef = useRef(0); // MutableRefObject<number>

// useReducer – discriminated actions + exhaustive
type State = { count: number; history: number[] };
type Action =
  | { type: "increment"; by: number }
  | { type: "decrement"; by: number }
  | { type: "reset" };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "increment": return { count: state.count + action.by, history: [...state.history, state.count] };
    case "decrement": return { count: state.count - action.by, history: [...state.history, state.count] };
    case "reset": return { count: 0, history: [] };
    default: { const _x: never = action; return state; }
  }
}

// useContext – null-safe pattern ★
const AuthContext = React.createContext<AuthValue | null>(null);
export function useAuth() {
  const ctx = React.useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used within AuthProvider");
  return ctx; // ✅ ఇక్కడ నుండి AuthValue (null కాదు)
}

// Custom hook – tuple return కి `as const`
function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = useCallback(() => setOn((v) => !v), []);
  return [on, toggle] as const; // [boolean, () => void] ✅
}

```

Generic components

```
// Generic function component
type ListProps<T> = {
  items: readonly T[];
  keyExtractor: (item: T) => string;
  renderItem: (item: T, index: number) => React.ReactNode;
  emptyState?: React.ReactNode;
};

function List<T>({ items, keyExtractor, renderItem, emptyState }: ListProps<T>) {
  if (items.length === 0) return <{emptyState ?? "No items"}>;
  return <ul>{items.map((it, i) => <li key={keyExtractor(it)}>{renderItem(it, i)}</li>)}</ul>;
}

<List
  items={users}
  keyExtractor={u => u.id} // u: User ✅ inferred
  renderItem={u => <span>{u.email}</span>}
/>

// ⚠️ .tsx files లో arrow generic - comma అవసరం (JSX లో గందరగోళం నివారణ)
const List2 = <T>,(props: ListProps<T>) => { /* ... */ };
// ↑ ఈ comma తప్పనిసరి

// Polymorphic component (`as` prop) - advanced
type PolymorphicProps<E extends React.ElementType> = {
  as?: E;
  children?: React.ReactNode;
} & Omit<React.ComponentPropsWithoutRef<E>, "as" | "children">;

function Box<E extends React.ElementType = "div">({ as, ...rest }: PolymorphicProps<E>) {
  const Component = as ?? "div";
  return <Component {...rest} />;
}

<Box as="a" href="/x" /> // ✅ href allowed
<Box as="div" href="/x" /> // ❌ Error ✅
```

Event & DOM types

```
React.MouseEvent<HTMLButtonElement>
React.ChangeEvent<HTMLInputElement> // e.target.value: string
React.FormEvent<HTMLFormElement>
React.KeyboardEvent<HTMLInputElement>
React.FocusEvent<HTMLInputElement>
React.DragEvent<HTMLDivElement>
React.CSSProperties // style prop
React.ReactNode / ReactElement / JSX.Element
React.PropsWithChildren<P>
React.Dispatch<React.SetStateAction<T>> // setState ని prop గా పంపేటప్పుడు
```

```
// setState ని child కి పంపడం
```

```
type Props = { setCount: React.Dispatch<React.SetStateAction<number>> };
```

React 19 typing మార్పులు

```
// ✅ ref ఇప్పుడు సాధారణ prop – forwardRef అవసరం లేదు
type InputProps = React.ComponentProps<"input"> & { ref?: React.Ref<HTMLInputElement> };
function Input({ ref, ...props }: InputProps) { return <input ref={ref} {...props} />; }

// ✅ useActionState
const [state, action, pending] = useActionState<FormState, FormData>(submitFn, initialState);

// ⚠️ @types/react 19 లో: children ఇక implicit కాదు (React.FC నుండి తీసేశారు – ఇది 18 లోనే)
```

Gotchas

- **React.FC వాడటం** – generics awkward, implicit children (పాత versions), defaultProps support పోయింది → **plain function + typed props** వాడాలి.
- **.tsx లో <T> arrow generic** → JSX అనుకుంటుంది; **<T,>** రాయాలి.
- **useState([])** → never[] .
- **e.target vs e.currentTarget** – currentTarget మాత్రమే సరిగ్గా typed.
- **JSX.Element vs ReactNode** – props కి ReactNode (strings, arrays, null అనుమతిస్తుంది).
- **@types/react version mismatch** – React 19 తో @types/react 18 → గందరగోళ errors.
- **ComponentProps vs ComponentPropsWithoutRef** – spread చేసేటప్పుడు ref conflict.

Key Points

- **React.ComponentProps<"tag">** = native props extend చేయడానికి ఉత్తమం.
- **Discriminated union props** = impossible prop combos ని అపుతాయి.
- **Context**: createContext<T | null>(null) + throwing hook.
- **Generic components**: .tsx లో <T,> ; polymorphic as prop pattern.
- **React.FC** నివారించండి; **as const** tuple hooks కి.

Interview దృష్టి

- "Generic React component రాయ" → typed <List<T> /> + inference demo.
- "Props ని ఎలా type చేస్తావు?" → ComponentProps extend + discriminated unions.

27. TypeScript + Node/Express

విషయం — typed backend

```
// Express + TypeScript setup
import express, { Request, Response, NextFunction, RequestHandler } from "express";

// 1. Typed route handlers (generics: Params, ResBody, ReqBody, Query)
interface CreateUserBody { email: string; name: string }
interface UserParams { id: string }
interface ListQuery { page?: string; limit?: string }

app.post("/users", (req: Request<{}, User, CreateUserBody>, res: Response<User>) => {
  const { email, name } = req.body; // ✅ typed
  res.json(createUser(email, name));
});

app.get("/users/:id", (req: Request<UserParams>, res) => {
  const id = req.params.id; // ✅ string
});

// 2. Custom typed handler helper (ఎక్స్ప్రెస్ చదవగలిగేది)
type Handler<TBody = unknown, TParams = unknown, TRes = unknown> =
  (req: Request<TParams, TRes, TBody>, res: Response<TRes>, next: NextFunction) =>
  Promise<void> | void;

const createUser: Handler<CreateUserBody, {}, User> = async (req, res) => {
  const user = await db.user.create({ data: req.body });
  res.status(201).json(user);
};
```

Request augmentation (auth middleware)

```
// types/express.d.ts
declare global {
  namespace Express {
    interface Request {
      user?: { id: string; role: "admin" | "user" };
      requestId: string;
    }
  }
}
export {};
```

```
// middleware
const authenticate: RequestHandler = async (req, res, next) => {
  const token = req.headers.authorization?.replace("Bearer ", "");
  if (!token) { res.status(401).json({ error: "Unauthorized" }); return; }
  try {
    req.user = await verifyToken(token); // ✅ typed
    next();
  } catch {
    res.status(401).json({ error: "Unauthorized" });
  }
};
```

```

    } catch { res.status(401).json({ error: "Invalid token" }); }
  };

```

Environment variables — type-safe (అత్యంత ముఖ్యం)

```

// config/env.ts — startup లోనే validate చేయడం ✨
import { z } from "zod";

const envSchema = z.object({
  NODE_ENV: z.enum(["development", "production", "test"]).default("development"),
  PORT: z.coerce.number().int().positive().default(3000),
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
  REDIS_URL: z.string().url().optional(),
  LOG_LEVEL: z.enum(["debug", "info", "warn", "error"]).default("info"),
});

const parsed = envSchema.safeParse(process.env);
if (!parsed.success) {
  console.error("❌ Invalid environment variables:", parsed.error.flatten().fieldErrors);
  process.exit(1); // 🔑 fail fast — deploy అయిన 3 గంటల తర్వాత కాదు
}

export const env = parsed.data; // ✅ పూర్తిగా typed + validated
export type Env = typeof env;

// వాడకం
app.listen(env.PORT); // number ✅ (process.env.PORT string!)

```

Async error handling

```

// ❌ Express 4 లో async errors automatic గా పట్టుబడవు
app.get("/x", async (req, res) => { throw new Error("boom"); }); // unhandled rejection!

// ✅ Wrapper
const asyncHandler = <T extends RequestHandler>(fn: T): RequestHandler =>
  (req, res, next) => { Promise.resolve(fn(req, res, next)).catch(next); };

app.get("/x", asyncHandler(async (req, res) => { /* throw safe ✅ */ }));

// ✅ Typed error classes + central error middleware
class AppError extends Error {
  constructor(
    message: string,
    public statusCode: number = 500,
    public code: string = "INTERNAL_ERROR",
    public details?: unknown
  ) {
    super(message);
    this.name = this.constructor.name;
    Error.captureStackTrace?.(this, this.constructor);
  }
}

```

```

class NotFoundError extends AppError {
  constructor(resource: string) { super(`${resource} not found`, 404, "NOT_FOUND"); }
}
class ValidationError extends AppError {
  constructor(details: unknown) { super("Validation failed", 400, "VALIDATION_ERROR", details); }
}

const errorHandler = (err: unknown, req: Request, res: Response, _next: NextFunction) => {
  if (err instanceof AppError) {
    return res.status(err.statusCode).json({ error: err.code, message: err.message, details: err.details });
  }
  if (err instanceof z.ZodError) {
    return res.status(400).json({ error: "VALIDATION_ERROR", details: err.flatten() });
  }
  logger.error({ err, requestId: req.requestId });
  res.status(500).json({ error: "INTERNAL_ERROR" }); // ⚠ internal details leak చేయొద్దు
};
app.use(errorHandler); // 🔑 అన్ని routes తర్వాత

```

Request validation middleware (typed end-to-end)

```

function validate<TBody extends z.ZodTypeAny>(schema: TBody) {
  return (req: Request, res: Response, next: NextFunction) => {
    const result = schema.safeParse(req.body);
    if (!result.success) return next(new ValidationError(result.error.flatten()));
    req.body = result.data; // ✅ parsed + coerced data
    next();
  };
}

const createUserSchema = z.object({ email: z.string().email(), age: z.coerce.number().min(18) });
type CreateUserDto = z.infer<typeof createUserSchema>;

app.post("/users", validate(createUserSchema), asyncHandler(async (req, res) => {
  const dto = req.body as CreateUserDto; // ✅ validated
  res.json(await userService.create(dto));
}));

```

Layered architecture (typed)

```

// domain/user.ts – pure types & business rules
export interface User { id: UserId; email: Email; role: Role; createdAt: Date }
export type CreateUserInput = Omit<User, "id" | "createdAt">;

// repositories/userRepo.ts – data access interface (testable)
export interface UserRepository {
  findById(id: UserId): Promise<User | null>;
  findByEmail(email: Email): Promise<User | null>;
  create(input: CreateUserInput): Promise<User>;
}

```

```
// services/userService.ts – business logic (framework-agnostic ✅)
export class UserService {
  constructor(private repo: UserRepository, private mailer: Mailer) {}

  async register(input: CreateUserInput): Promise<User> {
    const existing = await this.repo.findByEmail(input.email);
    if (existing) throw new AppError("Email already registered", 409, "DUPLICATE_EMAIL");
    const user = await this.repo.create(input);
    await this.mailer.sendWelcome(user.email);
    return user;
  }
}
// ✅ Test లో: new UserService(fakeRepo, fakeMailer) – Express అవసరం లేదు
```

Node built-ins typing

```
import { readFile } from "node:fs/promises";
import type { Server } from "node:http";

// ⚠️ timers – Node vs browser
const t: ReturnType<typeof setTimeout> = setTimeout(() => {}, 100); // ✅ portable

// process.exit codes, signals
process.on("SIGTERM", async () => { await server.close(); process.exit(0); });

// tsconfig: "types": ["node"] + npm i -D @types/node
```

Gotchas

- `process.env` values ఎప్పుడూ `string` | `undefined` – PORT ని number అనుకోవడం classic bug.
- **Express async errors** – wrapper లేకపోతే unhandled rejections.
- **Error handler** ని routes కంటే ముందు `app.use` చేయడం → పని చేయదు.
- **Request augmentation** లో `export {}` మర్చిపోవడం.
- **Internal error details** ని client కి పంపడం – security leak.
- `@types/express` version mismatch (Express 5 తో).
- **Service layer** ని Express types మీద ఆధారపడేలా చేయడం – testability పోతుంది.

Key Points

- **Env vars** ని startup లో **Zod** తో **validate** చేసి fail-fast.
- Request augmentation (`declare global` + `namespace Express`).
- `asyncHandler` + typed `AppError` hierarchy + central error middleware.
- Validation middleware → `z.infer` తో end-to-end types.
- Business logic ని framework నుండి వేరు చేయడం (testability).

Interview దృష్టి

- "Env vars ఎలా type చేస్తావు?" → Zod validation at startup + **typeof** env — production maturity signal.
- "Async errors ఎలా handle చేస్తావు?" → wrapper/Express 5, error classes, central handler, no leaks.

28. Async, Promises & Error Handling Typing

వివరణ

```
// Promise types
async function getUser(id: string): Promise<User> { /* ... */ }
const p: Promise<User> = getUser("1");
type U = Awaited<ReturnType<typeof getUser>>; // User ★

// Promise combinators
const [a, b] = await Promise.all([getUser("1"), getPosts("1")]); // [User, Post[]] ✓ tuple
inference
const results = await Promise.allSettled([p1, p2]);
// PromiseSettledResult<T>[] — narrowing అవసరం
results.forEach((r) => {
  if (r.status === "fulfilled") console.log(r.value);
  else console.error(r.reason); // ⚠ reason: any (JS ఏదైనా throw చేయొచ్చు)
});

const first = await Promise.race([fetchData(), timeout(5000)]);
const any1 = await Promise.any([p1, p2]); // మొదటి fulfilled; అన్నీ fail → AggregateError
```

catch లో unknown — strict TypeScript

```
// useUnknownInCatchVariables: true (strict లో default)
try { await risky(); }
catch (error) { // error: unknown ✓
  // error.message; // ✗ Error
  if (error instanceof Error) console.error(error.message);
  else console.error("Unknown:", String(error));
}

// ✓ Reusable error normalizer
function toError(e: unknown): Error {
  if (e instanceof Error) return e;
  if (typeof e === "string") return new Error(e);
  try { return new Error(JSON.stringify(e)); } catch { return new Error("Unknown error"); }
}
```

Result type pattern (exceptions కి ప్రత్యామ్నాయం)

```
type Result<T, E = Error> =
  | { ok: true; value: T }
  | { ok: false; error: E };
```

```

const Ok = <T>(value: T): Result<T, never> => ({ ok: true, value });
const Err = <E>(error: E): Result<never, E> => ({ ok: false, error });

async function safeFetch<T>(url: string): Promise<Result<T, ApiError>> {
  try {
    const res = await fetch(url);
    if (!res.ok) return Err(new ApiError(res.status, res.statusText));
    return Ok((await res.json()) as T);
  } catch (e) {
    return Err(new ApiError(0, toError(e).message));
  }
}

// వాడకం - error handling ని *type system enforce చేస్తుంది* ✨
const result = await safeFetch<User>("/api/user");
if (result.ok) {
  console.log(result.value.name); // ✅ value ఇక్కడ మాత్రమే
} else {
  console.error(result.error.status); // ✅ error ఇక్కడ మాత్రమే
}
// result.value; ❌ Error - narrow చేయకుండా access చేయలేం ⚡

```

	Exceptions	Result type
Type visibility	❌ signature లో కనిపించదు	✅ return type లో
మర్చిపోవచ్చా	✅ (silent crash)	❌ compiler బలవంతం
Ergonomics	సులభం (try/catch)	verbose (ప్రతి చోటు check)
ఎప్పుడు	unexpected errors, infra failures	expected/domain errors (validation, not-found)

అచరణాత్మక సలహా: hybrid — **expected/domain errors** కి Result, **unexpected/programmer errors** కి exceptions.

Async patterns typed

```

// 1. Timeout wrapper
function withTimeout<T>(promise: Promise<T>, ms: number): Promise<T> {
  return Promise.race([
    promise,
    new Promise<never>((_, reject) =>
      setTimeout(() => reject(new Error(`Timeout after ${ms}ms`)), ms)
    ),
  ]);
}

// 2. Retry with exponential backoff
async function retry<T>(
  fn: () => Promise<T>,
  { retries = 3, delay = 1000, shouldRetry = () => true }: {
    retries?: number; delay?: number; shouldRetry?: (e: unknown) => boolean;
  }
) {
  let attempt = 0;
  while (true) {
    try {
      return await fn();
    } catch (e) {
      if (!shouldRetry(e)) throw e;
      await delay * Math.pow(2, attempt);
      attempt++;
    }
  }
}

```

```

    } = {}
  ): Promise<T> {
    let lastError: unknown;
    for (let i = 0; i <= retries; i++) {
      try { return await fn(); }
      catch (e) {
        lastError = e;
        if (i === retries || !shouldRetry(e)) break;
        await new Promise((r) => setTimeout(r, delay * 2 ** i));
      }
    }
    throw lastError;
  }

// 3. Concurrency limit (pool)
async function mapWithConcurrency<T, R>(
  items: readonly T[], limit: number, fn: (item: T, i: number) => Promise<R>
): Promise<R[]> {
  const results: R[] = new Array(items.length);
  let cursor = 0;
  const workers = Array.from({ length: Math.min(limit, items.length) }, async () => {
    while (cursor < items.length) {
      const i = cursor++;
      results[i] = await fn(items[i]!, i);
    }
  });
  await Promise.all(workers);
  return results;
}

// 4. Typed async generator
async function* paginate<T>(fetchPage: (cursor?: string) => Promise<{ items: T[]; next?: string }>):
  AsyncGenerator<T, void, undefined> {
  let cursor: string | undefined;
  do {
    const { items, next } = await fetchPage(cursor);
    yield* items;
    cursor = next;
  } while (cursor);
}
for await (const item of paginate(fetchUsers)) { /* item: User ✓ */ }

```

Gotchas

- **catch (e)** లో **e.message** — **e**: unknown , narrow చేయాలి.
- **Promise.all** ఒకటి fail అయితే మొత్తం **reject** → partial results కావాలంటే **allSettled** .
- **Floating promises** — **await** మర్చిపోవడం:

```

// ESLint: "@typescript-eslint/no-floating-promises": "error" ✓ తప్పనిసరి
saveData();           // ✗ unhandled rejection
void saveData();       // ✓ ఉద్దేశపూర్వకంగా అని చెప్పడం

```

- **async void callbacks** — errors swallow అవుతాయి (no-misused-promises rule).
- **Promise<void> vs void** — async function ని sync callback స్థానంలో ఇవ్వడం.
- **await in loops** — sequential (నెమ్మది); parallel కావాలంటే **Promise.all**.

Key Points

- **Awaited<T>** = nested promise unwrap.
- **catch** (e: **unknown**) → narrowing/normalizer తప్పనిసరి.
- **Result type** = expected errors ని type system లోకి తీసుకురావడం.
- Utilities: timeout, retry+backoff, concurrency pool, async generators — typed generics తో.
- ESLint: no-floating-promises , no-misused-promises తప్పనిసరి.

Interview దృష్టి

- "Errors ని ఎలా type చేస్తావు?" → unknown in catch, custom Error classes, Result type trade-offs.
- "Promise.all vs allSettled ?" → fail-fast vs partial + typed narrowing.

29. Runtime Validation — Zod & the Type Boundary

వివరణ

TypeScript యొక్క అతిపెద్ద అపోహ: "నా types ఉన్నాయి కాబట్టి data సరైనదే." ❌

Types compile time లో మాయమవుతాయి. **మీ program లోకి బయటి నుండి వచ్చే ప్రతిదీ అనుమానాస్పదం:**

API responses • localStorage • URL **params** • form inputs • env variables
webhooks • message queues • file contents • third-party SDKs • **user input**

```
// ❌ "నమ్మకం" మాత్రమే — TypeScript ఇక్కడ మౌనం
const user = await res.json() as User;
user.profile.email.toLowerCase(); // API `{}` పంపితే ✨
```

Real-life Scenario

Types = మీ ఇంటి లోపలి నియమాలు; validation = గేటు దగ్గర security. ఇంట్లో అందరూ చెప్పులు బయట వదులుతారని మీరు అనుకోవచ్చు (type annotation). కానీ **గేటు దగ్గర ఎవరూ తనిఖీ చేయకపోతే** ఎవరైనా చెప్పులతో లోపలికి వస్తారు. **Validation = గేటు దగ్గర తనిఖీ.** ఒకసారి లోపలికి వచ్చాక (validated), ఇంట్లో నియమాలు (types) నమ్మొచ్చు.

Zod — schema-first approach

```
import { z } from "zod";

// 1 Schema define (single source of truth) ★
const UserSchema = z.object({
  id: z.string().uuid(),
  email: z.string().email(),
  age: z.number().int().min(0).max(150),
  role: z.enum(["admin", "user", "guest"]),
  profile: z.object({
    bio: z.string().max(500).optional(),
    avatarUrl: z.string().url().nullable(),
  }),
  tags: z.array(z.string()).default([]),
  createdAt: z.coerce.date(), // string → Date automatic ★
});

// 2 Type ని schema నుండి derive చేయడం (duplicate రాయనవసరం లేదు!)
type User = z.infer<typeof UserSchema>;
// {
//   id: string; email: string; age: number; role: "admin"|"user"|"guest";
//   profile: { bio?: string; avatarUrl: string | null };
//   tags: string[]; createdAt: Date;
// }

// 3 Runtime validation
const user = UserSchema.parse(data); // ✖ invalid అయితే throws ZodError
const result = UserSchema.safeParse(data); // ✔ { success, data | error }
if (!result.success) {
  console.error(result.error.flatten().fieldErrors);
  // { email: ["Invalid email"], age: ["Number must be >= 0"] }
}
```

సూత్రం: "Parse, don't validate." Validate = "ఇది సరైనదా?" (boolean) → type మారదు. Parse = "దీన్ని సరైన rupam గా మార్చు" → కొత్త, నమ్మదగిన type వస్తుంది. Zod parse చేస్తుంది.

వాస్తవిక integration patterns

```
// 1. Typed API client – validation built-in ★
async function apiGet<T extends z.ZodTypeAny>(
  url: string, schema: T, init?: RequestInit
): Promise<z.infer<T>> {
  const res = await fetch(url, init);
  if (!res.ok) throw new ApiError(res.status, await res.text());
  const json = await res.json();
  const parsed = schema.safeParse(json);
  if (!parsed.success) {
    // ⚠ ఇది monitoring కి పంపాలి – API contract మారినదని సంకేతం!
    logger.error("API contract violation", { url, issues: parsed.error.issues });
    throw new ApiError(500, "Invalid API response shape");
  }
  return parsed.data;
}
```

```

}

const users = await apiGet("/api/users", z.array(UserSchema)); // User[] ✅ validated

// 2. Forms (React Hook Form)
const formSchema = z.object({
  password: z.string().min(8, "Min 8 characters"),
  confirm: z.string(),
}).refine((d) => d.password === d.confirm, { message: "Passwords don't match", path: ["confirm"] });

// 3. Env (Topic 27)
// 4. localStorage
function loadState<T extends z.ZodTypeAny>(key: string, schema: T, fallback: z.infer<T>) {
  try {
    const raw = localStorage.getItem(key);
    if (!raw) return fallback;
    const parsed = schema.safeParse(JSON.parse(raw));
    return parsed.success ? parsed.data : fallback; // corrupt/పాత schema → fallback ✅
  } catch { return fallback; }
}

// 5. URL search params
const QuerySchema = z.object({
  page: z.coerce.number().int().positive().default(1),
  sort: z.enum(["asc", "desc"]).default("asc"),
  q: z.string().optional(),
});
const query = QuerySchema.parse(Object.fromEntries(new URLSearchParams(location.search)));

```

Zod advanced features

```

// Transform – parse తర్వాత మార్పుడం
const DateSchema = z.string().transform((s) => new Date(s));

// Refine – custom validation
const PasswordSchema = z.string().refine((p) => /[A-Z]/.test(p), "Need uppercase");
const AsyncSchema = z.string().refine(async (email) => !(await emailExists(email)), "Taken");

// Discriminated union (performance ⚡ – normal union కంటే వేగం)
const EventSchema = z.discriminatedUnion("type", [
  z.object({ type: z.literal("click"), x: z.number(), y: z.number() }),
  z.object({ type: z.literal("keypress"), key: z.string() }),
]);

// Composition
const BaseEntity = z.object({ id: z.string(), createdAt: z.coerce.date() });
const Post = BaseEntity.extend({ title: z.string(), body: z.string() });
const PostPreview = Post.pick({ id: true, title: true });
const PostUpdate = Post.omit({ id: true, createdAt: true }).partial();

// Recursive schemas
type Category = { name: string; children: Category[] };
const CategorySchema: z.ZodType<Category> = z.lazy(() =>
  z.object({ name: z.string(), children: z.array(CategorySchema) })
);

```

```
);
```

```
// Branded types (Topic 21 తో కలిపి!)
```

```
const EmailSchema = z.string().email().brand<"Email">();
```

```
type Email = z.infer<typeof EmailSchema>; // string & z.BRAND<"Email"> ★
```

ప్రత్యామ్నాయాలు

Library	Size	ప్రత్యేకత
Zod	~14KB	ప్రామాణికం, ecosystem పెద్దది, DX ఉత్తమం
Valibot	~1KB	modular, tree-shakeable (bundle-sensitive apps కి)
ArkType	~10KB	TS syntax లానే, అత్యంత వేగం
io-ts	—	fp-ts ecosystem (functional)
TypeBox	—	JSON Schema compatible (OpenAPI, Fastify)
Yup	~20KB	పాతది, types బలహీనం

ఎక్కడ validate చేయాలి — boundaries

[బయటి ప్రపంచం] →  VALIDATE HERE  → [typed internal code]

API response → `schema.parse()` →  types నమ్మొచ్చు

Form submit → `schema.parse()` → 

env → `schema.parse()` → 

DB read → (ORM types నమ్మొచ్చు, కానీ raw SQL కి parse)

Internal function →  validate చేయొద్దు (types సరిపోతాయి — performance waste)

Gotchas

- **as T casting** — ఇది validation కాదు, కేవలం "నమ్మ".
- **Types మరియు schema రెండూ విడిగా రాయడం** → drift. `z.infer` వాడాలి.
- **Hot path లో ప్రతిచోటా parse చేయడం** — boundaries లోనే; internal calls లో కాదు.
- **parse (throws) vs safeParse** — UI code లో `safeParse` మేలు.
- **Validation errors ని silent గా swallow చేయడం** — API contract మార్పులు తెలియవు (monitoring కి పంపాలి).
- **`z.string()` కి `.min(1)` మర్చిపోవడం** — `""` valid string!
- **Large schemas parse cost** — hot paths లో discriminated unions + lazy వాడాలి.

Key Points

- **Types ≠ runtime safety** — boundaries లో validation తప్పనిసరి.
- **Zod = single source of truth** → `z.infer` తో types.
- **"Parse, don't validate"** — validated data కి కొత్త type.

- Boundaries: API, forms, env, storage, URL, webhooks.
- Schema violations = monitoring signal (contract drift).

Interview దృష్టి

- "API data ని ఎలా type-safe చేస్తావు?" → Zod parse at boundary + `z.infer` + contract-violation logging (production maturity).
- "*as User* ఎందుకు సరిపోదు?" → unchecked assertion; erasure.

30. API Types at Scale — Codegen, tRPC, GraphQL

వివరణ — frontend/backend types ని sync లో ఉంచడం

సమస్య: backend API మారితే frontend types పాతవి అవుతాయి → runtime crash. Manual sync scale కాదు.

ఎంపిక 1 — OpenAPI codegen (REST + వేరే teams/భాషలు)

```
npx openapi-typescript https://api.example.com/openapi.json -o src/types/api.d.ts
```

```
import type { paths, components } from './types/api';

type User = components['schemas']['User'];
type GetUsersResponse = paths['/users']['get']['responses']['200']['content']
  ['application/json'];
type CreateUserBody = paths['/users']['post']['requestBody']['content']['application/json'];

// openapi-fetch – పూర్తిగా typed client (runtime ~2KB)
import createClient from 'openapi-fetch';
const client = createClient<paths>({ baseUrl: '/api' });

const { data, error } = await client.GET('/users/{id}', { params: { path: { id: '1' } } });
// data: User | undefined ✅ error typed ✅
```

ఎప్పుడు: backend వేరే భాషలో (Go/Java/Python), public API, multiple consumers.

ఎంపిక 2 — tRPC (full-stack TypeScript, codegen లేదు 🌟)

```
// server/router.ts
import { initTRPC, TRPCError } from '@trpc/server';
import { z } from 'zod';

const t = initTRPC.context<Context>().create();
const protectedProcedure = t.procedure.use(({ ctx, next }) => {
  if (!ctx.user) throw new TRPCError({ code: 'UNAUTHORIZED' });
  return next({ ctx: { ...ctx, user: ctx.user } }); // ✅ user ఇప్పుడు non-null typed
});
```

```
export const appRouter = t.router({
  user: t.router({
    byId: t.procedure
      .input(z.object({ id: z.string().uuid() }))
      .query(async ({ input, ctx }) => ctx.db.user.findUnique({ where: { id: input.id } })),

    create: protectedProcedure
      .input(z.object({ email: z.string().email(), name: z.string().min(1) }))
      .mutation(async ({ input, ctx }) => ctx.db.user.create({ data: input })),
  }),
});
export type AppRouter = typeof appRouter; // 🔑 ఇదొక్కటే client కి వెళ్తుంది (type-only!)
```

```
// client - codegen లేదు, build step లేదు, types నేరుగా inferred ☆
import { trpc } from './trpc';

function Profile({ id }: { id: string }) {
  const { data, isLoading } = trpc.user.byId.useQuery({ id }); // data: User | null ✅
  const create = trpc.user.create.useMutation();

  create.mutate({ email: "a@b.com", name: "S" }); // ✅ typed
  create.mutate({ email: "x" }); // ❌ compile error ⚡
}
```

ఎప్పుడు: monorepo, full-stack TypeScript, ఒకే team. **లాభం:** end-to-end type safety, codegen లేదు, refactor చేస్తే frontend వెంటనే error. **పరిమితి:** TypeScript-only, public API కి కాదు.

ఎంపిక 3 — GraphQL Code Generator

```
# codegen.yml
schema: http://localhost:4000/graphql
documents: "src/**/*.graphql"
generates:
  src/generated/graphql.ts:
    plugins: [typescript, typescript-operations, typescript-react-query]
    config: { fetcher: fetch }
```

```
// Generated hooks - query shape ఖచ్చితంగా typed (over-fetching లేదు)
const { data } = useGetUserQuery({ id: "1" });
// data.user.name ✅ - query లో అడిగినవి మాత్రమే type లో ఉంటాయి
```

ఎంపిక 4 — Shared package (monorepo)

```
// packages/contracts/src/user.ts - ఒకే schema, రెండు వైపులా
export const UserSchema = z.object({ id: z.string(), email: z.string().email() });
export type User = z.infer<typeof UserSchema>;

// apps/api - validate incoming
// apps/web - validate response + typed
```

పోలిక

	OpenAPI codegen	tRPC	GraphQL	Shared Zod
Codegen అవసరమా	✓	✗	✓	✗
Backend భాష	ఏదైనా	TS మాత్రమే	ఏదైనా	TS మాత్రమే
Runtime validation	✗ (types మాత్రమే)	✓ (Zod)	✗	✓
Over-fetching	✓ ఉంది	✓ ఉంది	✗ లేదు	✓ ఉంది
Public API కి	✓	✗	✓	✗
Setup complexity	మధ్యస్థం	తక్కువ	ఎక్కువ	తక్కువ

Prisma/Drizzle — DB types

```
// Prisma - schema నుండి types generate
const user = await prisma.user.findUnique({
  where: { id },
  include: { posts: { include: { comments: true } } },
});
// user యొక్క type include ప్రకారం ఖచ్చితంగా inferred ☆

type UserWithPosts = Prisma.UserGetPayload<{ include: { posts: true } }>;

// Drizzle - SQL-like, types inferred
const users = await db.select().from(usersTable).where(eq(usersTable.id, id));
type User = typeof usersTable.$inferSelect;
type NewUser = typeof usersTable.$inferInsert;
```

Gotchas

- **Generated files ని manually edit చేయడం** → regeneration లో పోతాయి (git లో commit చేయాలా వద్దా అనేది team decision; commit చేస్తే CI లో drift check).
- **Codegen ని CI లో run చేయకపోవడం** → schema drift తెలియదు.
- **Generated types ని runtime validation అనుకోవడం** — OpenAPI/GraphQL types కేవలం compile-time.
- **tRPC ని public API కి వాడటం** — versioning, non-TS clients సమస్య.
- **export type AppRouter బదులు value export** → server code client bundle లోకి! (import type తప్పనిసరి).

Key Points

- Manual type duplication = drift = production bugs.
- **tRPC** = full-stack TS కి ఉత్తమం (zero codegen); **OpenAPI codegen** = polyglot/public APIs; **GraphQL** = flexible queries.
- Types ≠ validation — codegen చేసినా boundary validation అవసరం.

- Prisma/Drizzle = DB → types automatic.

Interview దృష్టి

- "Frontend/backend types ఎలా sync చేస్తావు?" → 4 ఎంపికలు + trade-offs (team, భాష, public/private).
- "tRPC ఎలా type safety ఇస్తుంది codegen లేకుండా?" → router type ని **import** type ద్వారా share; TS inference.

31. Testing with TypeScript

వివరణ

```
// Vitest + TypeScript (setup అవసరం లేదు – native support)
import { describe, it, expect, vi, beforeEach } from "vitest";

describe("UserService", () => {
  it("creates a user", async () => {
    const result = await service.register({ email: "a@b.com" as Email, role: "user" });
    expect(result.email).toBe("a@b.com");
  });
});
```

Typed mocks

```
// 1. Interface mock (typed, dependency inversion తో సులభం)
const mockRepo: UserRepository = {
  findById: vi.fn().mockResolvedValue(null),
  findByEmail: vi.fn().mockResolvedValue(null),
  create: vi.fn().mockImplementation(async (input) => ({ ...input, id: "1", createdAt: new Date() })),
};
const service = new UserService(mockRepo, mockMailer);

// 2. vi.mocked – module mock ని typed గా
import { fetchUser } from "../api";
vi.mock("../api");
const mockedFetch = vi.mocked(fetchUser);
mockedFetch.mockResolvedValue({ id: "1", name: "S" }); // ✅ return type checked

// 3. Partial mocks
function createMockUser(overrides: Partial<User> = {}): User {
  return { id: "1", email: "a@b.com" as Email, role: "user", createdAt: new Date(), ...overrides };
}
const admin = createMockUser({ role: "admin" }); // ✅ typed factory
```

Type-level tests (types ని test చేయడం 🌟)

```
// 1. Vitest expectTypeOf
import { expectTypeOf, assertType } from "vitest";

it("types are correct", () => {
  expectTypeOf<Add<2, 3>>().toEqualTypeOf<5>();
  expectTypeOf(getUser).returns.resolves.toEqualTypeOf<User>();
  expectTypeOf(getUser).parameters.toEqualTypeOf<[string]>();
  expectTypeOf<Partial<User>>().toHaveProperty("email");
});

// 2. Manual type assertions (test runner అవసరం లేదు)
type Expect<T extends true> = T;
type Equals<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ?
true : false;

type _t1 = Expect<Equals<ReturnType<typeof getUser>, Promise<User>>>>;
type _t2 = Expect<Equals<Paths<Config>, "db" | "db.host">>>;
// compile అయితే pass; కాకపోతే build fail ⚡

// 3. @ts-expect-error - తప్పు జరగాలని ఆశించడం
// @ts-expect-error - email must be a valid Email brand
getUser("plain-string");
// ⚠️ error రాకపోతే *ఇదే* error అవుతుంది (unused directive) - perfect negative test
```

@ts-expect-error vs @ts-ignore : **expect-error** = "ఇక్కడ error ఉండాలి; లేకపోతే నాకు చెప్పు" ✅.
ts-ignore = "ఏముందో పట్టించుకోకు" ❌ (silent). **ఎప్పుడూ @ts-expect-error వాడాలి.**

Test configuration

```
// tsconfig.json - tests ని include చేయాలి కానీ build లో కాదు
{
  "include": ["src/**/*.ts", "tests/**/*.ts"],
}
// tsconfig.build.json
{
  "extends": "./tsconfig.json",
  "exclude": ["**/*.test.ts", "**/*.spec.ts", "tests"]
}
```

```
// vitest.config.ts
export default defineConfig({
  test: {
    globals: true,
    environment: "node", // లేదా "jsdom" (React)
    typecheck: { enabled: true, include: ["**/*.test-d.ts"] }, // ✅ type tests
    coverage: { provider: "v8", thresholds: { lines: 80 } },
  },
});
```

Testing patterns

```
// 1. Test data builders (typed, composable)
class UserBuilder {
  private user: User = createMockUser();
  withRole(role: Role) { this.user = { ...this.user, role }; return this; }
  withEmail(email: string) { this.user = { ...this.user, email: email as Email }; return this; }
}
build(): User { return this.user; }
}
const admin = new UserBuilder().withRole("admin").build();

// 2. Typed fixtures
const fixtures = {
  validUser: { email: "a@b.com", age: 25 },
  invalidUser: { email: "not-email", age: -1 },
} satisfies Record<string, Partial<CreateUserDto>>; // ✅ satisfies!

// 3. Zod schemas & test data generation
import { generateMock } from "@anatine/zod-mock";
const randomUser = generateMock(UserSchema); // ✅ schema-valid random data
```

Gotchas

- **any mocks** — type safety పోతుంది; typed factories వాడాలి.
- **@ts-ignore in tests** — errors దాచడం; **@ts-expect-error** వాడాలి.
- **Tests ని tsconfig నుండి exclude చేయడం** → tests type-check కావు (bugs దాగుతాయి).
- **Build లో tests include చేయడం** → dist లో test files.
- **vi.mock hoisting** — module scope లో variables వాడలేం (**vi.hoisted** అవసరం).
- **Type tests రాయకపోవడం** — complex utility types silent గా break అవుతాయి.

Key Points

- Interfaces + DI = typed mocks సులభం.
- **expectTypeOf / Expect<Equals><>>** = type-level tests (utility types కి తప్పనిసరి).
- **@ts-expect-error > @ts-ignore** (self-verifying).
- Tests type-check అవ్వాలి కానీ build లో ఉండకూడదు (separate tsconfig).
- Typed builders/factories/fixtures + **satisfies** .

Interview దృష్టి

- "Types ని test చేస్తావా?" → **expectTypeOf** , **Expect<Equals><>>** , **@ts-expect-error** — library authors కి కీలకం.
- "Typed mocks ఎలా?" → interface-based DI + **vi.mocked** + factories.

Part 5 — Configuration, Tooling & Performance

32. tsconfig.json Deep Dive

వివరణ — ప్రతి ముఖ్యమైన flag

The **strict** family (అత్యంత ముఖ్యం)

```
{ "strict": true } // ఈ 8 flags ని ఒకేసారి enable చేస్తుంది ↓
```

Flag	ఏం చేస్తుంది	ఎందుకు ముఖ్యం
noImplicitAny	Implicit any ని నిషేధిస్తుంది	Type safety యొక్క పునాది
strictNullChecks	null / undefined ని types నుండి వేరు చేస్తుంది	★ అత్యంత విలువైనది – "billion dollar mistake" పరిష్కారం
strictFunctionTypes	Function params contravariant	Unsafe callbacks ఆపుతుంది (Topic 22)
strictBindCallApply	bind /call/apply type-check	Argument mismatches
strictPropertyInitialization	Class fields initialize కావాలి	undefined field access
noImplicitThis	this implicit any కాకూడదు	this bugs
useUnknownInCatchVariables	catch (e: unknown)	Error handling correctness
alwaysStrict	"use strict" emit	JS strict mode

```
// strictNullChecks ప్రభావం
function getLength(s: string | null) {
  return s.length; // off: ✅ compile (⚡ runtime) | on: ❌ compile error ✅
}
```

strict లో లేని కానీ ఉండాలన్న flags ★

```
{
  "noUncheckedIndexedAccess": true, // ★ array/index access కి | undefined
  "exactOptionalPropertyTypes": true, // a?: string ≠ a: string | undefined
  "noImplicitOverride": true, // override keyword తప్పనిసరి
  "noFallthroughCasesInSwitch": true, // switch fallthrough bugs
  "noPropertyAccessFromIndexSignature": true, // obj.foo బదులు obj["foo"] (index signatures కి)
  "noImplicitReturns": true, // అన్ని paths return చేయాలి
  "noUnusedLocals": true, // ⚠️ dev లో చిరాకు (ESLint మేలు)
```

```
"noUnusedParameters": true, // ⚠ అదే
"allowUnreachableCode": false,
}
```

```
// noUncheckedIndexedAccess - నిజమైన bugs ఆపుతుంది
const arr = [1, 2, 3];
const x = arr[10]; // off: number | on: number | undefined ✓
x.toFixed(); // off: ✨ runtime | on: ✗ compile error ✓

const map: Record<string, User> = {};
map["missing"].name; // off: ✨ | on: ✗ ✓

// ⚠ ఇది కొంచెం verbose చేస్తుంది
for (let i = 0; i < arr.length; i++) {
  arr[i]!.toFixed(); // ! అవసరం (లేదా for...of వాడాలి - మేలు)
}
for (const n of arr) { n.toFixed(); } // ✓ undefined కాదు
```

Module & resolution

```
{
  "target": "ES2022", // emit అయ్యే JS version
  "lib": ["ES2022", "DOM"], // అందుబాటులో ఉన్న built-in APIs
  "module": "ESNext", // emit అయ్యే module system
  "moduleResolution": "bundler", // imports ఎలా resolve చేయాలి
  "esModuleInterop": true, // CJS ని ESM లా import చేయడానికి
  "allowSyntheticDefaultImports": true,
  "resolveJsonModule": true, // import data from "./data.json"
  "isolatedModules": true, // per-file transpilation safety
  "verbatimModuleSyntax": true, // import/export ని అలాగే emit
  "allowImportingTsExtensions": true, // import "./x.ts" (noEmit తో)
}
```

moduleResolution	ఎప్పుడు
"bundler"	Vite/webpack/esbuild ✓ (extensions optional, exports support)
"NodeNext"	Node native ESM/CJS (extensions తప్పనిసరి)
"Node10"	legacy

Emit options

```
{
  "outDir": "./dist",
  "rootDir": "./src",
  "noEmit": true, // bundler emit చేస్తుంటే (type check only)
  "declaration": true, // .d.ts (libraries)
  "declarationMap": true, // .d.ts.map - go-to-source ✓
  "sourceMap": true,
  "removeComments": false,
  "importHelpers": true, // tslib నుండి helpers (bundle size ↓)
```

```
"downlevelIteration": true, // ES5 target లో for...of correctness
}
```

Performance & project structure

```
{
  "skipLibCheck": true, // ★ node_modules .d.ts skip (build వేగం భారీగా ↑)
  "incremental": true, // .tsbuildinfo cache
  "tsBuildInfoFile": "./node_modules/.cache/tsbuildinfo",
  "composite": true, // project references కి తప్పనిసరి
  "assumeChangesOnlyAffectDirectDependencies": true, // watch mode వేగం
}
```

Project references (monorepos)

```
// packages/app/tsconfig.json
{
  "compilerOptions": { "composite": true, "outDir": "dist" },
  "references": [ { "path": "../shared" }, { "path": "../ui" } ]
}
```

```
tsc --build // dependency order లో build (incremental ✅)
tsc --build --watch
tsc --build --clean
```

లాభాలు: incremental builds, enforced boundaries, parallel builds, editor performance.

Path mapping

```
{
  "baseUrl": ".",
  "paths": {
    "@/*": ["/src/*"],
    "@shared/*": ["/../shared/src/*"],
    "@components/*": ["/src/components/*"]
  }
}
```

⚠️ **paths కేవలం TypeScript కి.** Runtime/bundler కి కూడా చెప్పాలి:

- Vite: vite-tsconfig-paths plugin
- Jest/Vitest: moduleNameMapper / **alias**
- Node: tsconfig-paths లేదా imports field in package.json (#internal/*)

JS interop (migration కి)

```
{
  "allowJs": true,           // .js files compile చేయడానికి
  "checkJs": true,          // .js files ని కూడా type-check (JSDoc వాడే)
  "maxNodeModuleJsDepth": 0
}
```

సిఫార్సు చేసిన configs

```
// Frontend app (Vite + React)
{
  "compilerOptions": {
    "target": "ES2022", "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext", "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true, "noUncheckedIndexedAccess": true, "noImplicitOverride": true,
    "noEmit": true, "isolatedModules": true, "verbatimModuleSyntax": true,
    "skipLibCheck": true, "resolveJsonModule": true,
    "baseUrl": ".", "paths": { "@/*": [".src/*"] }
  },
  "include": ["src"]
}

// Node backend
{
  "compilerOptions": {
    "target": "ES2022", "lib": ["ES2022"],
    "module": "NodeNext", "moduleResolution": "NodeNext",
    "types": ["node"],
    "strict": true, "noUncheckedIndexedAccess": true,
    "outDir": "dist", "rootDir": "src",
    "sourceMap": true, "declaration": false,
    "skipLibCheck": true, "incremental": true
  },
  "include": ["src"], "exclude": ["**/*.test.ts"]
}

// Library (npm package)
{
  "compilerOptions": {
    "target": "ES2020", "module": "ESNext", "moduleResolution": "bundler",
    "strict": true, "declaration": true, "declarationMap": true,
    "outDir": "dist", "rootDir": "src", "sourceMap": true,
    "isolatedDeclarations": true // TS 5.5 - వేగవంతమైన .d.ts
  }
}
```

`@tsconfig/*` **bases:** `npm i -D @tsconfig/node22 @tsconfig/strictest` → `"extends": "@tsconfig/strictest/tsconfig.json"` .

Gotchas

- **strict: false** — TypeScript యొక్క సగం విలువ నష్టం.
- **skipLibCheck: false** → builds చాలా నెమ్మది.
- **paths** ని **runtime కి configure చేయకపోవడం** → "module not found" at runtime.
- **target: ES5** అనవసరంగా → పెద్ద, నెమ్మది output.
- **allowJs + checkJs** ఒకేసారి పెద్ద **codebase కి** → వేల errors (incremental గా చేయాలి).
- **include / exclude** తప్పు → tests type-check కావు లేదా dist లో వెళ్ళాయి.
- **noUnusedLocals** in dev → editing చేస్తున్నప్పుడు చిరాకు (ESLint warning మేలు).

Key Points

- **strict: true + noUncheckedIndexedAccess** = కనీస baseline.
- **moduleResolution: bundler** (frontend) vs **NodeNext** (Node, extensions తప్పనిసరి).
- **skipLibCheck: true + incremental** = build వేగం.
- **Project references** = monorepo incremental builds + boundaries.
- **paths** ని runtime tooling లో కూడా mirror చేయాలి.

Interview దృష్టి

- "strict flags ఏమిటి, ఏది అత్యంత ముఖ్యం?" → 8 flags; **strictNullChecks** ★.
- "noUncheckedIndexedAccess ఎందుకు?" → array/record access **undefined** కావచ్చు; **strict** లో లేదు.
- "Monorepo TS build వేగం ఎలా?" → project references, incremental, skipLibCheck, isolatedDeclarations.

33. Compiler Internals & Type-Checking Performance

వివరణ — tsc ఎలా పని చేస్తుంది

```
Source (.ts)
  ↓ 1. Scanner/Lexer      → tokens
  ↓ 2. Parser             → AST
  ↓ 3. Binder            → symbols + scopes (declarations ↔ references)
  ↓ 4. Checker            → type checking (⚠ 80% సమయం ఇక్కడే)
  ↓ 5. Transformer/Emitter → JavaScript + .d.ts + source maps
Output (.js)
```

Type checking నెమ్మదికి కారణాలు: structural comparison (deep, recursive), type instantiation (generics), inference (constraint solving), union/intersection normalization.

Real-life Scenario

Type checking = ఒక పెద్ద పెళ్లి ఇంట్లో బంధుత్వాలు లెక్కించడం. "ఇతను అతనికి ఏమవుతాడు?" – ప్రతి జతను పోల్చాలి. 100 మందైతే సులభం; 10,000 మంది + ప్రతి ఒక్కరికీ 5 తరాల చరిత్ర (nested generics) ఉంటే – గంటలు పడుతుంది. **Caching = "ఇద్దరి బంధుత్వం ఒకసారి లెక్కిసి రాసుకోవడం."**

Performance ని కొలవడం

```
# 1. మొత్తం diagnostics
tsc --noEmit --extendedDiagnostics
# Files: 1204 | Types: 45,231 | Instantiations: 1,203,442
# Check time: 8.32s ← ఇదే ముఖ్యం
# Total time: 12.1s

# 2. ఏ files నెమ్మది
tsc --noEmit --generateTrace ./trace
npx @typescript/analyze-trace ./trace
# → "Hot spots: src/types/advanced.ts (4.2s)"

# 3. chrome://tracing లో trace.json open చేసి visualize
```

ఆరోగ్యకరమైన thresholds: Instantiations < 1M, Check time < 10s, Memory < 2GB.

సాధారణ performance హంతకులు & పరిష్కారాలు

```
// ❌ 1. పెద్ద union types (100+ members)
type AllIcons = "icon-1" | "icon-2" | /* ... 500 more */;
// ✅ template literal లేదా runtime validation
type Icon = `icon-${number}`;

// ❌ 2. Deeply recursive conditional types
type DeepFlatten<T> = /* 10 levels of recursion */;
// ✅ depth పరిమితం చేయడం లేదా tail-recursive రాయడం

// ❌ 3. Complex intersections ని పదే పదే
type Props = A & B & C & D & E & F; // ప్రతి వాడకంలో మళ్ళీ compute
// ✅ interface extends వాడటం (caching మెరుగు)
interface Props extends A, B, C {}

// ❌ 4. Return type inference on large functions
export function huge() { /* 500 lines */ return complexObject; }
// ✅ explicit return type – checker పని తగ్గుతుంది ✨
export function huge(): HugeResult { }

// ❌ 5. Barrel files (import "everything")
import { X } from "@components"; // 200 modules load
// ✅ direct import
import { X } from "@components/X";

// ❌ 6. skipLibCheck: false
// ✅ true (node_modules .d.ts check అనవసరం)
```

Editor performance

```
// .vscode/settings.json
{
  "typescript.tsserver.maxTsServerMemory": 8192,
  "typescript.disableAutomaticTypeAcquisition": true,
  "typescript.tsserver.experimental.enableProjectDiagnostics": false,
  "files.watcherExclude": { "**/node_modules/**": true, "**/dist/**": true }
}
```

```
// tsconfig - watch performance
{
  "watchOptions": {
    "watchFile": "useFsEvents",
    "watchDirectory": "useFsEvents",
    "excludeDirectories": ["**/node_modules", "dist"]
  }
}
```

Monorepo build strategy

1. Project references + tsc --build → incremental, dependency-aware
2. Turborepo/Nx caching → unchanged packages skip
3. isolatedDeclarations (TS 5.5) → .d.ts generation < (parallel)
4. Type check ని build నుండి వేరు చేయడం → esbuild build + tsc --noEmit parallel

```
// turbo.json
{
  "tasks": {
    "typecheck": { "dependsOn": ["^build"], "outputs": [] },
    "build": { "dependsOn": ["^build"], "outputs": ["dist/**"] }
  }
}
```

tsgo — native TypeScript compiler (2025+)

Microsoft **Go** లో TypeScript compiler ని rewrite చేస్తోంది (typescript-go / TS 7.0 goal): ~10× వేగవంతమైన type checking, తక్కువ memory, native concurrency. 2026 లో preview దశలో — పెద్ద codebases కి ఇది గేమ్ ఛేంజర్.

Gotchas

- **Type-level programming compile time ని పీల్చడం** — ఒక్క complex type build ని రెట్టింపు చేయొచ్చు.
- `--extendedDiagnostics` లేకుండా "నెమ్మది" అని guess చేయడం.
- **Editor slow = compiler slow అనుకోవడం** — tsserver వేరే process; memory limits వేరు.
- **incremental cache ని CI లో persist చేయకపోవడం** — ప్రతిసారి full build.
- **include** లో అనవసర files (dist, coverage, node_modules).

Key Points

- Pipeline: scanner → parser → binder → **checker (నెమ్మది)** → emitter.
- కొలత: `--extendedDiagnostics`, `--generateTrace` + `analyze-trace`.
- హంతకులు: పెద్ద unions, deep recursion, inferred return types, barrels, `skipLibCheck: false`.
- Monorepo: project references + build caching + isolatedDeclarations.
- **tsgo** = Go-based compiler, ~10× వేగం (భవిష్యత్తు).

Interview దృష్టి

- "TS build నెమ్మది – ఏం చేస్తావు?" → `measure` (`--extendedDiagnostics` / `trace`) → hot types/files → explicit return types, simplify types, `skipLibCheck`, project references, incremental – structured answer.

34. ESLint + typescript-eslint

వివరణ

TypeScript compiler ≠ linter. Compiler types ని check చేస్తుంది; **ESLint code quality, patterns, correctness** ని చూస్తుంది.

```
npm i -D eslint typescript-eslint
```

```
// eslint.config.js (flat config – ESLint 9+)
import tseslint from "typescript-eslint";

export default tseslint.config(
  ...tseslint.configs.strictTypeChecked, // ★ type-aware rules
  ...tseslint.configs stylisticTypeChecked,
  {
    languageOptions: {
      parserOptions: { projectService: true, tsconfigRootDir: import.meta.dirname },
    },
    rules: {
      "@typescript-eslint/no-explicit-any": "error",
      "@typescript-eslint/no-floating-promises": "error", // ★ await మర్చిపోవడం
      "@typescript-eslint/no-misused-promises": "error", // ★ async in void position
      "@typescript-eslint/consistent-type-imports": "error", // import type enforce
      "@typescript-eslint/no-unnecessary-condition": "warn", // ఎప్పుడూ true/false
    },
    conditions:
      "@typescript-eslint/switch-exhaustiveness-check": "error", // ★ exhaustive switches
      "@typescript-eslint/no-unsafe-assignment": "error", // any నుండి assignment
      "@typescript-eslint/no-unsafe-member-access": "error",
      "@typescript-eslint/no-unsafe-call": "error",
      "@typescript-eslint/no-unsafe-return": "error",
      "@typescript-eslint/prefer-nullish-coalescing": "error", // || బదులు ??
      "@typescript-eslint/prefer-optional-chain": "error",
      "@typescript-eslint/no-non-null-assertion": "warn", // ! వాడకం
      "@typescript-eslint/explicit-module-boundary-types": "warn",
  }
);
```

```

    "@typescript-eslint/require-await": "error",
    "@typescript-eslint/return-await": ["error", "in-try-catch"],
  },
}
);

```

అత్యంత విలువైన type-aware rules (ఇవి compiler పట్టుకోలేవు)

```

// 1. no-floating-promises - ★ production bugs అవుతుంది
saveUser(data); // ✗ Error - unhandled rejection
await saveUser(data); // ✓
void saveUser(data); // ✓ ఉద్దేశపూర్వకం అని ప్రకటించడం

// 2. no-misused-promises - ★
<button onClick={async () => { await save(); }} /> // ✗ errors swallow
element.addEventListener("click", async () => {}); // ✗

// 3. no-unnecessary-condition - dead code / logic bugs
function f(s: string) {
  if (s) {} // ⚠ s ఎప్పుడూ truthy కాదు... (string "" కావచ్చు కాబట్టి OK)
  if (s !== undefined) {} // ✗ Error - s ఎప్పుడూ undefined కాదు (dead check)
}

// 4. switch-exhaustiveness-check - never trick అవసరం లేదు
switch (shape.kind) { // ✗ Error: "rect" case handle కాలేదు
  case "circle": break;
}

// 5. prefer-nullish-coalescing - subtle bugs
const port = config.port || 3000; // ✗ port = 0 అయితే 3000 అవుతుంది! ✨
const port2 = config.port ?? 3000; // ✓

// 6. no-unnecessary-type-assertion
const x = foo as string; // ✗ foo ఇప్పటికే string

```

Type-aware linting యొక్క ఖర్చు

Type-aware rules కి **full type information** అవసరం → lint నెమ్మది (compile అంత సమయం). పరిష్కారాలు:

- projectService: **true** (typescript-eslint v8) – వేగవంతమైన project loading.
- Test/config files కి type-aware rules disable చేయడం.
- CI లో lint ని parallel job గా run చేయడం.
- `--cache` flag.

ఇతర ముఖ్య tools

```

# Prettier - formatting (ESLint తో conflict రాకుండా)
npm i -D prettier eslint-config-prettier

# Type coverage - ఎంత % code typed?
npx type-coverage --detail --at-least 95

```

```
# Unused exports/deps
npx knip

# Circular dependencies
npx madge --circular --extensions ts,tsx src/

# Bundle impact of types
npx are-the-types-wrong ./package.tgz      # library publishing కి ⭐
```

CI pipeline

```
- run: npm ci
- run: npm run typecheck      # tsc --noEmit
- run: npm run lint          # eslint
- run: npm run test
- run: npm run build
# ✅ typecheck మరియు lint విడిగా - వేగవంతమైన feedback
```

Gotchas

- **ESLint formatting rules వాడటం** – Prettier కి వదిలేయాలి (eslint-config-prettier తో conflicts తీసేయాలి).
- **Type-aware rules ని parserOptions.project లేకుండా** → rules silent గా పని చేయవు.
- **no-floating-promises లేకపోవడం** → production లో silent failures.
- **|| vs ??** – 0 , "" , false valid values అయినప్పుడు bug.
- **eslint-disable అతిగా** – comments లో reason రాయాలి; CI లో count track చేయాలి.
- **Legacy .eslintrc vs flat config** గందరగోళం (ESLint 9+ = flat).

Key Points

- Compiler ≠ linter – రెండూ అవసరం.
- **Type-aware rules** (no-floating-promises , no-misused-promises , switch-exhaustiveness) = అత్యధిక విలువ.
- prefer-nullish-coalescing = subtle production bugs.
- Prettier formatting, ESLint correctness.
- CI: typecheck + lint + test + build విడిగా.

Interview దృష్టి

- "ఏ ESLint rules తప్పనిసరి?" → floating-promises, misused-promises, no-explicit-any, exhaustiveness, nullish-coalescing – ఎందుకో చెప్పడం ముఖ్యం.

35. Common Compiler Errors Decoded

అత్యంత సాధారణ errors & పరిష్కారాలు

TS2322 — Type 'X' is not assignable to type 'Y'

```
const n: number = "5"; // ✗
// కారణం: assignability violation
// పరిష్కారం: type సరిచేయడం, లేదా conversion (Number("5"))

// సాధారణ subtle version:
const status: "active" | "inactive" = someString; // ✗ string is not assignable
// ✔ as const, satisfies, లేదా narrowing
```

TS2345 — Argument of type 'X' is not assignable to parameter of type 'Y'

```
function f(x: string) {}
f(42); // ✗
// చాలాసార్లు: `string | undefined` ని `string` కి పంపడం → guard అవసరం
```

TS7006 — Parameter 'x' implicitly has an 'any' type

```
const f = (x) => x; // ✗ noImplicitAny
const f2 = (x: string) => x; // ✔
// contextual typing లేని చోట parameters annotate చేయాలి
```

TS2531/TS18047/TS18048 — Object is possibly 'null'/'undefined'

```
const el = document.getElementById("x");
el.click(); // ✗ TS18047
// ✔ if (el) { el.click(); } | el?.click() | el!.click() (ప్రమాదకరం)
```

TS2339 — Property 'x' does not exist on type 'Y'

```
const obj = { a: 1 };
obj.b; // ✗
// కారణాలు: typo, తప్పు type, narrowing అవసరం, augmentation missing
// Union అయితే: narrow చేయాలి. Window/global అయితే: declaration merging.
```

TS2571 — Object is of type 'unknown'

```
catch (e) { e.message; } // ✗
// ✔ if (e instanceof Error) { e.message }
```

TS2589 — Type instantiation is excessively deep and possibly infinite

```
// కారణం: recursive type depth limit
// ✔ tail-recursion, depth limit parameter, type simplification
```

```
type Deep<T, D extends number = 5> = D extends 0 ? T : /* recurse with D-1 */;
```

TS2739/TS2741 — Property 'x' is missing in type

```
const u: User = { name: "S" }; // ❌ 'id' is missing  
// ✅ అన్ని required properties ఇవ్వడం, లేదా Partial<User>
```

TS2367 — This comparison appears unintentional

```
if (status === "activ") {} // ❌ literal union లో లేదు – typo caught ⚡
```

TS2554 — Expected N arguments, but got M

TS1005 / TS1109 — syntax errors (, expected, expression expected) — తరచుగా `.ts` file లో JSX రాయడం (`.tsx` కావాలి) లేదా `<T>` generic in `.tsx` (`<T,>` కావాలి).

TS2307 — Cannot find module 'x' or its corresponding type declarations

కారణాలు: (1) package install కాలేదు, (2) @types missing,
(3) paths/moduleResolution config తప్పు, (4) NodeNext లో .js extension missing
✅ `npm i -D @types/x | declare module "x" | tsconfig paths`

TS2688 — Cannot find type definition file for 'x' → types array లో ఉన్న package install కాలేదు.

TS4023/TS4053 — Exported variable has or is using name from external module but cannot be named

```
// కారణం: declaration emit లో private type reference  
// ✅ ఆ type ని కూడా export చేయడం
```

Error messages ని చదవడం — strategy

1. చివరి line మొదట చదవండి — అసలు కారణం తరచుగా అక్కడ ("Types of property 'x' are incompatible" → ఆ property చూడండి)
2. పెద్ద type errors ని Prettify<T> తో విస్తరించి చూడండి
3. Intermediate types ని alias చేసి hover చేయండి
4. "Type instantiation" errors → recursion తగ్గించండి
5. తెలియకపోతే — variable ని చిన్న ముక్కలుగా విడగొట్టి ఎక్కడ break అవుతుందో చూడండి

```
// Debugging trick — ఏ type ఏమిటో చూడటం  
type Debug<T> = { [K in keyof T]: T[K] } & {};  
type WhatIsThis = Debug<ComplexInferredType>; // hover ✅
```

Key Points

- TS2322/2345 = assignability; TS7006 = implicit any; TS18047/48 = null safety.
- TS2589 = recursion depth → simplify/tail-recursive.

- TS2307 = module/types resolution (install, @types, paths, extensions).
 - Error message **చివరి line** = అసలు కారణం.
 - Prettify<T> = complex types ని చదవగలిగేలా చేయడం.
-

Part 6 — Architecture, Migration & Interview

36. JavaScript → TypeScript Migration Strategy

వివరణ

నియమం #1: పెద్ద codebase ని ఒకసారి migrate చేయొద్దు. 6 నెలల "big bang rewrite" ఎప్పుడూ విఫలమవుతుంది. Incremental migration — app పని చేస్తూనే ఉండాలి.

Real-life Scenario

Migration = రద్దీగా ఉన్న రోడ్డుని విస్తరించడం. రోడ్డు మొత్తం మూసేసి (big bang rewrite) పని చేస్తే నగరం స్తంభిస్తుంది. ఒక్కో lane చొప్పున పని చేస్తారు — traffic నడుస్తూనే ఉంటుంది, ప్రతి రోజూ కొంచెం మెరుగు.

Phase-wise plan

Phase 0 — సన్నద్ధత (1-2 రోజులు)

```
npm i -D typescript @types/node
npx tsc --init
```

```
// tsconfig.json - అత్యంత సరళమైన config తో మొదలు
{
  "compilerOptions": {
    "allowJs": true,           // 🔑 .js files ని compile చేయనివ్వడం
    "checkJs": false,         // ఇంకా JS ని check చేయొద్దు
    "strict": false,          // 🔑 తర్వాత ఆన్ చేస్తాం
    "noEmit": true,
    "skipLibCheck": true,
    "target": "ES2022", "module": "ESNext", "moduleResolution": "bundler"
  },
  "include": ["src"]
}
```

Phase 1 — leaf files మొదట (dependencies లేనివి)

migration order (dependency graph ఆధారంగా):

1. constants, config, pure utils ← ఎవరి మీదా ఆధారపడవు
2. types/models (కొత్తగా రాయాలి)
3. services/api layer
4. hooks / business logic
5. components (leaf → container)
6. entry points (App, main)

```
# ఏ files leaf అని కనుగొనడం
npx madge --extensions js,jsx --orphans src/
```

Phase 2 – ఒక్కో file: .js → .ts

```
// ⚡ మొదటి pass: compile అయ్యేలా చేయడం (perfect types కాదు)
// - implicit any లు ఉండనివ్వండి (strict off)
// - తెలియని చీట్ `any` + `// TODO: type this`
// - పెద్ద refactors చేయొద్దు – కేవలం rename + minimum fixes
```

Phase 3 – strictness ని క్రమంగా పెంచడం

```
// ఒక్కొక్కటిగా ఆన్ చేయండి – ప్రతిదానికి errors fix చేసి commit
{ "noImplicitAny": true } // 1. అత్యధిక ప్రభావం
{ "strictNullChecks": true } // 2. ⭐ అత్యంత విలువైనది (కానీ errors ఎక్కువ)
{ "strictFunctionTypes": true } // 3.
{ "strictBindCallApply": true } // 4.
{ "strictPropertyInitialization": true } // 5.
{ "noImplicitThis": true } // 6.
{ "useUnknownInCatchVariables": true } // 7.
{ "strict": true } // 8. అన్నీ ✅
{ "noUncheckedIndexedAccess": true } // 9. bonus
```

Phase 4 – any debt ని తగ్గించడం

```
npx type-coverage --detail --at-least 90
# 92.4% (12043/13032) – ఏ lines untyped అని చూపిస్తుంది
```

```
// package.json – CI లో ratchet (తగ్గకుండా కాపాడటం)
"scripts": { "type-coverage": "type-coverage --at-least 92 --strict" }
```

ఉపయోగకరమైన techniques

```
// 1. JSDoc – .js files లోనే types (rename చేయకుండా) ⭐
/** @type {import("../types").User} */
const user = getUser();

/**
 * @param {string} name
 * @param {number} [age]
 * @returns {Promise<User>}
 */
async function createUser(name, age) {}

// checkJs: true పడితే ఇవి type-check అవుతాయి – file ని .ts చేయకుండానే!

// 2. తాత్కాలిక escape hatches (కనిపించేలా)
type TODO = any; // grep చేయొచ్చు ✅
const data: TODO = legacyCall();
```

```
// @ts-expect-error TODO(TS-migration): types needed for legacy module
import legacy from "./legacy";

// 3. Per-file opt-out (అత్యంత సమస్యాత్మక files కి)
// @ts-nocheck
// ⚠️ తాత్కాలికం మాత్రమే; ticket create చేయాలి

// 4. Automated conversion tools
// npx ts-migrate migrate <folder> (Airbnb tool – bulk rename + any annotations)
// npx typescript-eslint --fix
```

Team & process

- ✓ కొత్త code మొత్తం TypeScript లో (నియమం #1 – debt పెరగకూడదు)
- ✓ ప్రతి PR లో touch చేసిన file ని migrate చేయడం ("boy scout rule")
- ✓ CI లో: tsc --noEmit + type-coverage ratchet
- ✓ ప్రతి sprint కి ఒక migration target (metrics track చేయడం)
- ✓ Team కి TypeScript training (లేకపోతే `any` వరద)
- ✗ Feature freeze చేసి rewrite ప్రయత్నించడం
- ✗ Perfect types కోసం మొదటి pass లోనే ఆగిపోవడం

వాస్తవిక timeline (అనుభవం ఆధారంగా)

Codebase size	సుమారు సమయం (incremental, features నడుస్తూనే)
< 10k lines	1-2 వారాలు
50k lines	2-3 నెలలు
200k+ lines	6-12 నెలలు (multiple teams)

Gotchas

- strict: true ని మొదట్లోనే ఆన్ చేయడం → వేల errors → team demotivation → abandonment.
- Big bang rewrite – ఎప్పుడూ విఫలం.
- Migration సమయంలో refactor చేయడం – రెండు risks కలిపి; rename మాత్రమే చేయాలి.
- any ని permanent solution చేయడం – TODO + tracking లేకపోతే శాశ్వతం.
- @ts-ignore వాడటం – @ts-expect-error వాడాలి (fix అయ్యాక అదే చెప్తుంది).
- Test files ని migrate చేయకపోవడం – అవే bugs ని పట్టుకునేవి.
- Types ని .d.ts files లో మాత్రమే రాయడం – actual code తో drift.

Key Points

- Incremental – allowJs + leaf-first + strictness ratchet.
- Strict flags ఒక్కొక్కటిగా; strictNullChecks అత్యంత విలువైనది (మరియు అత్యంత శ్రమ).
- JSDoc = rename చేయకుండా typing (checkJs).
- type-coverage ratchet in CI = debt తగ్గుతూ ఉండేలా.

- కొత్త code ఎప్పుడూ TS లో.

Interview దృష్టి

- "50k line JS app ని ఎలా migrate చేస్తావు?" → phased plan + tooling + CI ratchet + team process — SSE-level answer.
- "any debt ని ఎలా నియంత్రిస్తావు?" → type-coverage, ESLint no-explicit-any, TODO alias, PR review.

37. Design Patterns in TypeScript

1. Discriminated Union State Machines ★

```
type Machine =
  | { state: "idle" }
  | { state: "loading"; startedAt: number }
  | { state: "success"; data: User[]; fetchedAt: number }
  | { state: "error"; error: Error; retryCount: number };

type Event =
  | { type: "FETCH" }
  | { type: "RESOLVE"; data: User[] }
  | { type: "REJECT"; error: Error }
  | { type: "RETRY" };

function transition(m: Machine, e: Event): Machine {
  switch (m.state) {
    case "idle":
      return e.type === "FETCH" ? { state: "loading", startedAt: Date.now() } : m;
    case "loading":
      if (e.type === "RESOLVE") return { state: "success", data: e.data, fetchedAt: Date.now() };
      if (e.type === "REJECT") return { state: "error", error: e.error, retryCount: 0 };
      return m;
    case "error":
      return e.type === "RETRY" ? { state: "loading", startedAt: Date.now() } : m;
    case "success":
      return e.type === "FETCH" ? { state: "loading", startedAt: Date.now() } : m;
    default: { const _: never = m; return m; }
  }
}
// ✅ impossible states impossible; అన్ని transitions explicit
```

2. Builder Pattern (fluent + type-safe)

```
class QueryBuilder<T, Selected = T> {
  private constructor(
    private table: string,
    private conditions: string[] = [],
    private fields?: (keyof T)[]
  ) {}
}
```

```

) {}

static from<T>(table: string) { return new QueryBuilder<T>(table); }

select<K extends keyof T>(...fields: K[]): QueryBuilder<T, Pick<T, K>> {
  return new QueryBuilder<T, Pick<T, K>>(this.table, this.conditions, fields as any);
}
where<K extends keyof T>(field: K, op: "=" | ">" | "<", value: T[K]): this {
  this.conditions.push(`${String(field)} ${op} ${JSON.stringify(value)}`);
  return this;
}
async execute(): Promise<Selected[]> { /* ... */ return [] as Selected[]; }
}

const users = await QueryBuilder.from<User>("users")
  .select("id", "email")           // ✅ return type ఇప్పుడు Pick<User, "id"|"email">
  .where("age", ">", 18)           // ✅ value type checked (number)
  .where("age", ">", "18")         // ❌ Error ⚡
  .execute();
// users: { id: string; email: string }[] ⭐

```

3. Dependency Injection (constructor-based, framework లేకుండా)

```

// Interfaces = contracts
interface Logger { info(msg: string, meta?: object): void; error(msg: string, e?: unknown): void }
interface Cache { get<T>(key: string): Promise<T | null>; set<T>(k: string, v: T, ttl?: number): Promise<void> }

class UserService {
  constructor(
    private readonly repo: UserRepository,
    private readonly cache: Cache,
    private readonly logger: Logger
  ) {}

  async getUser(id: UserId): Promise<User | null> {
    const cached = await this.cache.get<User>(`user:${id}`);
    if (cached) return cached;
    const user = await this.repo.findById(id);
    if (user) await this.cache.set(`user:${id}`, user, 300);
    return user;
  }
}

// ✅ Test: new UserService(fakeRepo, fakeCache, noopLogger) – mocking library అవసరం లేదు

// Typed container (సరళమైన DI)
type Services = { userService: UserService; logger: Logger; cache: Cache };
function createContainer(): Services { /* wire dependencies */ }

```

4. Factory + Registry (typed)

```
type PaymentMethod = "card" | "upi" | "netbanking";

interface PaymentProcessor { process(amount: number): Promise<Receipt> }

const processors = {
  card: () => new CardProcessor(),
  upi: () => new UpiProcessor(),
  netbanking: () => new NetBankingProcessor(),
} satisfies Record<PaymentMethod, () => PaymentProcessor>;
// ↑ satisfies - కొత్త PaymentMethod add చేస్తే ఇక్కడ compile error ⚡

function createProcessor(method: PaymentMethod): PaymentProcessor {
  return processors[method]();
}
```

5. Branded types + smart constructors (Topic 21)

```
type Email = string & { readonly __brand: "Email" };
type NonEmptyString = string & { readonly __brand: "NonEmpty" };

const Email = {
  parse(s: string): Email {
    if (!/^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(s)) throw new ValidationError("Invalid email");
    return s as Email;
  },
  tryParse(s: string): Email | null {
    try { return Email.parse(s); } catch { return null; }
  },
};
// ✅ Email type ఉన్న ప్రతి చోట అది validated అని హామీ
```

6. Repository + Unit of Work

```
interface Repository<T, ID = string> {
  findById(id: ID): Promise<T | null>;
  save(entity: T): Promise<T>;
  delete(id: ID): Promise<void>;
}

interface UnitOfWork {
  users: Repository<User>;
  orders: Repository<Order>;
  transaction<T>(fn: (uow: UnitOfWork) => Promise<T>): Promise<T>;
}

await uow.transaction(async (tx) => {
  const user = await tx.users.findById(id);
  await tx.orders.save(newOrder);
  // ✅ ఏదైనా throw అయితే మొత్తం rollback
});
```

7. Type-safe Event Bus (Topic 12 లో చూశాం) & Middleware pipeline

```
type Middleware<Ctx> = (ctx: Ctx, next: () => Promise<void>) => Promise<void>;

function compose<Ctx>(middlewares: Middleware<Ctx>[]) {
  return async (ctx: Ctx) => {
    let index = -1;
    const dispatch = async (i: number): Promise<void> => {
      if (i <= index) throw new Error("next() called multiple times");
      index = i;
      const fn = middlewares[i];
      if (fn) await fn(ctx, () => dispatch(i + 1));
    };
    await dispatch(0);
  };
}
```

8. Exhaustive mapping (**satisfies** + Record)

```
type Status = "draft" | "published" | "archived";

const statusConfig = {
  draft: { label: "Draft", color: "gray", canEdit: true },
  published: { label: "Published", color: "green", canEdit: false },
  archived: { label: "Archived", color: "red", canEdit: false },
} satisfies Record<Status, { label: string; color: string; canEdit: boolean }>;
// ✔ కొత్త Status add చేస్తే compile error – handle చేయడం మర్చిపోవడం అసాధ్యం ⚡
// ✔ satisfies వల్ల keys నిలుస్తాయి: keyof typeof statusConfig = "draft" | "published"
// (annotation `: Record<Status, ...>` ఇస్తే ఈ నిర్దిష్టత పోతుంది)
// ⚠ గమనిక: constraint `string` కాబట్టి color → string గా widen అవుతుంది;
// canEdit → `true`/`false` literal గానే నిలుస్తుంది (boolean constraint literal-friendly).
// అన్నీ literals కావాలంటే: `} as const satisfies Record<Status, {...}>`
```

Key Points

- **Discriminated unions** = state machines, impossible states నివారణ.
- **Builder** = fluent + type transformation (Pick chaining).
- **DI via constructor + interfaces** = testability (framework అవసరం లేదు).
- **satisfies Record<Union, T>** = exhaustive config maps ★.
- **Branded types + smart constructors** = "parse, don't validate".

Interview దృష్టి

- "State ని ఎలా model చేస్తావు?" → discriminated union state machine.
- "Testable code ఎలా రాస్తావు?" → interfaces + constructor DI + pure business logic.

38. Anti-Patterns & Best Practices Cheat-Sheet

Types

❌ Anti-pattern	✅ సరైనది
<code>any</code> ప్రతిచోటా	unknown + narrowing
<code>as</code> casting తో errors "fix" చేయడం	narrowing, type guards, <code>satisfies</code>
<code>as unknown as T</code> (double assertion)	సరైన types లేదా validation
<code>type UserId = string</code> (fake safety)	branded types
<code>{}</code> / <code>object</code> / <code>Object</code>	నిర్దిష్ట shape / <code>Record<string, unknown></code>
<code>Function</code> type	<code>(a: X) => Y</code> నిర్దిష్ట signature
Numeric enums	string literal union / <code>as const</code>
Optional flags తో state (<code>{loading?, error?}</code>)	discriminated union
<code>!</code> non-null assertion అలవాటు	guard + explicit error
Types + Zod schema రెండూ విడిగా	<code>z.infer</code>

Functions & APIs

❌	✅
Return-only generics (<code>parse<T>(s): T</code>)	unknown + validation
Public function కి return type లేకపోవడం	explicit return type
Boolean parameters (<code>f(true, false)</code>)	options object / union
5+ positional parameters	options object
Overloads అనవసరంగా	union / generic / conditional
Method shorthand callbacks (bivariant)	property syntax function types

Structure

✗	✓
<code>strict: false</code>	<code>strict: true</code> + <code>noUncheckedIndexedAccess</code>
CI లో typecheck లేకపోవడం	<code>tsc --noEmit</code>
Deep barrel files	direct imports / feature-root barrels
Default exports అన్నిచోట్లా	named exports
<code>import { Type } (value import)</code>	<code>import type { Type }</code>
Types ని duplicate చేయడం	<code>derive (Pick , Omit , z.infer , typeof)</code>
Global types అతిగా	module-scoped types
200-line type puzzles (app code)	సరళత; puzzles libraries కి

Runtime safety

✗	✓
<code>await res.json() as User</code>	<code>UserSchema.parse(await res.json())</code>
<code>process.env.PORT</code> నేరుగా	validated env object
Client validation మాత్రమే	server validation కూడా
<code>catch (e) { e.message }</code>	<code>e: unknown</code> + narrowing
Floating promises	<code>await</code> / <code>void</code> + ESLint rule
`	

Best practices summary (బంగారు నియమాలు)

1. `strict: true` – చర్చ లేదు
2. `any` బదులు `unknown`
3. Boundaries లో `validate` (Zod), లోపల `types` నమ్మండి
4. Impossible states ని అసాధ్యం చేయండి (discriminated unions)
5. `Types` ని `derive` చేయండి, `duplicate` చేయకండి
6. Inference ని నమ్మండి; `public` APIs కి `explicit`
7. `'satisfies'` = configs కి `default`
8. Branded `types` = IDs & validated `values`
9. `Type-aware` ESLint rules అన్ చేయండి
10. `Types` చదవడానికి 5 నిమిషాలు పడితే – అది తప్పు abstraction

39. Rapid-Fire Interview Q&A

A. Fundamentals

- 1. TypeScript అంటే?** JavaScript superset with compile-time static types; type erasure తో plain JS కి compile అవుతుంది.
- 2. Types runtime లో ఉంటాయా?** లేదు (type erasure). మినహాయింపులు: `enum` , `namespace` , legacy decorators + `emitDecoratorMetadata` , parameter properties (వీటికి code emit అవుతుంది).
- 3. any VS unknown VS never ?** `any` = checks off; `unknown` = top type, వాడేముందు narrow చేయాలి; `never` = bottom type, ఏ value లేదు.
- 4. type VS interface ?** `type` = unions/tuples/mapped/conditional; `interface` = declaration merging + extendable public APIs.
- 5. Structural typing అంటే?** Shape ఆధారంగా compatibility (name కాదు). ప్రమాదం: `type UserId = string` ఏ రక్షణ ఇవ్వదు → branded types.
- 6. TypeScript sound ఆ?** కాదు, ఉద్దేశపూర్వకంగా. ఉదాహరణలు: array covariance, `any` , type assertions, method bivarience, unvalidated I/O.
- 7. strict ఏం enable చేస్తుంది?** 8 flags — ముఖ్యంగా `noImplicitAny` , `strictNullChecks` , `strictFunctionTypes` , `useUnknownInCatchVariables` .
- 8. noUncheckedIndexedAccess ఎందుకు?** `arr[i]` / `record[k]` కి `| undefined` జోడిస్తుంది; `strict` లో లేదు కానీ నిజమైన bugs అవుతుంది.

B. Types & narrowing

- 9. Type guard ఎలా రాస్తావు?** `function isX(v: unknown): v is X { ... }` — కానీ compiler దాన్ని verify చేయదు (Zod మేలు).
- 10. Assertion function అంటే?** `function assert(v: unknown): asserts v is T` — throw చేసి narrow చేస్తుంది; explicit type annotation తప్పనిసరి.
- 11. Exhaustiveness ఎలా enforce చేస్తావు?** `default: const _: never = value` లేదా `switch-exhaustiveness-check` ESLint rule.
- 12. Discriminated union అంటే?** Common literal tag property + narrowing; impossible states ని అసాధ్యం చేస్తుంది.
- 13. Narrowing ఎప్పుడు పోతుంది?** Object properties మీద function call తర్వాత, `let` closures లో, reassignment తర్వాత → local `const` లోకి extract చేయాలి.
- 14. as const ఏం చేస్తుంది?** Literal types + deep readonly + tuple inference.
- 15. satisfies ఎందుకు?** Validation  + inference  రెండూ (annotation inference ని widen చేస్తుంది, `as` check చేయదు).

C. Generics & advanced

- 16. Generic constraint ఎందుకు?** T మీద operations చేయడానికి + inference నియంత్రణ.
- 17. keyof / typeof / indexed access?** keys union / value → type / property type. (typeof ARR)[number] = element union.
- 18. Conditional type distribution అంటే?** Naked T unions మీద member-wise apply అవుతుంది; [T] extends [U] అవుతుంది.
- 19. infer ఏం చేస్తుంది?** Conditional type లో pattern matching – భాగాన్ని type variable లోకి capture.
- 20. ReturnType<T> implement చెయ్యి?** T extends (...a: any) => infer R ? R : never .
- 21. Mapped type + key remapping?** { [K in keyof T as NewKey]: T[K] } ; never = key తొలగింపు.
- 22. Homomorphic mapped type అంటే?** [K in keyof T] రూపం – modifiers & array structure ని కాపాడుతుంది.
- 23. Template literal types ఎక్కడ ఉపయోగం?** Route params, CSS classes, event names, path types; ⚠ combinatorial explosion.
- 24. Omit లో లోపం?** Key typos పట్టుకోదు (keyof any) + unions మీద distribute కాదు.
- 25. TS2589 ఎందుకు, fix?** Recursion depth limit; tail-recursion, depth parameter, simplification.

D. Type system semantics

- 26. Covariance vs contravariance?** Return types covariant (safe), parameters contravariant (safe); arrays covariant (unsafe); method params bivariant (unsafe).
- 27. strictFunctionTypes methods కి వర్తిస్తుందా?** లేదు – method shorthand bivariant గానే ఉంటుంది (backward compat).
- 28. Excess property checking ఎప్పుడు?** Fresh object literals కి మాత్రమే; variable ద్వారా bypass.
- 29. Types ని sets గా వివరించు.** never = ∅, unknown = universe; assignability = subset; objects లో ఎక్కువ properties = narrower type.
- 30. Branded types ఎందుకు, ఎలా?** Nominal typing simulation; T & { __brand: "X" } + validating constructor; zero runtime cost.

E. Practice

- 31. API response ని ఎలా type-safe చేస్తావు?** Boundary లో Zod parse + z.infer ; as కాదు; contract violations ని log చేయడం.
- 32. Env vars?** Startup లో Zod schema parse → fail fast → typed env object.
- 33. catch లో type?** unknown (strict); instanceof narrowing + normalizer.

- 34. Result type vs exceptions?** Result = errors signature లో కనిపిస్తాయి, మర్చిపోలేం; exceptions = ergonomic. Hybrid: domain errors → Result, unexpected → throw.
- 35. Frontend/backend types sync?** tRPC (TS monorepo), OpenAPI codegen (polyglot), GraphQL codegen, shared Zod package.
- 36. import type ఎందుకు?** Erasure హామీ, circular deps నివారణ, transpiler (esbuild/swc) isolation.
- 37. Declaration merging ఎక్కడ ఉపయోగం?** Express Request augmentation, Window , ProcessEnv , library theme types.
- 38. .d.ts ఎలా రాస్తావు?** `declare module "lib" { ... } ;` global కి `declare global + export {}` .
- 39. Enum ఎందుకు నివారించాలి?** Runtime code emit, numeric fragility, reverse mappings, `const enum` tooling సమస్యలు → union/ `as const` .
- 40. private vs #private ?** Compile-time vs runtime enforcement; JSON leak; subclass collisions.

F. Tooling & scale

- 41. Vite build లో type errors ఎందుకు రావు?** esbuild transpile-only → `tsc --noEmit` విడిగా CI లో.
- 42. TS build నెమ్మది — ఏం చేస్తావు?** `--extendedDiagnostics / --generateTrace` → explicit return types, simplify types, `skipLibCheck` , project references, incremental.
- 43. Project references ఎందుకు?** Monorepo incremental builds + enforced boundaries + editor performance.
- 44. అత్యంత విలువైన ESLint rules?** `no-floating-promises` , `no-misused-promises` , `no-explicit-any` , `switch-exhaustiveness-check` , `prefer-nullish-coalescing` , `consistent-type-imports` .
- 45. @ts-ignore vs @ts-expect-error ?** `expect-error` = error రాకపోతే అదే error అవుతుంది (self-verifying) .
- 46. JS→TS migration plan?** `allowJs` → `leaf-first` → strict flags ఒక్కొక్కటిగా → `type-coverage` ratchet; big bang వద్దు.
- 47. Types ని test చేస్తావా?** `expectTypeOf` , `Expect<Equals<A,B>>` , `@ts-expect-error` — utility types కి తప్పనిసరి.
- 48. tsc pipeline?** Scanner → parser → binder → **checker** → emitter.

40. Memory Tips — ఒకసారి చదివి గుర్తుపెట్టుకునే Framework

1. ఒక్క వాక్యంలో TypeScript

"Types compile time లో ఒక ఒప్పందం; runtime లో ఏమీ లేదు."

దీని నుండి మిగతావన్నీ వస్తాయి – validation ఎందుకు అవసరం, enum ఎందుకు వింత, **as** ఎందుకు ప్రమాదకరం, bundlers ఎందుకు type check చేయవు.

2. మూడు worlds

```
VALUE world      : const x = 5; function f() {}  
TYPE world       : type X = number; interface I {}  
BRIDGE           : typeof (value→type), ReturnType/InstanceType, z.infer
```

3. Type = set (అన్ని assignability ప్రశ్నలకు సమాధానం)

```
never (∅) < "hello" < string < unknown (universe)  
{a, b} < {a} ← ఎక్కువ properties = చిన్న set  
S assignable to T ⇔ S ⊆ T
```

4. ప్రతి సమస్యకు 4 సాధనాలు

```
"ఇది ఏమిటో తెలియదు" → unknown + narrowing  
"ఇది A లేదా B" → discriminated union  
"ఇది ఏ type అయినా కావచ్చు" → generic (2 చోట్ల ఉండాలి)  
"బయటి నుండి వచ్చింది" → Zod parse
```

5. Advanced types నేర్చుకునే క్రమం (ఒకదాని మీద ఒకటి)

```
keyof / typeof / T[K] ← పునాది (ఇవి లేకుండా ముందుకు వెళ్ళలేం)  
↓  
Conditional types + infer ← if/else + pattern matching  
↓  
Mapped types + key remapping ← loop + rename  
↓  
Template literal types ← string manipulation  
↓  
Recursive types ← ఇవన్నీ కలిపి
```

6. Analogy map

Concept	Analogy
Type erasure	Building plan approval (కట్టేసాక engineer ఉండడు)
any vs unknown	VIP pass vs ID చూపించాలి
Structural typing	Skills interview (college కాదు)
Branded types	Roll number (ఒకే పేరు ఉన్నవాళ్ళను వేరు చేయడం)
Generics	ఒకే రకం పట్టే డబ్బా
Conditional types	"Veg అయితే పప్పు, కాకపోతే చికెన్"
Mapped types	జాబితాలో ప్రతి ఒక్కరికీ ఒకే మార్పు
Contravariance	"ఏ జంతువునైనా చూసుకుంటాను" > "కుక్కల్ని మాత్రమే"
Zod validation	గేటు దగ్గర security
satisfies	Validation + inference రెండూ (annotation ఒకటే ఇస్తుంది)
never exhaustiveness	"ఈ గది ఉనికిలో లేదు" – కొత్త case మర్చిపోలేం

7. Interview కి ముందు 30 నిమిషాల revision

1. Type erasure + validation boundary

(3 min)

2. any/unknown/never + assignability sets

(4 min)

3. Narrowing: guards, predicates, exhaustiveness

(4 min)

4. Generics golden rule + keyof/typeof/T[K]

(4 min)

5. Conditional + infer + distribution

(4 min)

6. Mapped types + key remapping

(3 min)

7. Structural typing + branded types

(3 min)

8. Variance (arrays unsound, method bivarience)

(3 min)

9. strict flags + tsconfig essentials

(2 min)

8. చివరి మాట

TypeScript లో syntax నేర్చుకోవడం ఒక వారం; ఆలోచనా విధానం నేర్చుకోవడం ఒక సంవత్సరం.

తేడా ఇది: junior engineer "ఈ error ఎలా పోగొట్టాలి?" అని అడుగుతాడు (సమాధానం: as any). Senior engineer "ఈ error నాకు ఏం చెప్తోంది?" అని అడుగుతాడు – తరచుగా అది నిజమైన bug గురించి చెప్తోంది.

SDE2 అంటే – types సరిగ్గా రాసి, compile అయ్యేలా చేయగలగడం. SSE అంటే – type system ని ఒక design tool గా వాడటం: impossible states ని అసాధ్యం చేయడం, boundaries ని validate చేయడం, team కి maintain చేయగలిగే abstractions ఇవ్వడం, ఎప్పుడు type gymnastics వద్దు అని చెప్పగలగడం.

గుర్తుంచుకోండి: ఉత్తమమైన type అనేది అత్యంత తెలివైనది కాదు – తప్పు code ని రాయడం అసాధ్యం చేసేది, మరియు మీ team చదవగలిగేది.

అనుబంధం – వేగవంతమైన Reference

Utility types

```
Partial<T> Required<T> Readonly<T> Pick<T,K> Omit<T,K> Record<K,V>
Exclude<T,U> Extract<T,U> NonNullable<T> NoInfer<T>
Parameters<F> ReturnType<F> ConstructorParameters<C> InstanceType<C> Awaited<P>
ThisParameterType<F> OmitThisParameter<F> ThisType<T>
Uppercase<S> Lowercase<S> Capitalize<S> Uncapitalize<S>
```

Custom utilities (copy-paste ready)

```
type Prettyify<T> = { [K in keyof T]: T[K] } & {};
type DeepPartial<T> = T extends object ? { [K in keyof T]?: DeepPartial<T[K]> } : T;
type PartialBy<T, K extends keyof T> = Omit<T, K> & Partial<Pick<T, K>>;
type RequiredBy<T, K extends keyof T> = Omit<T, K> & Required<Pick<T, K>>;
type ValueOf<T> = T[keyof T];
type ArrayElement<T> = T extends readonly (infer E)[] ? E : never;
type Mutable<T> = { -readonly [K in keyof T]: T[K] };
type NonEmptyArray<T> = [T, ...T[]];
type Brand<T, B> = T & { readonly __brand: B };
type Result<T, E = Error> = { ok: true; value: T } | { ok: false; error: E };
type Equals<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ?
true : false;
type Expect<T extends true> = T;
```

tsconfig baseline

```
{
  "compilerOptions": {
    "target": "ES2022", "lib": ["ES2022", "DOM"],
    "module": "ESNext", "moduleResolution": "bundler",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noImplicitOverride": true,
    "noFallthroughCasesInSwitch": true,
    "exactOptionalPropertyTypes": true,
    "isolatedModules": true, "verbatimModuleSyntax": true,
    "skipLibCheck": true, "esModuleInterop": true,
    "resolveJsonModule": true, "forceConsistentCasingInFileNames": true,
    "noEmit": true
  },
  "include": ["src"]
}
```

Production checklist

- `strict: true` + `noUncheckedIndexedAccess`
- CI లో `tsc --noEmit` (*bundler build కి విడిగా*)
- `typescript-eslint` `strictTypeChecked` + `no-floating-promises`
- అన్ని I/O boundaries లో Zod validation (API, env, `storage`, forms, URL)
- `any usage` tracked (`type-coverage` ratchet in CI)
- `Public API functions` కి explicit `return types`
- `import type` ప్రతిచోటా (`consistent-type-imports`)
- Discriminated unions `for` state (`boolean` flags కాదు)
- Branded `types` `for` IDs & validated `values`
- Utility `types` కి `type` tests (`expectTypeOf`)
- Libraries: declaration + declarationMap + `exports.types` first + `are-the-types-wrong`
- Monorepo: project `references` + incremental + build caching

ఈ guide పూర్తి చేసినందుకు అభినందనలు! TypeScript ని నిజంగా నేర్చుకోవడం అంటే — ఒక real project లో `strict: true` పెట్టి, `any` లేకుండా ఒక feature పూర్తి చేయడం. Boundary లో Zod, state కి discriminated union, IDs కి branded types — ఈ మూడు అలవాట్లైతే మీ code లో runtime type bugs దాదాపు అదృశ్యమవుతాయి.

Companion docs: [JavaScript_Telugu.md](#) · [React_Telugu.md](#) · [OOPS_Telugu.md](#) · [HLD_Telugu.md](#) · [LLD_Telugu.md](#) · [SystemDesign_Go_Telugu.md](#) · [Security_Telugu.md](#) · [SoftwareEngineering_Telugu.md](#) · [DSA_00..10_Telugu.md](#)